

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Fachhochschule Aachen, Campus Jülich

Fachbereich 09 - Medizintechnik und Technomathematik
Studiengang Angewandte Mathematik und Informatik

Bachelorarbeit

Implementierung und Evaluation KI-gestützter Code Review in der Softwareentwicklung mit GitLab

---

Autor:
Betreuer:

TODO: fh logo entfernen bei abgabe

Aachen, 13. August 2026

Zusammenfassung

Code-Reviews erfordern ein tiefes Verständnis der Softwarearchitektur, welches bei einer reinen Betrachtung von Code-Änderungen meist fehlt. Bisherige KI-Werkzeuge arbeiten statisch und ignorieren oft wichtige Architekturentscheidungen, was in der Praxis zu zahlreichen Fehlern und Fehldeutungen führt.
Diese Bachelorarbeit entwickelt und evaluiert einen neuen, KI-basierten Code-Review-Agenten für GitLab, der dokumentierte Architekturentscheidungen gezielt einbezieht. Dabei wird ein gängiges, starres Verfahren einem agentischen Ansatz gegenübergestellt. Während die starre Methode auf vorher festgelegte Code-Ausschnitte zurückgreift, durchsucht der Agent den Quellcode eigenständig mit spezifischen Werkzeugen.
Die empirische Auswertung zeigt einen deutlichen Leistungsunterschied. Im Mittel über drei Läufe erreicht der Agent eine Genauigkeit von 75,6 Prozent, während die starre Baseline bei 55,9 Prozent liegt. Dieser statistisch abgesicherte Vorsprung zeigt sich konsistent über alle drei untersuchten Sprachmodelle hinweg. Bei der Erkennung echter Regelverstöße erweist sich allerdings mehr als die Hälfte der Meldungen als Fehlalarm.
Weiterführende Experimente untersuchen die Ursachen dieses Vorsprungs und liefern unerwartete Ergebnisse. Der bloße Zugang zu Werkzeugen reicht demnach nicht aus. Erst das Zusammenspiel aus drei Faktoren erzeugt den Leistungsvorsprung: eine Arbeitsanweisung, die fehlende Belege explizit zulässt, eine funktionale Trennung von Such- und Lesevorgängen sowie ein ausreichend großes Suchbudget. Fehlt auch nur eine dieser Bedingungen, schrumpft der Vorteil drastisch oder entfällt gänzlich. Zudem zeigt sich, dass ein erheblicher Teil des Leistungsabstands darauf zurückzuführen ist, dass der Agent insgesamt mehr Code einsehen kann.
Als praktisches Ergebnis dieser Arbeit entstand ein funktionsfähiger Prototyp, der den Agenten direkt in den GitLab-Workflow integriert und sich konsequent an den ermittelten Designbedingungen orientiert. Die Arbeit belegt damit, dass ein autonom agierender Agent automatisierte Code-Reviews maßgeblich verlässlicher machen kann. Sie verdeutlicht zugleich, dass ein solcher Erfolg kein Selbstläufer ist, sondern einen sorgfältig abgestimmten Entwurf erfordert.

Eigenständigkeitserklärung

Hiermit erkläre ich verbindlich, dass ich die vorliegende Bachelorarbeit selbstständig und unter Beachtung der geltenden Prüfungsordnung verfasst habe. Alle von mir verwendeten Hilfsmittel und Quellen sind vollständig und korrekt in der Arbeit angegeben. Ich versichere, dass der eingereichte Ausdruck inhaltlich mit der von mir erstellten digitalen Version übereinstimmt. Die Arbeit wurde weder ganz noch in Teilen bereits als Prüfungsleistung eingereicht. Mir ist bewusst, dass Verstöße gegen diese Erklärung als Täuschungsversuch gewertet werden und zum Ausschluss der Anerkennung der Seminararbeit als Prüfungsleistung führen.

Aachen, 13. August 2026

(Ort, Datum) (Unterschrift)

Abbildungs- und Tabellenverzeichnis

2.1. Kontextlücke im Modern Code Review 5
2.2. Ablauf des ReAct-Musters 7
2.3. Positionseffekt im Kontextfenster 10
2.4. Architekturerosion und Architekturdrift 11
2.5. Aufbau eines ADR und Rolle der Abschnitte 13

3.1. Einbindung projektspezifischen Wissens in den Review-Prozess. Gegenüberstellung bestehender Werkzeuge (links) und des in dieser Arbeit verfolgten Ansatzes (rechts). 23
3.2. Einordnung der besprochenen Arbeiten nach Art der Evidenzbeschaffung und Art der Kontextquelle. Der Quadrant oben rechts ist bislang unbesetzt und markiert die Forschungslücke. 26

5.1. Ablauf der statischen RAG-Pipeline 36
5.2. Ablauf der agentischen Pipeline 37
5.3. Gegenüberstellung der beiden Verfahren 38

6.1. Genauigkeit der verglichenen Verfahren 49
6.2. Werkzeugnutzung nach gewähltem Label 53
6.3. Urteilsprofil der Verfahren 59
6.4. Genauigkeit und CIA-Urteile über die vier Budgetstufen 63
6.5. Beide Verfahren über beide Modelle 66

7.1. Ablauf eines Reviews im Prototyp 76

2.1. Statische und agentische Evidenzbeschaffung im Vergleich. 9
2.2. Konfusionsmatrix für eine Klasse in der Eins-gegen-Rest-Betrachtung. . . . 16
2.3. Schema der gepaarten Vierfeldertafel für den McNemar-Test. 18

3.1. KI-gestützte Code-Review-Werkzeuge und ihre Mechanismen zur Einbindung projektspezifischen Wissens. 22

5.1. Label-Semantik der unabhängigen Referenz. 40
5.2. Basiskonfiguration der Vergleichsläufe 41

5.3. Aufrufparameter beider Verfahren 42

6.1. Überblick über die Experimentreihe 46
6.2. Genauigkeit gegen die unabhängige Referenz 48
6.3. Makro-gemittelte Kennzahlen der drei eigenen Verfahren gegen die unabhängige Referenz, jeweils Lauf 1. Gemittelt wird über die drei Klassen, nicht über Wiederholungsläufe. 49
6.4. Gepaarte Vierfeldertafel beider Verfahren gegen die unabhängige Referenz ($n = 59$). 49
6.5. Signifikanzprüfung des Hauptvergleichs 50
6.6. Wiederholungsläufe bei identischer Konfiguration. 50
6.7. Urteilswechsel zwischen den Wiederholungsläufen, summiert über alle drei Laufpaare. 51
6.8. Mittlere Werkzeugnutzung des agentischen Judge, gruppiert nach dem gewählten Label. 52
6.9. Mittlere Werkzeugnutzung des agentischen Judge, gruppiert nach Korrektheit des Urteils. 53
6.10. Streitfälle beider Judges, nach Richtung getrennt, mit der jeweiligen Evidenzlage. 54
6.11. Suchaufwand nach Fallschwierigkeit, gemittelt über die drei agentischen Wiederholungsläufe. 55
6.12. Verfahren und Prompt im Zweifaktorenfeld 57
6.13. Statisch, Original-Prompt (Accuracy 55,9 %, $n = 59$). 58
6.14. Statisch, permissiver Prompt (Accuracy 65,0 % in Lauf 1, $n = 60$). 58
6.15. Agentisch (Accuracy 76,7 %, $n = 60$). 58
6.16. Agentisch, restriktiver Prompt (Accuracy 58,3 %, $n = 60$). 58
6.17. Rollentrennung gegen Ein-Agenten-Fassung 60
6.18. Werkzeugnutzung mit und ohne Rollentrennung 61
6.19. Ein Agent, keine Rollentrennung (Accuracy 68,3 % in Lauf 1, $n = 60$). . . . 61
6.20. Budgetstufen der Ablation E4. Die Standardstufe entspricht der Basiskonfiguration und damit dem agentischen Lauf aus E1. 62
6.21. Genauigkeit und Werkzeugnutzung über die Budgetstufen 63
6.22. Modellstufen der Rotation E5. Die Basisstufe entspricht der Basiskonfiguration und damit den Läufen aus E1. 65
6.23. Beide Verfahren über drei Modelle 66
6.24. Kontextstufen der Ablation E6 68
6.25. Genauigkeit über die vier Kontextstufen 69
6.26. Stand des Ausschlussverfahrens 70
6.27. Beitrag der Experimente und Auswertungen zu den drei Forschungsfragen. 71

7.1. Messbefunde aus Kapitel 6 und die daraus abgeleiteten Entscheidungen für den Prototyp. 73
7.2. Komponenten der Werkzeugkette des Proof-of-Concepts. 75

A.1. Kennzahlen zur Abhängigkeit von y_{true} von den bewerteten Modellausgaben. 100
A.2. Ursachen der *CIA*-Vergabe in der unabhängigen Referenz. 101
A.3. Suchproduktivität und abgewiesene Zugriffe aus den Event-Logs beider agentischer Läufe. 102
A.4. Aufwand je geprüfter Regel, gemittelt über die 81 annotierten Fälle. 102

A.5. Genauigkeit über alle Fälle des Zufallsteils gegenüber der Teilmenge, deren Label mit hoher Konfidenz vergeben wurde. 103
A.6. Gepaarter Vergleich über die Auswertungsbasen 103
A.7. Statisch, `gpt-5.4-nano` (Accuracy 42,4 %, $n = 59$). 103
A.8. Agentisch, `gpt-5.4-nano` (Accuracy 68,3 %, $n = 60$). 103
A.9. Mittlere Werkzeugnutzung des agentischen Judge mit `gpt-5.4-nano`, gruppiert nach gewähltem Label. 104
A.10. Statisch, `gpt-5.6-luna` (Accuracy 48,3 %, $n = 60$). 104
A.11. Agentisch, `gpt-5.6-luna` (Accuracy 75,0 %, $n = 60$). 104
A.12. Abstand zwischen makro-gemitteltem und gewichtetem F_1-Wert. Ein kleiner Abstand bedeutet, dass die Leistung nicht überwiegend von der häufigsten Klasse getragen wird. 105

Inhaltsverzeichnis

1. Einleitung **1**
- 1.1. Motivation . . . 1
- 1.2. Problemstellung . . . 1
- 1.3. Zielsetzung und Forschungsfragen . . . 3

2. Grundlagen **4**
- 2.1. Code Reviews im Softwareentwicklungsprozess . . . 4
- 2.2. Large Language Models und agentische Systeme . . . 6
- 2.3. Softwarearchitektur und Architekturkonformität . . . 11
- 2.4. Architecture Decision Records (ADRs) als Kontextquelle . . . 12
- 2.5. Das Ökosystem von GitLab . . . 14
- 2.6. Metriken zur Bewertung von Klassifikationsleistung . . . 15

3. Literaturrecherche **20**
- 3.1. Übersicht aktueller KI-gestützter Code-Review-Tools . . . 20
- 3.2. Aktueller Stand der Forschung im Bereich agentischer Code-Reviews . . . 24

4. Anforderungsanalyse **28**
- 4.1. Anforderungen an das System . . . 28
  - 4.1.1. Funktionale Anforderungen . . . 28
  - 4.1.3. Nicht-funktionale Anforderungen . . . 30
  - 4.1.4. Bewusst nicht aufgenommene Anforderungen . . . 31
- 4.2. Auswahl der Evaluationsmetriken . . . 32

5. Versuchsaufbau **35**
- 5.1. Die beiden verglichenen Verfahren . . . 35
  - 5.1.1. Baseline: der statische RAG-Judge . . . 35
  - 5.1.2. Der agentische Judge . . . 36
  - 5.1.3. Gemeinsame Suchraumbeschränkung . . . 37
  - 5.1.4. Gemeinsames Urteilsformat . . . 38
- 5.2. Datengrundlage . . . 39
  - 5.2.1. Testfälle und Repositories . . . 39
  - 5.2.2. Modellauswahl . . . 39
- 5.3. Die Referenz . . . 39
  - 5.3.1. Erstellung der unabhängigen Referenz . . . 39
  - 5.3.2. Grenzen einer Einzelannotation . . . 40
- 5.4. Basiskonfiguration der Vergleichsläufe . . . 41
- 5.5. Erhebung und Auswertung der Metriken . . . 42
  - 5.5.1. Berechnung der Klassifikationsmetriken . . . 42
- 5.6. Prinzip der Experimentreihe . . . 43

6. Experimente und Ergebnisse **45**
6.1. Vorbemerkung zu den Bezugsgrößen . . . 46
6.2. E1 – Agentische statt statischer Evidenzbeschaffung . . . 47
6.2.1. Aufbau des Experiments . . . 47
6.2.2. Ergebnis des Hauptvergleichs . . . 48
6.2.3. Absicherung gegen die Fallauswahl . . . 49
6.2.4. Kontrolllauf: Streuung bei reiner Wiederholung . . . 50
6.3. Auswertung der Werkzeugprotokolle des Hauptlaufs . . . 52
6.3.1. Werkzeugnutzung je nach gewähltem Urteil . . . 52
6.3.2. Zusammenhang zwischen Suche und Korrektheit . . . 53
6.3.3. Analyse der abweichenden Urteile . . . 54
6.3.4. Suchfehler oder Urteilsfehler . . . 55
6.4. E2 – Variation des Prompts . . . 56
6.4.1. Aufbau des Experiments . . . 56
6.4.2. Ergebnis . . . 57
6.4.3. Wohin die Fehler wandern . . . 58
6.5. E3 – Variation der agentischen Architektur . . . 60
6.5.1. Aufbau des Experiments . . . 60
6.5.2. Ergebnis . . . 60
6.6. E4 – Variation des Suchbudgets . . . 61
6.6.1. Aufbau des Experiments . . . 62
6.6.2. Ergebnis . . . 63
6.7. E5 – Variation des Modells . . . 65
6.7.1. Aufbau des Experiments . . . 65
6.7.2. Ergebnis . . . 66
6.8. E6 – Variation der Kontextmenge der Baseline . . . 67
6.8.1. Aufbau des Experiments . . . 68
6.8.2. Ergebnis . . . 68
6.9. Zusammenführung: was von den Erklärungen übrig bleibt . . . 69
6.10. Beantwortung der Forschungsfragen . . . 71

7. Konzeption und technische Umsetzung **73**
7.1. Auswahl der Werkzeugkette für den Proof-of-Concept . . . 74
7.2. Gesamtkonzept und Ablauf eines Reviews . . . 75
7.3. Rückmeldung im Merge Request . . . 80
7.4. Grenzen der Regelaufbereitung . . . 80
7.5. Demonstration des Prototyps . . . 81

8. Diskussion **82**
8.1. Einordnung der Ergebnisse . . . 82
8.2. Folgen für den Prototyp . . . 84
8.3. Einschränkungen . . . 85
8.4. Methodische Lehren für vergleichende Evaluationen . . . 87

9. Fazit und Ausblick **88**
9.1. Fazit . . . 88
9.2. Ausblick . . . 89

10. Literaturverzeichnis **92**

A. Anhang **98**
A.1. Beispiel eines Architecture Decision Record 98
A.2. Details der Reproduktion der Originalpipeline 99
A.2.1. Das Original-Judge-Modell 99
A.2.2. Der berichtete Baseline-Wert ist verifizierter Originaloutput 99
A.2.3. Warum y_{true} von den bewerteten Modellen abhängt 99
A.3. Ergänzende Auswertungen zum Hauptvergleich 101
A.3.1. Was die unabhängige Referenz enthält 101
A.3.2. Zusammensetzung der *CIA*-Klasse 101
A.3.3. Suchproduktivität der agentischen Läufe 102
A.3.4. Aufwand je geprüfter Regel 102
A.3.5. Robustheit gegen die Wahl der Auswertungsbasis 102
A.3.6. Konfusionsmatrizen der weiteren Modellstufen 103
A.3.7. Abstand zwischen makro-gemitteltem und gewichtetem F_1 105

Glossar

Dieses Glossar fasst die wiederkehrenden Fachbegriffe in je ein bis zwei Sätzen zusammen. Ausführlich eingeordnet werden sie in Kapitel 2, der konkrete Aufbau der untersuchten Verfahren steht in Kapitel 5.

Agent Ein Sprachmodell, das nicht einmalig antwortet, sondern in einer Schleife arbeitet und dabei selbst entscheidet, welche Werkzeuge es aufruft, bis es genug Information für seine Antwort hat (Abschnitt 2.2).

Architecture Decision Record (ADR) Ein kurzes Dokument, das eine einzelne Architekturentscheidung samt Kontext, Begründung und Konsequenzen festhält und im Repository neben dem Quellcode versioniert wird (Abschnitt 2.4).

Chunk Ein Textausschnitt fester Länge, in den der Quellcode für die Ähnlichkeitssuche vorab zerlegt wird. In dieser Arbeit sind das 800 Zeichen mit 200 Zeichen Überlappung.

Diff Der zeilengenaue Unterschied zwischen zwei Codeständen, also genau der Änderungssatz, den ein Review begutachtet.

Embedding Die Darstellung eines Textstücks als Zahlenvektor. Inhaltlich ähnliche Texte liegen in diesem Vektorraum nah beieinander, wodurch sich Ähnlichkeit maschinell berechnen lässt.

Fall Die Analyseeinheit dieser Arbeit, bestehend aus einer einzelnen Architekturregel und dem Codestand eines Repositories, gegen den sie geprüft wird. Ein Urteil bezieht sich immer auf einen Fall, nie auf ein ganzes ADR.

Ground Truth Der Referenzdatensatz, gegen den die Urteile der Verfahren gemessen werden. Er gilt als korrekte Antwort und bestimmt damit, was überhaupt als Fehler zählt (Abschnitt 2.6).

Halluzination Eine Modellausgabe, die sprachlich schlüssig wirkt, im geprüften Material aber keine Grundlage hat, etwa ein gemeldeter Regelverstoß ohne passende Codestelle.

Judge Die Instanz, die für einen Fall das abschließende Urteil fällt. Unterschieden werden der statische RAG-Judge, der vorab ausgewählte Ausschnitte erhält, und der agentische Judge, der seine Evidenz selbst sucht (Abschnitt 5.1).

Kontextfenster Die Textmenge, die ein Modell bei einem Aufruf verarbeiten kann. Alles, was nicht hineinpasst, muss vorher ausgewählt oder nachträglich nachgeschlagen werden.

Large Language Model (LLM) Ein auf großen Textmengen trainiertes neuronales Netz, das eine Eingabe fortsetzt, indem es schrittweise das jeweils wahrscheinlichste nächste Token vorhersagt (Abschnitt 2.2).

Lauf Die vollständige Ausführung einer Konfiguration über die gesamte Fallmenge. Weil Modelle nicht deterministisch antworten, wird jede Konfiguration dreimal ausgeführt und über die Läufe gemittelt.

Locator Im agentischen Verfahren die Rolle, die den Quellbaum durchsucht und Kandidatenstellen meldet, ohne selbst zu urteilen (Abschnitt 5.1.2).

Merge Request (MR) Der Antrag, einen Änderungssatz in einen Zielbranch zu übernehmen. Er bündelt Diff, Pipeline-Ergebnisse und Diskussion und ist der Ort, an dem das Review stattfindet (Abschnitt 2.5).

Prompt Die Texteingabe, mit der ein Modell gesteuert wird. Üblich ist die Trennung in einen System-Prompt mit Rolle und festen Regeln und die eigentliche Aufgabenstellung.

Reader Im agentischen Verfahren die Rolle, die die vom Locator gemeldeten Stellen liest und daraus das Urteil ableitet (Abschnitt 5.1.2).

Retrieval-Augmented Generation (RAG) Ein Verfahren, das vor dem Modellaufruf passende Textausschnitte aus einer Sammlung heraussucht und dem Prompt beilegt. Der Kontext steht damit fest, bevor das Modell zu arbeiten beginnt (Abschnitt 2.2.4).

Signal-Rausch-Verhältnis (Signal to Noise Ratio, SNR) Verhältnis von nützlichen, umsetzbaren Review-Kommentaren (Signal) zu irrelevanten, falsch positiven oder halluzinierten Anmerkungen (Rauschen). Ein hohes Verhältnis bedeutet, dass Entwickler wenig Aufwand mit dem Aussortieren unbrauchbarer Hinweise haben. Ein niedriges Verhältnis senkt die Akzeptanz automatisierter Code-Reviews, da nützliche Befunde im Rauschen untergehen.

Temperatur Ein Aufrufparameter, der steuert, wie stark ein Modell von der jeweils wahrscheinlichsten Fortsetzung abweicht. Niedrige Werte machen die Ausgabe gleichförmiger, beseitigen die Streuung zwischen Läufen aber nicht.

Token Die kleinste Verarbeitungseinheit eines Sprachmodells, ein Wort oder Wortteil. Kontextfenster und Kosten werden in Token gerechnet, nicht in Zeichen.

Werkzeugaufruf (Tool Call) Statt Fließtext erzeugt das Modell ein strukturiertes Datenobjekt, das eine Funktion und deren Parameter benennt, etwa eine Suche im Quellbaum. Ausgeführt wird der Aufruf nicht vom Modell selbst, sondern von der umgebenden Software, die das Ergebnis anschließend zurückgibt (Abschnitt 2.2).

1. Einleitung

1.1. Motivation

Code-Reviews sind heute ein unverzichtbarer Bestandteil der Softwareentwicklung, binden jedoch enorme zeitliche und kognitive Ressourcen [43, 34]. Die größte Hürde für Prüfende ist dabei meist nicht das bloße Lesen der geänderten Codezeilen, sondern das Beschaffen des nötigen Kontextes: Wer fremden Code begutachten soll, wird aus dem eigenen Arbeitsfluss gerissen und muss sich mühsam in die Hintergründe der Änderung einarbeiten [4, 2]. Sadowski et al. beschreiben, dass Unternehmen wie Google massiv in Werkzeuge und Prozesse investieren müssen, um diesen Vorgang bei wachsenden Codebasen überhaupt handhabbar zu halten [44].
Um diese kognitive Last zu reduzieren, setzen Entwickler zunehmend auf die Unterstützung durch Large Language Models (LLMs) [38]. In modernen DevOps-Plattformen wie GitLab, in denen der Merge Request der zentrale Ort für kollaborative Code-Änderungen ist [57, 36], sind solche KI-Assistenten mittlerweile direkt integriert. Dabei stehen sich aktuell zwei technische Ansätze gegenüber: Das klassische, nicht-agentische GitLab Duo Code-Review bewertet den Code auf Basis starrer, vorab definierter Informationen wie dem reinen Datei-Diff [14]. Der neuere, agentische Code-Review Flow geht hingegen einen Schritt weiter und sucht sich fehlende Hintergrundinformationen (wie verwandte Dateien, Tests oder Abhängigkeiten) selbstständig aus dem Repository zusammen [9].
Die empirische Praxis deckt jedoch deutliche Limitationen auf. Groß angelegte Studien belegen, dass maschinelle Code-Reviews oft inhaltlich falsch sind, zu viele Fehlalarme (Rauschen) produzieren und von Entwicklern deutlich seltener übernommen werden als menschliches Feedback [7, 63]. Der aktuelle Stand der Forschung verdeutlicht, dass KI-Modelle den Menschen beim Review derzeit zwar unterstützen, aber keinesfalls ersetzen können (detaillierte Kennzahlen hierzu liefert Kapitel 3).
Vor diesem Hintergrund untersucht die vorliegende Arbeit die Diskrepanz zwischen dem theoretischen Potenzial aktueller KI-Werkzeuge und ihrer nachgewiesenen Fehleranfälligkeit, um die Leistungsfähigkeit und die Grenzen statischer sowie agentischer Kontextbeschaffung systematisch zu bewerten.

1.2. Problemstellung

Aus der beschriebenen Ausgangslage leiten sich drei Kernprobleme ab, die den Ausgangspunkt für diese Arbeit bilden.

Fehlende Kontextsensitivität aktueller Lösungen Menschliche Prüfer nutzen beim Code-Review intuitiv ihr gesamtes Hintergrundwissen über die Projektarchitektur [4]. Genau dieses essenzielle Kontextwissen fehlt den aktuellen KI-Werkzeugen. Dieses Entscheidungswissen geht im Entwicklungsalltag schnell verloren, wenn es nicht explizit dokumentiert wird [18, 53]. Zwar bieten *Architecture Decision Records* (ADRs) hierfür eine

standardisierte, versionierte Lösung [37, 3], doch als strukturierte Wissensquelle bindet bislang kein gängiges KI-Review-Tool sie ein [14, 1, 10].
Aktuelle Systeme erlauben zwar benutzerdefinierte Prüfregeln, erwarten dafür aber meist unstrukturierte Freitext-Prompts [14, 1, 10]. Eine systematische Anbindung an die tatsächliche Architekturdokumentation fehlt komplett. Das Werkzeug weiß im Moment der Prüfung also nicht, welche Architekturentscheidung gerade gültig, abgelöst oder für den konkreten Code-Diff überhaupt relevant ist.
Da Sprachmodelle durch irrelevante Zusatzinformationen jedoch messbar an Präzision verlieren [47], reicht es auch nicht aus, der KI einfach alle Projektdokumente unstrukturiert zu übergeben [32, 49]. Ein Ansatz, der solch strukturiertes Architekturwissen zielgerichtet und kontextsensitiv als Informationsquelle für maschinelle Code-Reviews erschließt, fehlt bislang und markiert genau die Forschungslücke dieser Arbeit.

Unzureichende Qualität und mangelnde Vertrauenswürdigkeit Aktuelle KI-Agenten erzeugen im Code-Review oft noch Kommentare ohne echten Mehrwert. Wie die schwachen Übernahmequoten aus Abschnitt 1.1 bereits angedeutet haben, scheitern die Modelle in der Praxis häufig an inhaltlichen Fehlern [7, 63].
Das größte Problem für die Entwickler sind dabei unbegründete Fehlalarme (False Positives). Aus der Forschung zu klassischen statischen Analysewerkzeugen ist schon lange bekannt: Wenn ein Tool zu oft fälschlicherweise warnt, wird es vom Team irgendwann schlichtweg ignoriert [20]. Genau diese Skepsis überträgt sich nun auf LLMs. Mangelndes Vertrauen und zu viele Fehlalarme sind für Entwickler die zentralen Vorbehalte gegen KI-gestützte Reviews [2].
Das Ausmaß dieser Vorbehalte verdeutlicht der Stack Overflow Developer Survey 2025: Kaum 3 % der Befragten vertrauen den Ausgaben solcher Werkzeuge in hohem Maße, während fast die Hälfte (rund 46 %) ihnen explizit misstraut [38]. Die Konsequenz für die Praxis ist eindeutig: Liefern die Modelle zu oft ungenaues Feedback, geht der Mehrwert verloren, da das Team anfängt, die maschinellen Hinweise routinemäßig zu übergehen.

Fehlende systematische Evaluationsmethodik In einer umfassenden Literaturstudie bemängeln Khan et al., dass bestehende Benchmarks für Code-Reviews stark zersplittert sind und KI-Modelle fast ausschließlich anhand rein statischer Datensätze bewerten [22]. Die Forschung fordert daher dringend Evaluationen zur Laufzeit. Analysen in echten, produktiven Umgebungen bilden bisher jedoch die absolute Ausnahme und stammen zudem oft von den Werkzeugherstellern selbst, was ihre Aussagekraft einschränkt [52, 50].
Vor allem aber fehlt in all diesen Evaluationen ein entscheidender Baustein: der reale Architekturkontext. Es gibt bislang keine belastbaren Daten darüber, wie sich ein maschineller Prüfprozess verhält, wenn man ihn in einer echten DevOps-Plattform wie GitLab mit strukturiertem Architekturwissen versorgt. Es ist schlichtweg noch ungeklärt, ob sich die Signalqualität und Zuverlässigkeit der Modelle dadurch wirklich spürbar steigern lässt.
Aus diesem Spannungsfeld ergibt sich das Kernziel der vorliegenden Arbeit: In der Praxis besteht ein hoher Bedarf an skalierbaren Code-Review-Verfahren. Aktuelle KI-Tools scheitern jedoch oft an mangelndem Kontext, und der Forschung fehlt eine saubere Evaluationsgrundlage.
Um diese Lücke zu schließen, entwickelt diese Arbeit ein Konzept, das maschinelle Reviews gezielt mit explizitem Architekturwissen (ADRs) anreichert. Dieser architekturzentrierte Ansatz wird als funktionierender Agent in den realen Entwicklungsprozess von GitLab integriert. In einer systematischen, methodisch fundierten Evaluation wird anschließend

gemessen, ob und wie stark dieser Ansatz die etablierten Methoden der Kontextbeschaffung qualitativ übertrifft.

1.3. Zielsetzung und Forschungsfragen

Zielsetzung Zentrales Ziel dieser Bachelorarbeit ist die Konzeption, prototypische Implementierung und systematische Evaluation eines kontextsensitiven, LLM-basierten Code-Review-Agenten für GitLab. Dabei soll untersucht werden, inwiefern die gezielte Anreicherung des Modellkontexts mit strukturiertem Architekturwissen, konkret in Form von Architecture Decision Records (ADRs), die Qualität und Zuverlässigkeit der maschinellen Code-Prüfung verbessert.

Forschungsfragen Um dieses Ziel zu erreichen, beantwortet die vorliegende Arbeit die folgenden drei Forschungsfragen:

1. Inwiefern verbessert eine agentische Evidenzbeschaffung die Erkennung von Architekturverstößen (ADRs) im KI-gestützen Code-Review im Vergleich zu einer statischen RAG-Pipeline?
2. Worauf lässt sich ein Leistungsunterschied zwischen statischem und agentischem Ansatz zurückführen, auf die verbesserte Evidenzlage durch autonome Navigation oder auf bloße Prompting-Effekte?
3. Wie lässt sich ein agentischer Architektur-Judge praxisgerecht in den GitLab-Workflow (Merge Requests) integrieren und welche Grenzen müssen dabei beachtet werden?

2. Grundlagen

TODO: nochmal prüfen, ob alles so stehen kann

2.1. Code Reviews im Softwareentwicklungsprozess

2.1.1. Begriff, Ziele und Formen

Der Begriff Code-Review bezeichnet den Prozess, bei dem Quellcode einer systematischen Kontrolle durch unabhängige Dritte oder Fachkollegen unterzogen wird [4, 43]. Die methodischen Ursprünge reichen bis in die 1970er Jahre zurück, als Fagan erstmals formalisierte Code-Inspektionen mit festen Rollen und strengen Ablaufplänen etablierte [13]. Das klassische Hauptziel dabei ist es, Fehler und Schwachstellen frühzeitig abzufangen, was die spätere Softwarequalität nachweislich verbessert [34].
Im heutigen Entwicklungsalltag geht der Nutzen dieser Praxis jedoch weit über das reine Auffinden von Defekten hinaus. Eine Untersuchung bei Microsoft zeigt: Obwohl Entwickler die Fehlersuche oft als Hauptmotivation angeben, liegt der tatsächliche Mehrwert vor allem im Wissenstransfer und dem Aufbau eines gemeinsamen Codeverständnisses [4]. Wer regelmäßig fremden Code begutachtet, vergrößert seine Vertrautheit mit den Projektdateien enorm [43].
Genau dieser Aspekt macht das Review zu einem zentralen Instrument, um architektonisches Entwurfswissen im Team zu verbreiten [4, 43]. Es liegt daher nahe, im Review genau jene Entscheidungen zu diskutieren und zu überprüfen, die in Architecture Decision Records (ADRs, siehe Abschnitt 2.4) explizit dokumentiert sind.

2.1.2. Modern Code Review als werkzeuggestützter Prozess

In der Praxis haben sich die schwergewichtigen und stark formalisierten Inspektionen nach Fagan weitgehend nicht durchgesetzt [43, 4]. Stattdessen hat sich branchenübergreifend ein Ansatz etabliert, den Bacchelli und Bird als *Modern Code Review* bezeichnen [4, 43]. Dieser Prozess ist deutlich leichtgewichtiger, verlässt sich stark auf moderne Werkzeuge und lässt sich nahtlos in den täglichen Entwicklungsablauf integrieren [4].
Im Kern zeichnet sich dieses moderne Vorgehen durch zwei Eigenschaften aus, die es von der klassischen Inspektion abheben [43]:

- **Fokus auf den Diff:** Es wird nicht mehr das gesamte System in einer Sitzung geprüft, sondern immer nur der isolierte Änderungssatz (der Diff)[43].
- **Asynchroner Ablauf:** Die Begutachtung findet nicht zeitgleich statt, sondern asynchron über Plattform-Funktionen wie Pull Requests oder Merge Requests (siehe Abschnitt 2.5) [43].

Genau diese Konzentration auf den reinen Code-Diff bringt jedoch eine entscheidende Schwäche mit sich, die für diese Arbeit zentral ist: Wenn man nur die geänderten Zeilen

betrachtet, fehlt oft das Gefühl für das architektonische Gesamtbild. Reviewer müssen sich den Kontext und die Motivation hinter der Änderung oft mühsam aus anderen Quellen zusammensuchen [4].
Wie sehr das den Arbeitsfluss in der Praxis stört, zeigt eine Untersuchung bei Google: Sadowski et al. stellten dort fest, dass fehlender Kontext (*Context*) eine der Hauptursachen für Missverständnisse im Review ist, schlichtweg deshalb, weil den Prüfenden der Hintergrund einer Code-Änderung unklar bleibt [44].
Abbildung 2.1 stellt diese Ausgangslage gegenüber: Was im Diff steht, liegt den Prüfenden unmittelbar vor, während das architektonische Entwurfswissen aus anderen Quellen beschafft werden muss. Genau dieser zusätzliche Beschaffungsschritt ist der eigentliche Engpass des Verfahrens.

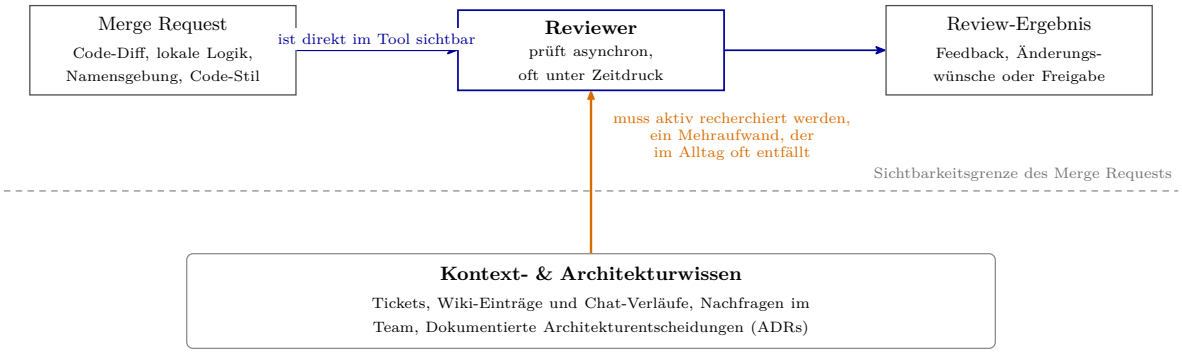


Abbildung 2.1.: Kontextlücke im Modern Code Review. Der Änderungssatz liefert nur einen Teil der für die Beurteilung nötigen Information; das Entwurfswissen liegt außerhalb des Merge Requests. Eigene Darstellung nach [4, 44].

2.1.3. Grenzen des menschlichen Reviews

Entgegen der landläufigen Meinung scheitern Code-Reviews selten an reiner Zeitnot. Daten von Google zeigen beispielsweise, dass selbst sehr umfangreiche Änderungen meist zügig innerhalb weniger Stunden geprüft werden [44]. Die eigentliche Hürde ist laut Bacchelli und Bird kognitiver Natur: Es ist schlichtweg anstrengend, fremden Code und dessen Kontext komplett zu durchdringen [4]. Da aktuelle Werkzeuge die Entwickler bei dieser mühsamen Kontextbeschaffung kaum unterstützen, entstehen genau an dieser Verständnishürde die eigentlichen Verzögerungen im Prozess [4].
Besonders gravierend ist dieses fehlende Kontextwissen bei architektonischen Vorgaben. Sadowski et al. stellten fest, dass in vielen Teams Uneinigkeit darüber herrscht, ob das Code-Review überhaupt der richtige Ort ist, um Software-Design zu prüfen [44]. Die primäre Herausforderung besteht darin, dass die Begründung für eine Architektur, also das Wissen, warum ein System so und nicht anders aufgebaut ist, in der Regel nicht im Quellcode steht [40]. Ein Prüfer kann einen Architekturverstoß also nur dann erkennen, wenn er die andernorts dokumentierten Entwurfsentscheidungen (siehe Abschnitt 2.3) nicht nur kennt, sondern im entscheidenden Moment auch aktiv im Kopf hat.
Dass Menschen dabei ohne maschinelle Hilfe oft überfordert sind, belegt ein Experiment von Li et al.: Entwickler übersahen Architekturprobleme im Code mehrheitlich, erkannten sie aber deutlich zuverlässiger, sobald ihnen gezielte Hinweise auf mögliche Regelverletzungen vorlagen [29]. Die konkreten Erkennungsraten und die Einordnung dieser Studie

folgen in Abschnitt 3.2.3.
Klassische Werkzeuge zur statischen Code-Analyse können diese nötigen Hinweise in der Praxis jedoch oft nicht liefern. Sie erfordern starr formalisierte Regeln (wie Abhängigkeitsmatrizen), die sich für die oft komplexen Absichten hinter Architekturentscheidungen gar nicht formulieren lassen [23]. Zudem werden solche Tools von Entwicklern häufig ignoriert, weil sie schlecht in den Arbeitsfluss integriert sind und zu viele Fehlalarme produzieren [20]. Genau an dieser Lücke, zwischen starren, statischen Regeln auf der einen und der kognitiven Überlastung des menschlichen Prüfers auf der anderen Seite, setzen die in dieser Arbeit untersuchten KI-gestützten Verfahren an.

2.2. Large Language Models und agentische Systeme

2.2.1. Grundbegriffe und Verhaltenseigenschaften

Ein Large Language Model (LLM) ist im Kern ein neuronales Netz der Transformer-Architektur [54], das mit gewaltigen Textmengen trainiert wurde [16]. Die grundlegende Funktionsweise ist simpel: Das Modell setzt eine Eingabe kontinuierlich fort, indem es schrittweise das jeweils nächste Textstück (Token) vorhersagt [16].
Die Datenmenge, die ein Modell pro Interaktion verarbeiten kann, wird durch das sogenannte Kontextfenster begrenzt. Diese technische Grenze ist für KI-gestützte Code-Reviews entscheidend: Wenn ein Projekt größer ist als dieses Fenster, muss das System filtern und exakt auswählen, welche Code-Abschnitte für das Modell überhaupt relevant sind. Gesteuert wird das LLM über Prompts. In der Praxis trennt man dabei meist zwischen einem System-Prompt, der dem Modell seine Rolle und feste Regeln vorgibt, und der eigentlichen Eingabe des Nutzers.
Für die spätere methodische Auswertung sind zwei typische Eigenheiten von LLMs besonders wichtig:

- **Fehlender Determinismus:** Die Textgenerierung basiert auf Wahrscheinlichkeiten. Das bedeutet, dass exakt dieselbe Eingabe zu völlig unterschiedlichen Ausgaben führen kann. Ein einzelner Testlauf ist deshalb immer nur eine Momentaufnahme und liefert keine absolut reproduzierbaren Messwerte.

- **Halluzinationen:** LLMs neigen dazu, Fakten zu erfinden, die sprachlich extrem logisch und überzeugend klingen, in der Realität aber schlichtweg falsch sind [19]. Für den Code-Review bedeutet das konkret: Das Modell meldet manchmal einen Regelverstoß, für den es im echten Quellcode überhaupt keine Grundlage gibt. Genau solche Fehlalarme (False Positives) sind ein zentrales Problem, das wir mit den späteren Metriken (siehe Abschnitt 2.6) abbilden müssen.

2.2.2. Strukturierte Ausgaben und Werkzeugnutzung

Damit ein Sprachmodell mehr tun kann als reine Texte zu generieren, müssen seine Ausgaben maschinell verarbeitbar sein. Dass Modelle lernen können, selbstständig externe Werkzeuge und Schnittstellen aufzurufen, zeigten Schick et al. mit dem *Toolformer*-Ansatz [45]. In modernen Modell-APIs ist diese Funktion als *Tool Calling* etabliert: Anstelle von freiem Fließtext erzeugt das LLM ein strukturiertes Datenobjekt (meist JSON), das eine konkrete Funktion und deren Parameter benennt.

Die eigentliche Ausführung übernimmt dabei nie das Modell selbst, sondern die umgebende Software [31]. Das LLM dient lediglich als Steuerzentrale: Es schlägt vor, welches Werkzeug benötigt wird, erhält das Ergebnis zurück und zieht daraus seine nächsten Schlüsse [31].
In der Praxis gibt es dabei jedoch eine entscheidende Hürde: Nicht jedes Modell hält sich zuverlässig an die geforderten Strukturvorgaben, und Formatfehler beim Erzeugen der JSON-Aufrufe machen ein Modell für einen werkzeugnutzenden Agenten unbrauchbar. Eine strikte Formattreue ist daher ein zentrales Auswahlkriterium für die Eignung der Modelle in dieser Arbeit.

2.2.3. Agentische Systeme und das ReAct-Muster

In dieser Arbeit wird ein LLM dann als *Agent* bezeichnet, wenn es nicht nur starr auf eine Eingabe antwortet, sondern in einer Schleife arbeitet und autonom entscheidet, welche Werkzeuge es einsetzt [31]. Die methodische Grundlage dafür bildet meist das sogenannte ReAct-Muster (Reasoning and Acting) von Yao et al. [59]. Das Prinzip dahinter ist sehr natürlich: Das Modell wechselt kontinuierlich zwischen logischem Schlussfolgern („Was muss ich als Nächstes tun?“) und aktivem Handeln („Ich rufe jetzt Werkzeug X auf“). Die Rückgaben der Werkzeuge liest das Modell ein und arbeitet damit weiter, bis es eine finale Lösung gefunden hat [59]. Da dieser Prozess theoretisch in einer Endlosschleife stecken bleiben könnte, erzwingt man in der Praxis meist ein hartes Abbruchlimit für die Iterationen [31]. Abbildung 2.2 zeigt diesen Ablauf im Überblick.

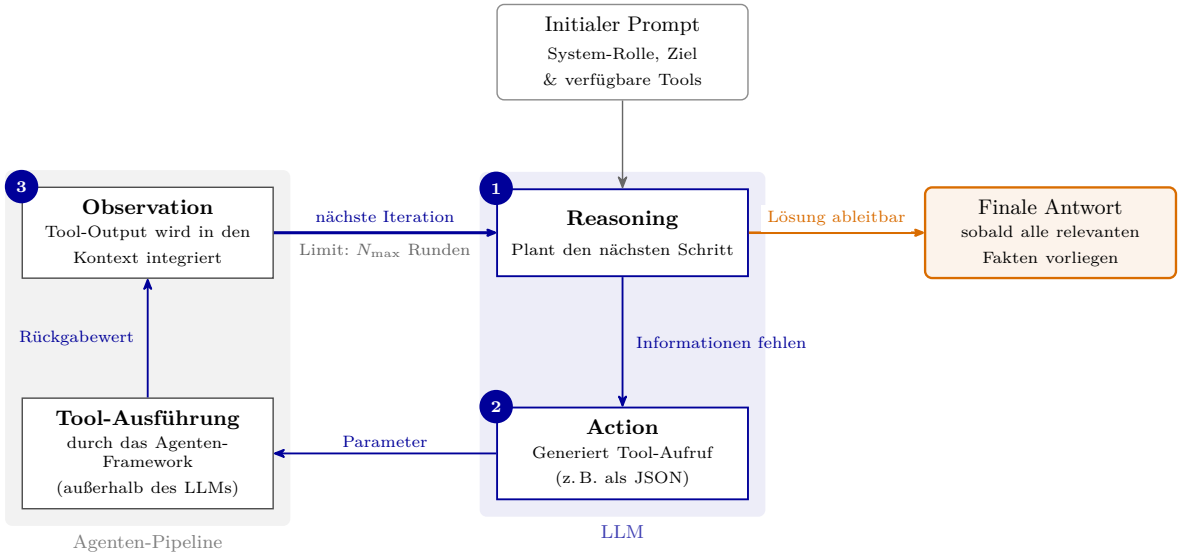


Abbildung 2.2.: Iterativer Ablauf des ReAct-Musters. Das Modell wechselt zwischen Schlussfolgern und Handeln; die Ausführung der Werkzeuge übernimmt die umgebende Software. Eigene Darstellung nach [59, 31].

Wenn Aufgaben für ein einzelnes Modell zu komplex werden, teilt man sie oft auf mehrere spezialisierte Agenten auf, die miteinander interagieren [31]. Dass dieses Konzept auch im Code-Review funktioniert, zeigten Tang et al. bereits mit ihrem Multi-Agenten-System *CodeAgent* [51].
Für die vorliegende Arbeit hat diese agentische Arbeitsweise eine ganz entscheidende Konsequenz: Der Kontext, den das LLM zur Beurteilung nutzt, wird ihm nicht vorab als sta-

tischer Textblock übergeben. Stattdessen konstruiert sich der Agent sein Kontextfenster dynamisch zur Laufzeit selbst, geleitet durch seine eigenen Such- und Navigationsentscheidungen.

2.2.4. Retrieval-Augmented Generation als statische Kontextbeschaffung

Den Gegenentwurf zu agentengetriebenen Systemen bildet die Retrieval-Augmented Generation (RAG) nach Lewis et al. [27]. Der entscheidende Unterschied: Das Modell sucht sich seinen Kontext nicht selbst, sondern bekommt ihn durch ein vorgeschaltetes Suchverfahren einmalig und statisch vorgegeben.
Dazu wird der Quellcode vorab in kleine Abschnitte zerlegt (*Chunking*) und mithilfe eines Embedding-Modells [42] in Vektoren umgewandelt. Bei einer Abfrage berechnet ein Vektorindex [21] die k ähnlichsten Code-Schnipsel und übergibt diese an das LLM. Das Modell sieht ausschließlich diese Vorauswahl und hat keine Möglichkeit, eigenständig weitere Dateien nachzuschlagen.
Aus dieser Architektur ergeben sich drei wesentliche Einschränkungen:

- **Starrer Kontextumfang:** Die Anzahl k der abgerufenen Schnipsel ist fest. Das Modell erhält also immer gleich viel Kontext, egal ob die Regel einfach ist oder ein komplexes Zusammenspiel betrifft.
- **Schwäche bei fehlendem Code:** Die Ähnlichkeitssuche basiert auf *vorhandenem* Text. Fehlt ein gefordertes Muster im Code (etwa eine vergessene Eingabevalidierung), lässt sich diese Abwesenheit über eine Vektorsuche strukturell kaum finden.
- **Zerstückelung durch Chunking:** Durch das Zerschneiden des Codes in feste Abschnitte gehen Zusammenhänge verloren, die sich über mehrere Dateien erstrecken.

Ob und wie stark sich diese methodischen Schwächen im Experiment bemerkbar machen, wird anhand der protokollierten Werkzeugaufrufe untersucht (Abschnitt 6.3).

2.2.5. Gegenüberstellung der beiden Strategien

Im Zentrum dieser Arbeit steht der direkte Vergleich der beiden beschriebenen Ansätze: Auf welchem Weg erschließt sich das Modell den benötigten Quellcode-Kontext? Tabelle 2.1 fasst die grundlegenden Unterschiede zwischen der statischen und der agentischen Strategie übersichtlich zusammen.
Um sicherzustellen, dass die späteren Evaluationsergebnisse wirklich nur auf diese unterschiedliche Art der Informationsbeschaffung zurückzuführen sind, bleiben alle anderen Bausteine der Pipeline im direkten Vergleich strikt unverändert.

Tabelle 2.1.: Statische und agentische Evidenzbeschaffung im Vergleich.

	Statisch (RAG)	Agentisch (Werkzeuge)
Zeitpunkt der Kontextauswahl	vor dem Modellaufruf	Dynamisch zur Laufzeit
Auswahlkriterium	Semantische Vektorähnlichkeit	Autonome Suchstrategie des Modells
Umfang des Kontexts	Statisch fixiert (k Abschnitte)	Variabel, bis zur Schrittgrenze
Dynamisches Nachfordern	Nicht möglich	Möglich
Modellaufrufe je Fall	Singulär	Iterativ (mehrfach)
Nachvollziehbarkeit	Abgerufene Abschnitte protokollierbar	Komplette Aufrufkette (Trace) protokollierbar

2.2.6. Reasoning-Modelle und Chain-of-Thought

Die Genauigkeit von Sprachmodellen bei komplexen Aufgaben steigt deutlich, wenn man sie dazu anhält, vor der eigentlichen Antwort logische Zwischenschritte zu formulieren. Dieses Vorgehen hat sich unter dem Begriff *Chain-of-Thought* (CoT) etabliert [56]. Moderne *Reasoning-Modelle* sind speziell darauf trainiert, solche Denkschritte autonom und sehr ausführlich zu durchlaufen. Sie investieren dafür direkt bei der Abfrage (zur Inferenzzeit) mehr Rechenleistung als klassische Allzweckmodelle [61, 48].
Diese Unterscheidung ist für die spätere Einordnung der Ergebnisse von zentraler Bedeutung: Während die Referenzstudie von Su et al. mit *Marco-o1* ein solches spezialisiertes Reasoning-Modell als Richter (*Judge*) einsetzte [49, 61], standen für die eigenen Experimente über den Hochschul-Gateway ausschließlich universelle Allzweckmodelle zur Verfügung (siehe Abschnitt 5.2.2 sowie Anhang A.2.1).

2.2.7. LLM-as-a-Judge

Wenn ein Sprachmodell keine eigenen Texte schreibt, sondern bestehende Inhalte bewerten soll, spricht man vom Konzept *LLM-as-a-Judge*. Studien wie die von Zheng et al. zeigen, dass gute Modelle hierbei ähnlich verlässlich urteilen wie menschliche Prüfer [62]. Da sie jedoch anfällig für systematische Verzerrungen (Biases) sind, verlässt man sich in der Praxis selten auf ein einzelnes Modell. Stattdessen wählt man oft ein mehrstufiges Kaskaden-System: Ein erstes Modell (der *Judge*) fällt ein vorläufiges Urteil, das dann von weiteren Instanzen (*Validatoren*) gegengeprüft wird. Genau diesen Aufbau, ein Richter, der Verstöße vorselektiert, und drei Validatoren, die ihn überprüfen, nutzt auch die Referenzstudie von Su et al. [49].
Diese Arbeitsteilung bringt jedoch ein ganz eigenes methodisches Risiko mit sich: Wenn ein Validator das Urteil des Judges überprüfen soll, dabei aber nicht exakt denselben Quellcode-Kontext sieht, muss er quasi „blind“ urteilen. In solchen Fällen neigen Sprachmodelle dazu, sich einfach der bereits vorgegebenen Entscheidung anzuschließen, statt auf einer objektiven Prüfung zu bestehen. In der Forschung ist dieses Phänomen als *Sycophancy* bekannt, Modelle haben die Tendenz, einer im Prompt vorgegebenen Meinung sprichwörtlich nach dem Mund zu reden [46].

Genau dieser Ankereffekt prägt auch den Referenzdatensatz dieser Arbeit (Anhang A.2.3). Um zu verhindern, dass die Validatoren die Fehlurteile des Judges einfach durchwinken, musste der Evaluationsaufbau dort konzeptionell angepasst werden. Derselbe Effekt liegt, in weitaus folgenreicherer Form, dem Referenzdatensatz der Originalarbeit zugrunde: Dort erhalten die prüfenden Modelle das Urteil des Judges samt Begründung vorgelegt, und ihre Zustimmung geht anschließend als Ground Truth in die Auswertung ein (Anhang A.2.3).

2.2.8. Kontextqualität als Einflussgröße

Bei Sprachmodellen gilt nicht automatisch der Grundsatz „mehr hilft mehr“. Ein bloßes Aufblähen des Kontexts führt oft sogar zu schlechteren Ergebnissen. Aus der aktuellen Forschung lassen sich drei wesentliche Hürden bei der Informationsverarbeitung von LLMs ableiten:

- **Ablenkung durch irrelevante Details:** Shi et al. zeigten, dass schon beiläufige, irrelevante Zusatzinformationen im Prompt die Lösungsgenauigkeit der Modelle spürbar verschlechtern [47].
- **Der *Lost-in-the-Middle*-Effekt:** Bei sehr langen Eingabetexten tritt ein starker Positionseffekt auf. Wie Liu et al. belegen, verarbeiten Modelle die Informationen am besten, wenn sie ganz am Anfang oder am Ende stehen, wichtige Details im Mittelteil werden dagegen oft drastisch schlechter erkannt [32].
- **Schwäche bei Negationen:** LLMs tun sich erstaunlich schwer mit verneinten Anweisungen. Jang et al. stellten hier ein geradezu paradoxes Verhalten fest: Je größer das Modell, desto fehleranfälliger reagiert es oft auf Negationen [17].

Besonders der Positionseffekt hat unmittelbare Folgen für die Aufbereitung des Kontexts. Abbildung 2.3 verdeutlicht den von Liu et al. berichteten Verlauf schematisch: Wächst die Eingabe, rückt ausgerechnet die zu prüfende Vorgabe in den Bereich, der am schlechtesten genutzt wird.

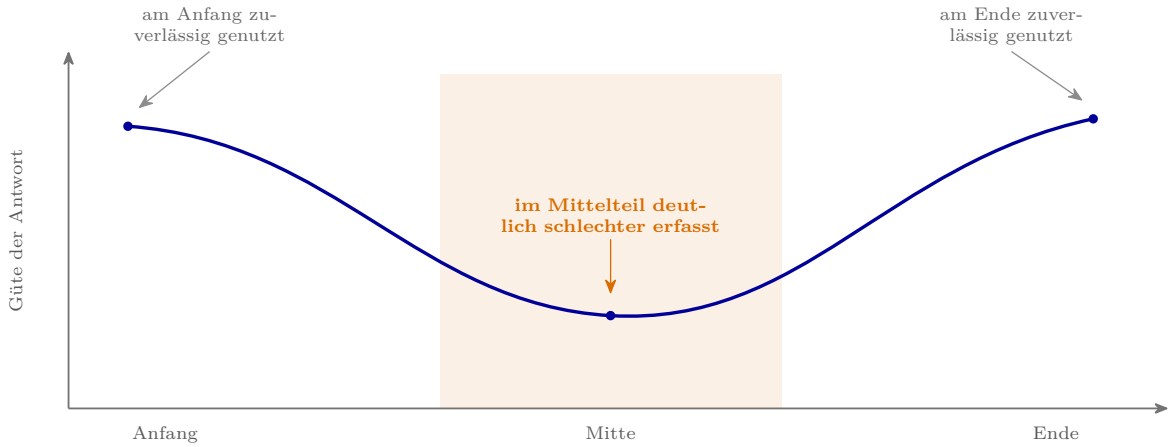


Abbildung 2.3.: Positionseffekt im Kontextfenster (*Lost in the Middle*). Rein schematische Darstellung ohne Skala und ohne abgetragene Messwerte, nach [32].

Auch wenn diese Erkenntnisse ursprünglich aus allgemeinen Text-Benchmarks stammen, liefern sie eine klare Leitlinie für die Praxis: Für ein LLM sind kompakte, positiv formulierte und auf das Wesentliche reduzierte Kontextquellen viel wertvoller als dicke, unstrukturierte Dokumentationsordner.
Genau diese Erkenntnis bildet das theoretische Fundament für die weitere Arbeit. Sie begründet die Kritik an den bisherigen Code-Review-Werkzeugen in Abschnitt 3.1 und erklärt, warum die Architekturentscheidungen (ADRs) in Kapitel 7 für den KI-Agenten erst noch gezielt gefiltert und aufbereitet werden müssen.

2.3. Softwarearchitektur und Architekturkonformität

2.3.1. Architekturerosion und Architekturdrift

In der Softwarearchitektur unterscheidet man nach Perry und Wolf zwei Hauptformen, wie ein System im Laufe der Zeit von seinem ursprünglichen Entwurf abweicht [40]:

- **Architekturerosion (explizite Verstöße):** Sie entsteht durch direkte Verletzungen architektonischer Vorgaben, etwa durch unzulässige Modulzugriffe. Das System verliert dadurch an Stabilität und wird zunehmend fragil.
- **Architekturdrift (schleichender Verfall):** Sie resultiert aus der unbewussten Vernachlässigung der ursprünglichen Entwurfskonzepte. Die Struktur verliert nach und nach an Klarheit, was das System aufweicht und zukünftige, noch schwerere Regelverstöße begünstigt.

Beide Phänomene führen dazu, dass die dokumentierte Soll-Architektur und die tatsächlich implementierte Ist-Architektur immer weiter auseinanderdriften. Abbildung 2.4 stellt beide Formen der Abweichung gegenüber. Der Unterschied ist für diese Arbeit zentral: Ein expliziter Verstoß lässt sich an einer konkreten Stelle im Code benennen, während die Drift erst im Zusammenspiel mehrerer Änderungen sichtbar wird.

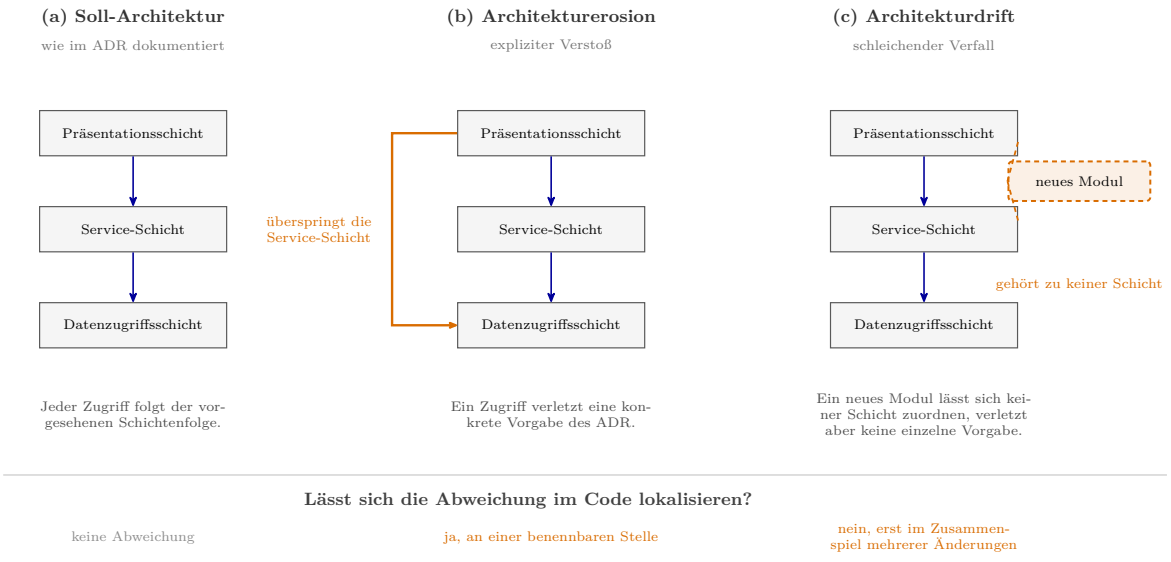


Abbildung 2.4.: Architekturerosion und Architekturdrift im Vergleich zur dokumentierten Soll-Architektur. Eigene Darstellung nach [40].

Besonders die Architekturdrift ist dabei eine direkte Folge des zuvor beschriebenen Verfügbarkeitsproblems: Wenn Entwickler und Reviewer die ursprünglichen Entwurfsentscheidungen im Moment einer Code-Änderung nicht präsent haben oder diese nicht ausreichend kommuniziert wurden, können sie schlichtweg nicht bewusst eingehalten werden.

2.3.2. Konformitätsprüfung als Aufgabe

Die Konformitätsprüfung gleicht die geplante Soll-Architektur mit der tatsächlich im Quellcode umgesetzten Ist-Architektur ab, um Abweichungen frühzeitig zu erkennen. Etablierte statische Verfahren, wie etwa Reflexionsmodelle oder regelbasierte Analysen, setzen dafür eine formal definierte Soll-Architektur voraus [23, 39]. Ihr Fokus liegt dabei fast ausschließlich auf der Analyse von Modulabhängigkeiten und Importstrukturen.
Daraus ergibt sich eine entscheidende Schwachstelle: Klassische Werkzeuge können nur solche Regeln überprüfen, die sich als direkte strukturelle Abhängigkeit zwischen Modulen ausdrücken lassen. Architekturentscheidungen, die eher Konventionen oder das Verhalten im Code betreffen, wie etwa Vorgaben zur Fehlerbehandlung, Logging-Standards oder einheitliche Datenformate, bleiben für sie systembedingt unsichtbar.
Eine detaillierte Gegenüberstellung dieser traditionellen Verfahren und die Einordnung aktueller LLM-basierter Alternativen folgt bei der Diskussion des Forschungsstandes in Abschnitt 3.2.3.

2.4. Architecture Decision Records (ADRs) als Kontextquelle

Eine der größten Herausforderungen der Softwarearchitektur ist, dass Entwurfsentscheidungen und ihre Begründungen in der Praxis oft nirgends festgehalten werden. Das Wissen über das System verdunstet im Laufe der Zeit quasi, ein schleichender Prozess, den Jansen und Bosch treffend als *Knowledge Vaporization* bezeichnen [18]. Besonders in agilen Projekten, in denen Dokumentation bewusst knapp gehalten wird, ist dieser Wissensverlust ein massives Risiko [53].

Das ADR-Konzept Als pragmatische Lösung für dieses Problem haben sich die sogenannten *Architecture Decision Records* (ADRs) nach Michael Nygard in der Industrie breitflächig etabliert [37, 3]. Ein ADR ist ein kurzes Dokument von meist nur ein bis zwei Seiten, das exakt eine wichtige Architekturentscheidung isoliert festhält. Um das Format einheitlich zu halten, besteht ein ADR klassischerweise aus fünf Kernelementen [37]:

- **Titel:** Eine kurze, prägnante Bezeichnung der Entscheidung.
- **Status:** Der aktuelle Lebenszyklus der Entscheidung, etwa *proposed*, *accepted*, *deprecated* oder *superseded*.
- **Kontext:** Eine wertneutrale Beschreibung der Ausgangslage, inklusive aller technologischen und organisatorischen Rahmenbedingungen.
- **Entscheidung:** Die getroffene Wahl, aktiv formuliert („Wir werden …“; im englischsprachigen Original „We will …“).

- **Konsequenzen:** Die Auswirkungen auf das System, einschließlich aller positiven und negativen Folgen (Trade-offs).

Ein entscheidender methodischer Vorteil von ADRs ist ihr Speicherort: Sie werden als unveränderliche, fortlaufend nummerierte Textdateien direkt mit dem Quellcode im Versionskontrollsystem abgelegt. Überholte Entscheidungen werden dabei nicht einfach gelöscht, sondern durch neue Dateien ersetzt und als *superseded* (abgelöst) markiert, sodass die Entscheidungs-Historie lückenlos nachvollziehbar bleibt [37].
Dieses Prinzip nennt sich *Documentation-as-Code* [25]. Die Architektur-Dokumentation wird mit denselben Werkzeugen (wie Git) verwaltet, in denselben Code-Reviews geprüft und im selben Rhythmus aktualisiert wie die Software selbst. Um diesen Ablauf noch reibungsloser zu gestalten, hat sich als Standard das Markdown-Format (*Markdown Architectural Decision Records*, MADR) durchgesetzt, welches lästige Medienwechsel für die Entwickler auf ein Minimum reduziert [24].
Abbildung 2.5 zeigt den Aufbau eines solchen ADR und ordnet jedem Abschnitt seine Rolle für eine automatisierte Prüfung zu. Zwei Elemente sind dabei besonders hervorzuheben: Der Abschnitt *Entscheidung* liefert die eigentliche, am Code überprüfbare Regel, während der *Status* darüber bestimmt, ob ein Dokument überhaupt als gültige Vorgabe herangezogen werden darf. Ein vollständiges Beispiel-ADR findet sich im Anhang A.1.

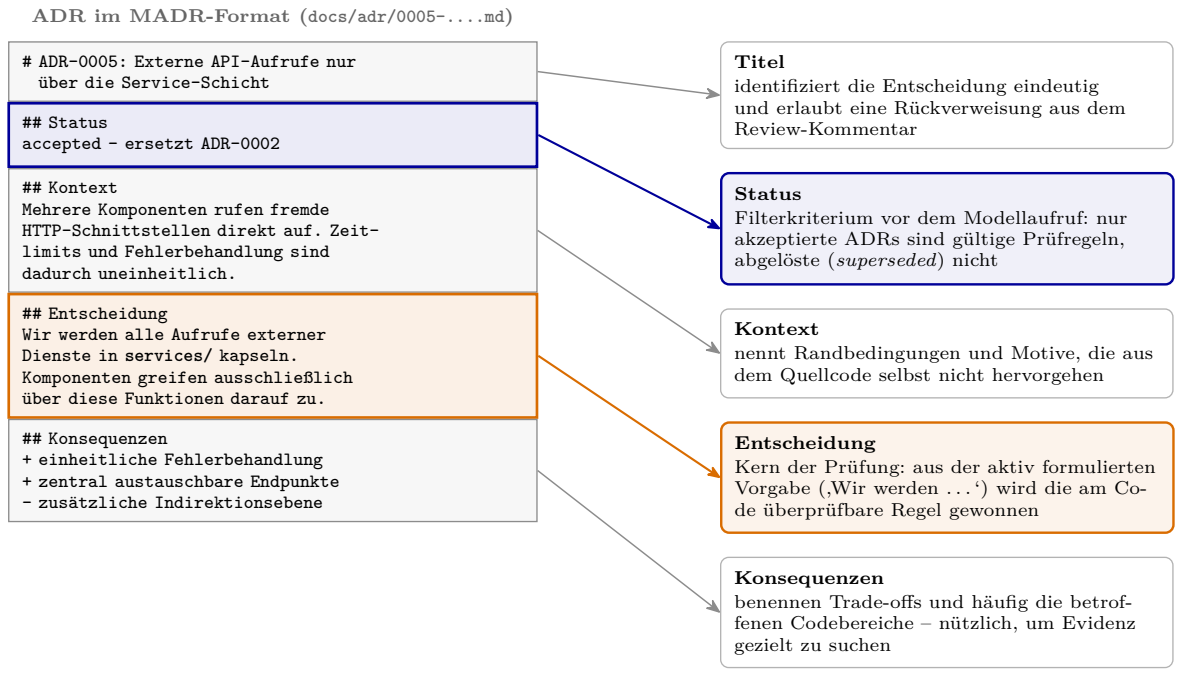


Abbildung 2.5.: Aufbau eines ADR im MADR-Format und Bedeutung der einzelnen Abschnitte für die automatisierte Konformitätsprüfung. Eigene Darstellung nach [37, 24].

Empirische Befunde zum Praxiseinsatz Dass dieses Konzept in der Praxis auch wirklich funktioniert, zeigen Ahmeti et al. [3]. Bei der Begleitung eines Softwareunternehmens zeigte sich: Das ADR-Format verbessert den Wissenstransfer spürbar, räumt mit Dokumentations-Defiziten auf und fördert die Zusammenarbeit im Team. Die Studie offenbart jedoch auch Hürden: Der tatsächliche Nutzen steht und fällt mit dem konkreten

Ablageort der Dokumente, und gerade bei stark verteilten Systemen bleibt eine übersichtliche Dokumentation eine Herausforderung.

ADRs als Kontextquelle für KI-gestützte Code-Reviews Für die Konzeption der vorliegenden Arbeit erweisen sich ADRs aus zwei zentralen Gründen als ideale Informationsbasis.
Erstens bringen sie genau die Struktur mit, die Large Language Models (LLMs) für die Verarbeitung brauchen: Sie liegen als einfache Textdateien im selben Repository wie der Quellcode und benennen exakt jene Entwurfsabsichten, die aus einem reinen Code-Diff nicht hervorgehen. Weil Sprachmodelle durch irrelevante Texte nachweislich an Präzision verlieren (siehe Abschnitt 2.2.8), ist das extrem hohe Signal-Rausch-Verhältnis dieser hochverdichteten ADRs ein massiver Vorteil gegenüber umfangreichen, unstrukturierten Handbüchern.
Zweitens zeigen aktuelle Studien, dass LLMs mit diesen Dokumenten tatsächlich arbeiten können. Su et al. haben untersucht, wie zuverlässig Modelle dokumentierte Architekturverletzungen im Code erkennen [49]. Ihr Ergebnis lässt sich auf ein Prinzip verdichten: Je expliziter eine Entscheidung im Quellcode verankert ist, desto treffsicherer urteilen die Modelle. Bei impliziten Vorgaben, die externes Firmenwissen voraussetzen, stoßen sie dagegen an ihre Grenzen. LLMs können die Konformitätsprüfung also effektiv unterstützen, ersetzen den Menschen bei kontextsensitiven Entscheidungen aber nicht. Der Umfang dieser Studie und ihre Kennzahlen werden in Abschnitt 3.2.3 behandelt, da sie zugleich die Vergleichsbasis dieser Arbeit bildet.
Genau diese Erkenntnis bildet das Fundament für die vorliegende Arbeit: Sie nutzt ADRs systematisch als externe Wissensquelle, um den KI-gestützten Code-Review-Prozess in GitLab zielgenau und kontextsensitiv zu steuern.

2.5. Das Ökosystem von GitLab

GitLab ist heute weit mehr als nur ein reines Versionskontrollsystem. Es hat sich zu einer vollumfänglichen DevOps-Plattform entwickelt [57]. Der architektonische Kern ist dabei das „Single Application"-Prinzip: Statt vieler isolierter Werkzeuge vereint GitLab den gesamten Softwarelebenszyklus in einer einzigen Anwendung [57]. Für die Entwicklung von KI-Agenten ist das ein enormer Vorteil, da alle relevanten Kontextinformationen, vom ursprünglichen Issue bis hin zum automatisierten Testergebnis, über eine gemeinsame Schnittstelle abrufbar sind.

Kernkomponenten der Softwarebereitstellung Das Ökosystem stützt sich auf zwei funktionale Säulen, die den Rahmen für das Code-Review bilden:

- **Versionsverwaltung (SCM):** Die technische Basis bildet Git, welches über strukturierte Branching-Strategien (wie den *GitLab Flow*) eine parallele und konfliktfreie Zusammenarbeit im Team ermöglicht [58].
- **CI/CD-Pipelines:** Bei jeder Code-Änderung laufen automatisierte Skripte an, die den Quellcode kompilieren, testen und via Linter prüfen [8]. Diese Pipelines fungieren als erste, rein regelbasierte Prüfinstanz, noch bevor das eigentliche (menschliche oder KI-gestützte) Review beginnt.

Der Merge Request als zentraler Knotenpunkt Der primäre Ort der Qualitätskontrolle ist der sogenannte Merge Request (MR). Er wird initiiert, sobald Entwicklungs-Code in einen Ziel-Branch überführt werden soll. Der MR ist dabei nicht nur ein technischer Vorgang, sondern der zentrale kollaborative Interaktionsraum [36]. In seiner Benutzeroberfläche fließen alle Fäden zusammen:

1. Der zeilengenaue Vergleich der Code-Änderungen (Diff).
2. Die Ergebnisse der CI/CD-Pipelines und Sicherheits-Scans.
3. Die textbasierte Diskussion der Entwickler.

Genau hier findet das klassische Code-Review statt, bei dem Zeilen kommentiert und Anpassungen gefordert werden. Durch diese hohe Informationsdichte ist der Merge Request der logische Andockpunkt für KI-gestützte Assistenzsysteme.

Erweiterbarkeit und Schnittstellen Damit externe KI-Modelle nahtlos in diesen Ablauf eingreifen können, bietet GitLab umfassende REST- und GraphQL-Schnittstellen [15, 55]. Das entscheidende Bindeglied für die Automatisierung sind dabei ereignisbasierte Webhooks. Sobald beispielsweise ein Merge Request erstellt oder kommentiert wird, sendet GitLab ein Signal an das externe KI-System. Das Sprachmodell kann den Code daraufhin analysieren und sein Feedback über die API direkt als Inline-Kommentar platzieren, exakt so, wie es ein menschlicher Entwickler tun würde.

2.6. Metriken zur Bewertung von Klassifikationsleistung

Dieser Abschnitt stellt die statistischen Metriken vor, mit denen die Leistung der KI-Modelle in Kapitel 6 gemessen wird. Warum genau diese Maße für die Auswertung ausgewählt wurden, wird in Abschnitt 4.2 methodisch begründet.

2.6.1. Klassifikationsaufgabe und Konfusionsmatrix

Die Prüfung einer Architekturregel lässt sich als dreistufiges Klassifikationsproblem verstehen:

- *Compliant* (C): Die Regel wird eingehalten.
- *Not Compliant* (NC): Die Regel wird explizit verletzt.
- *Code is Insufficient to Answer* (CIA): Der vorliegende Code liefert nicht genug Kontext für eine klare Entscheidung.

Besonders die CIA-Klasse erfüllt dabei eine extrem wichtige Funktion: Wenn ein System nicht selbstständig erkennt, dass ihm Informationen fehlen, wird es bei unklaren Code-Abschnitten zwangsläufig anfangen zu raten und Fehlurteile produzieren.
Um die Güte dieser Entscheidungen zu messen, betrachtet man jede Klasse einzeln gegen den Rest (Eins-gegen-Rest-Verfahren). Vergleicht man dann die Antwort des Modells mit dem tatsächlichen Referenzurteil (der sogenannten *Ground Truth*), ergeben sich vier

mögliche Fälle. Diese werden klassischerweise in einer Konfusionsmatrix abgebildet (Tabelle 2.2).

Tabelle 2.2.: Konfusionsmatrix für eine Klasse in der Eins-gegen-Rest-Betrachtung.

		Referenzurteil	
		Klasse	nicht Klasse
Modellurteil	Klasse	TP	FP
	nicht Klasse	FN	TN

2.6.2. Accuracy, Precision, Recall und F1

Aus den vier Basiswerten der Konfusionsmatrix berechnen sich die gängigen Evaluationsmetriken. Die *Accuracy* (Gesamtgenauigkeit) beschreibt schlicht den Anteil aller korrekten Vorhersagen an der Gesamtzahl der überprüften Fälle. *Precision* (Relevanz), *Recall* (Sensitivität) und deren harmonisches Mittel, der F_1-Score, sind wie folgt definiert:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Für den Entwicklungsalltag beantworten Precision und Recall zwei völlig unterschiedliche Fragen. Am Beispiel der Fehlerklasse NC (*Not Compliant*) bedeutet das:

- Die **Precision** verrät, wie viele der vom Modell gemeldeten Verstöße *tatsächlich* echte Architekturfehler sind. Sie misst also direkt, wie stark das Team durch Fehlalarme (False Positives) belastet wird.
- Der **Recall** misst dagegen, welchen Anteil der *real existierenden* Verstöße das System überhaupt aufgespürt hat. Er steht somit für die Vollständigkeit der Prüfung.

Zwischen diesen beiden Zielen besteht ein klassischer Zielkonflikt (Trade-off): Wenn ein System im Zweifelsfall lieber einen Verstoß meldet, übersieht es zwar kaum echte Fehler (hoher Recall), produziert aber unweigerlich mehr Fehlalarme (sinkende Precision). Genau dieser Kompromiss ist später der Schlüssel, um die Evaluationsergebnisse richtig interpretieren zu können: In Abschnitt 6.4 hebt eine einzige Prompt-Änderung den Recall der Klasse *CIA* von 36,8 % auf 94,7 % und senkt zugleich deren Precision – die Gesamtgenauigkeit steigt, das Verfahren wird aber nicht schlicht besser, sondern anders verzerrt.

2.6.3. Mittelung im Mehrklassenfall

Um aus den Metriken der einzelnen Klassen eine übersichtliche Gesamtkennzahl zu bilden, nutzt diese Arbeit die *Makro-Mittelung* (*Macro-Averaging*). Hierbei berechnet man schlicht das ungewichtete arithmetische Mittel über alle K Klassen, für den Recall und den F_1-Score gilt das analog:

$$\text{Precision}_{\text{makro}} = \frac{1}{K} \sum_{i=1}^{K} \text{Precision}_i.$$

Das bedeutet: Jede Klasse fließt mit exakt demselben Gewicht in das Endergebnis ein, völlig unabhängig davon, wie oft sie im Datensatz vorkommt. (Zum Vergleich: Eine *Mikro-Mittelung* würde stattdessen einfach alle Treffer in einen Topf werfen und dadurch häufige Klassen automatisch stärker gewichten).
Für die spätere Auswertung hat diese strikte Gleichgewichtung der Makro-Mittelung allerdings einen Haken: Wenn eine bestimmte Klasse in der Referenzlösung gar nicht vorkommt, ist ihr Recall mathematisch undefiniert und wird per Konvention auf null gesetzt. Weil die Makro-Mittelung aber stur den Durchschnitt über alle Klassen bildet, reißt diese "Null"das Gesamtergebnis mit vollem Gewicht $(1/K)$ nach unten.
Genau dieser statistische Verzerrungseffekt schlägt in der Evaluation bei der seltenen Klasse *NC* zu, auf die im Zufallsteil nur fünf Fälle entfallen (Abschnitt 8.3). Es ist wichtig, dieses methodische Detail im Hinterkopf zu behalten, um die dort berichteten Makro-Gesamtwerte später fair und richtig einordnen zu können.

2.6.4. Klassenungleichgewicht und der Matthews Correlation Coefficient

Wenn Testdaten ungleich verteilt sind (Class Imbalance), verliert die klassische Accuracy massiv an Aussagekraft. Dies lässt sich an einem Beispiel verdeutlichen: Besteht ein Datensatz zu 75 % aus Architekturverstößen (Klasse NC), würde ein nutzloses Modell, das einfach immer stur „Verstoß!“ meldet, bereits eine Accuracy von 75 % erzielen, obwohl es den Code überhaupt nicht verstanden hat.
Um sich von solchen Verzerrungen nicht blenden zu lassen, nutzt diese Arbeit zusätzlich den *Matthews Correlation Coefficient* (MCC) [33]. Sein großer Vorteil: Er bezieht alle vier Quadranten der Konfusionsmatrix gleichzeitig in die Berechnung ein. Da der MCC klassischerweise für binäre Probleme (zwei Klassen) gedacht ist, wird er hier für jede Klasse einzeln im Eins-gegen-Rest-Verfahren berechnet:

$$\mathrm{MCC} = \frac{\mathrm{TP} \cdot \mathrm{TN} - \mathrm{FP} \cdot \mathrm{FN}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})(\mathrm{TN}+\mathrm{FP})(\mathrm{TN}+\mathrm{FN})}}.$$

Das Ergebnis ist ein Wert zwischen -1 (komplett gegensätzliche Vorhersage) und $+1$ (perfekte Übereinstimmung), wobei 0 reinem Raten entspricht. Ein Modell bekommt hier nur dann einen hohen Score, wenn es wirklich in *allen* Kategorien treffsicher ist. Bei unbalancierten Datensätzen gilt der MCC daher als deutlich ehrlicher und verlässlicher als die klassische Accuracy oder der F_1-Score [6].
In der späteren Auswertung wird sich zeigen, dass Accuracy und MCC teilweise gegensätzliche Ergebnisse liefern: Der statische Judge mit permissivem Prompt erreicht in Lauf 1 eine niedrigere Accuracy als das Urteil des Original-Modells (65,0 % gegenüber 71,7 %), aber den höheren MCC (0,471 gegenüber 0,460); siehe Abschnitt 6.2. Genau diese metrische Diskrepanz ist dort der Schlüssel, um die tatsächliche Leistung der Modelle richtig bewerten zu können.

2.6.5. Übereinstimmungsmaße

Oft reicht es in der Auswertung nicht aus, nur auf die absoluten Trefferquoten zu schauen. Wenn man beispielsweise wissen will, wie oft sich zwei unabhängige Sprachmodelle (oder menschliche Prüfer) bei einem Code-Review *einig* sind, spricht man von der Inter-Rater-Reliabilität. Die einfache prozentuale Übereinstimmung taugt dafür allerdings wenig, da

sie Treffer nicht herausrechnet, die reine Glückssache waren.
Um diesen Zufallseffekt zu kompensieren, nutzt man klassischerweise Cohens κ (Kappa) [11]:

$$\kappa = \frac{p_o - p_e}{1 - p_e}.$$

Dabei steht p_o für die tatsächlich gemessene Übereinstimmung und p_e für die Wahrscheinlichkeit, dass beide Modelle rein zufällig dasselbe geantwortet hätten.
Um das Ergebnis einordnen zu können, hat sich in der Wissenschaft die Skala von Landis und Koch etabliert [26]. Demnach gelten κ-Werte bis 0,20 als *gering*, bis 0,40 als *mäßig*, bis 0,60 als *moderat*, bis 0,80 als *substanziell* und alles darüber als *nahezu perfekt*.
Einen wichtigen methodischen Haken gibt es bei dieser Metrik allerdings: Cohens κ misst wirklich *nur* die Einigkeit (Reliabilität). Zwei Modelle können also einen perfekten Kappa-Wert erzielen, selbst wenn beide bei der Prüfung des Quellcodes völligen Unsinn behaupten, Hauptsache, sie sind sich dabei einig.

2.6.6. Der exakte McNemar-Test

Ein bloßer Unterschied in der Accuracy zweier Verfahren reicht nicht aus, um auf eine tatsächliche Leistungsüberlegenheit zu schließen. Insbesondere bei kleinen Stichproben genügen wenige abweichende Fälle, um eine solche Differenz rein zufällig entstehen zu lassen. Um abzuschätzen, ob der gemessene Abstand lediglich ein Artefakt der spezifischen Fallauswahl ist, bedarf es eines Signifikanztests.
Da in der vorliegenden Arbeit beide Verfahren exakt *dieselben* Fälle beurteilen, liegen *gepaarte* (abhängige) Messungen vor. Ein Signifikanztest für unabhängige Stichproben würde diese Zuordnung ignorieren und damit die zentrale Information verwerfen, an welchen spezifischen Fällen ein Verfahren scheitert. Für dieses Studiendesign ist stattdessen der McNemar-Test [35] vorgesehen. Die Berechnung basiert auf der gepaarten Vierfeldertafel (Tabelle 2.3), in der die Einzelfälle danach klassifiziert werden, ob die Verfahren A und B sie korrekt oder inkorrekt bewertet haben.

Tabelle 2.3.: Schema der gepaarten Vierfeldertafel für den McNemar-Test.

	B richtig	B falsch
A richtig	a	b
A falsch	c	d

Die Felder a und d enthalten die *konkordanten* Paare, also die Fälle, in denen beide Verfahren übereinstimmende Urteile lieferten. Da sie nicht zur Unterscheidung der Verfahren beitragen, fließen sie nicht in die Teststatistik ein. Maßgeblich sind ausschließlich die *diskordanten* Paare (Felder b und c), bei denen genau ein Verfahren das richtige Urteil fand.
Unter der Annahme, dass beide Verfahren gleich gut klassifizieren (Nullhypothese), würden sich diese Streitfälle gleichmäßig auf beide Richtungen verteilen. Die statistische Prüfung entspricht somit einem Binomialtest über die $b + c$ diskordanten Fälle. Da durch den Abzug der konkordanten Paare meist nur eine geringe Fallzahl verbleibt, ist die übliche χ^2-Näherung hier ungeeignet. Daher kommt in dieser Auswertung durchgehend die *exakte* Variante des Tests zur Anwendung (vgl. Abschnitt 6.2.3).

Zu beachten bleibt, dass dieser Test lediglich die Varianz der Fallauswahl absichert und keine Aussage über die absolute Effektgröße trifft. Die zusätzliche Streuung, die durch das nicht-deterministische Antwortverhalten der Sprachmodelle bei identischer Konfiguration entsteht, wird getrennt über Wiederholungsläufe gemessen (vgl. Abschnitt 6.2.4).

2.6.7. Ground Truth und Annotationsgüte

Die *Ground Truth* ist der Referenzdatensatz einer Studie, sie gilt als die „absolute Wahrheit“ und dient als objektiver Maßstab für die Evaluation. Da die spätere Auswertung immer nur so verlässlich sein kann wie dieser zugrundeliegende Datensatz, muss transparent dokumentiert werden, wie er entstanden ist.
In der Praxis gibt es zwei Wege, um an solche Referenzdaten zu kommen:

- **Konstruierte Ground Truth:** Hierbei baut man absichtlich Architekturverstöße in den Quellcode ein. Dieser künstliche Datensatz ist zwar in sich zu 100 % fehlerfrei definiert, hat aber einen entscheidenden Haken: Er spiegelt meist nicht die echte Komplexität und Vielfalt realer Programmierfehler wider.
- **Annotierte Ground Truth:** Hierbei durchsuchen menschliche Experten echten, bestehenden Code manuell nach Verstößen. Damit das Ergebnis verlässlich ist, müssen zwingend mehrere Personen unabhängig voneinander prüfen (was sich dann über das eben erwähnte Cohens κ messen lässt). Ein massives Problem entsteht hier allerdings, wenn die Experten nicht selbst suchen, sondern lediglich die Vorhersagen einer KI überprüfen. Die Ground Truth ist dann nicht mehr unabhängig, sondern stark zugunsten der Maschine verzerrt.

Für die vorliegende Arbeit ist diese Unterscheidung extrem wichtig: Der Labelsatz der Originalarbeit weist genau die zweite Schwäche auf, denn er entstand aus der Überprüfung von Modellvorhersagen und ist deshalb zugunsten der Maschine verzerrt (Anhang A.2.3). Welche Referenz an ihre Stelle tritt, beschreibt Abschnitt 5.3.

3. Literaturrecherche

Während Kapitel 2 die begrifflichen Grundlagen gelegt hat, geht es hier um die Frage, was auf diesem Fundament bereits gebaut wurde, und vor allem was noch fehlt. Der Weg dorthin führt über zwei Stationen. Abschnitt 3.1 schaut sich an, welche KI-gestützten Review-Werkzeuge heute produktiv verfügbar sind und auf welchem Weg sie projektspezifisches Wissen überhaupt hineinbekommen. Abschnitt 3.2 stellt dem die empirische Forschungslage gegenüber und leitet daraus die Forschungslücke ab.

3.1. Übersicht aktueller KI-gestützter Code-Review-Tools

3.1.1. Regelbasierte und KI-gestützte Code-Analyse im Vergleich

Automatisierte Unterstützung im Code-Review ist alles andere als neu. Linter und statische Analysewerkzeuge prüfen Quellcode seit Jahren gegen fest kodierte Regelsätze und sind heute fester Bestandteil jeder CI/CD-Pipeline [8]. Ihre Stärke ist die Verlässlichkeit: Eine Regel greift oder sie greift nicht, das Ergebnis ist jederzeit nachvollziehbar. Ihre Schwäche wurde bereits in Abschnitt 2.3 deutlich: Eine Regel muss sich vorab formalisieren lassen. Eine architektonische Vorgabe wie „Zugriffe auf die Datenbank erfolgen ausschließlich über die Repository-Schicht“ lässt sich zwar noch als Abhängigkeitsregel ausdrücken, die Begründung dahinter aber nicht.
Mit leistungsfähigen Sprachmodellen hat sich diese Grenze verschoben. An die Stelle der vorab formalisierten Regel tritt eine natürlichsprachliche Aufgabenbeschreibung, die das Modell auf den vorliegenden Änderungssatz anwendet. Für den Code-Review im Speziellen legten Zhiyu Li et al. den Grundstein: Ihr Modell *CodeReviewer* wurde gezielt auf Code-Änderungen und Review-Kommentare vortrainiert und übertraf damit allgemeine Code-Modelle deutlich [30]. Wie breit sich dieser Ansatz inzwischen über den gesamten Softwarelebenszyklus ausgedehnt hat, zeigen Hou et al. anhand von 395 Publikationen [16].
Die folgende Übersicht konzentriert sich deshalb nicht darauf, *ob* ein Werkzeug ein LLM einsetzt, da dies mittlerweile den technologischen Standard darstellt. Entscheidend für diese Arbeit ist eine andere Frage: Auf welchem Weg gelangt projektspezifisches Wissen in den Prüfvorgang?

3.1.2. Plattformintegrierte Werkzeuge

GitLab. GitLab bietet zwei technisch grundverschiedene Verfahren an, deren Namen man leicht verwechselt. Das *GitLab Duo Code Review* arbeitet nicht-agentisch: Es bekommt den Diff des Merge Requests, die betroffenen Dateien in ihrem Zustand vor der Änderung sowie Titel und Beschreibung vorgesetzt und erzeugt daraus seine Kommentare [14]. Was das Modell zu sehen bekommt, steht damit schon vor dem Review fest.

Der neuere *Code Review Flow* der GitLab Duo Agent Platform geht den agentischen Weg. In einer vorgelagerten Analysephase leitet er aus den Diffs selbst ab, welchen weiteren Repository-Kontext er braucht, typischerweise Verzeichnislisten sowie zusammenhängende Dateien wie Tests und Abhängigkeiten. Erst danach folgt die eigentliche Urteilsphase über den so zusammengesuchten Kontext. Der Flow kam in GitLab 18.7 als Beta und ist seit 18.8 allgemein verfügbar; die geladene Kontextmenge ist auf rund ein Mebibyte gedeckelt [9].
Beide Verfahren lassen sich mit eigenen Prüfanweisungen versehen, sind also nicht auf einen generischen Auftrag festgelegt. Bei Duo Code Review hinterlegt man diese auf Projekt- oder Gruppenebene [14], der Code Review Flow erwartet eine Datei `mr-review-instructions.yaml` im Wurzelverzeichnis, über die sich Prüfschwerpunkte auch an Dateimuster binden lassen [9]. In beiden Fällen handelt es sich jedoch um frei formulierten Text ohne jeden Bezug zu einer vorhandenen Architekturdokumentation. Dieses Muster wird sich bei allen weiteren Werkzeugen wiederholen.

GitHub. Der Code-Review-Agent von GitHub Copilot zieht nach Herstellerangaben das gesamte Repository zur Kontextbildung heran. Eigene Vorgaben lassen sich gleich auf drei Ebenen hinterlegen: repositoryweit in `.github/copilot-instructions.md`, pfadspezifisch über `.github/instructions/**/*.instructions.md` und werkzeugübergreifend in einer `AGENTS.md` [1]. Auch hier gilt: viel Flexibilität im Format, aber wieder nur Freitext.

3.1.3. Spezialisierte Anbieter und industrielle Eigenentwicklungen

CodeRabbit. CodeRabbit kombiniert die LLM-Analyse mit über 50 quelloffenen Lintern und Sicherheitswerkzeugen. Dazu kommen eine Graph-Analyse der Code-Beziehungen, eine aus vergangenen Review-Konversationen aufgebaute Wissensbasis und pfadbasierte Anweisungen über Glob-Muster [10]. Von allen betrachteten Werkzeugen ist das der ausgereifteste Kontextaufbau.

PR-Agent. PR-Agent ist ein quelloffenes, selbst betreibbares Werkzeug zur Analyse von Pull- beziehungsweise Merge Requests und unterstützt neben GitHub auch GitLab [41]. Es entstand ursprünglich bei Qodo, wurde inzwischen aber an die Open-Source-Gemeinschaft übergeben und dort eigenständig weiterentwickelt. Das kommerzielle Angebot von Qodo ist davon getrennt [41]. Für diese Arbeit ist es das einzige realistische Vergleichsobjekt, da es sich mit eigenen LLM-Zugangsdaten betreiben lässt und damit denselben Restriktionen unterliegt wie die hier verwendete Gateway-Anbindung.

Industrielle Eigenentwicklungen. Neben den kommerziellen und quelloffenen Werkzeugen liefern zwei firmeninterne Systeme durch ihre umfassende Dokumentation und Evaluation im Produktivbetrieb wichtige praktische Erkenntnisse. *BitsAI-CR* von ByteDance arbeitet zweistufig: Ein *RuleChecker* spürt Auffälligkeiten auf, ein nachgeschalteter *ReviewFilter* sortiert die schwachen Meldungen wieder aus. Sun et al. berichten davon eine Präzision von 75 % bei über 12.000 wöchentlich aktiven Nutzern [50]. Interessanter als die Zahl selbst ist die Metrik, die sie dort einführen: Die *Outdated Rate* misst, wie häufig Entwickler einen Kommentar tatsächlich umsetzen [50]. Damit trifft sie ein Problem, das auch diese Arbeit begleitet, denn aus der Korrektheit eines Kommentars folgt noch lange

nicht dessen Nützlichkeit. Der *RovoDev Code Reviewer* von Atlassian wurde in einer großangelegten Online-Evaluation im Bitbucket-Workflow untersucht: 38,70 % der erzeugten Kommentare wurden aufgelöst, die Durchlaufzeit von Pull Requests sank um 30,8 %, die Zahl menschlich verfasster Kommentare um 35,6 % [52].
Eine Einzelheit aus der dortigen Ablationsstudie verdient dabei besondere Aufmerksamkeit, weil sie die Stoßrichtung dieser Arbeit stützt. RovoDev zieht als Kontext den Diff, Titel und Beschreibung des Pull Requests sowie die verknüpften Jira-Vorgänge heran. Genau diese Metadaten tragen jedoch kaum etwas bei, nämlich lediglich ein bis drei Prozentpunkte, und die Autoren führen das darauf zurück, dass solche Informationen zu allgemein gehalten sind, um konkrete Review-Kommentare zu tragen. Die kuratierten Review-Guidelines steuern dagegen fünf Prozentpunkte allein zur Lokalisierungsgenauigkeit bei [52]. Nicht die Menge an Kontext entscheidet also, sondern dessen Verdichtungsgrad, und genau darauf zielt der Einsatz von ADRs in dieser Arbeit ab. Anzumerken bleibt, dass auch diese Guidelines unternehmensweit gepflegt werden. Einen Mechanismus für projektspezifische Vorgaben sieht RovoDev nicht vor [52].

3.1.4. Zwischenfazit: Wie projektspezifisches Wissen heute eingebunden wird

Tabelle 3.1 stellt die betrachteten Werkzeuge entlang genau der Dimension gegenüber, auf die es dieser Arbeit ankommt.

Tabelle 3.1.: KI-gestützte Code-Review-Werkzeuge und ihre Mechanismen zur Einbindung projektspezifischen Wissens.

Werkzeug	Integrationspunkt	Kontextbeschaffung	Mechanismus für projektspezifische Vorgaben
GitLab Duo Code Review [14]	Merge Request	Diff, Dateiinhalte vor der Änderung, Titel/Beschreibung	benutzerdefinierte Anweisungen auf Projekt- und Gruppenebene (Freitext)
GitLab Code Review Flow [9]	Merge Request	agentischer Vorab-Scan des Repositories	`mr-review-instructions.yaml` (Freitext, dateimusterbasiert)
GitHub Copilot Code Review [1]	Pull Request	gesamtes Repository, MCP-Server	`copilot-instructions.md`, pfadspezifische `.instructions.md`, `AGENTS.md`
CodeRabbit [10]	Pull/Merge Request	Code-Graph, Linter, Wissensbasis	pfadbasierte Anweisungen über Glob-Muster
PR-Agent [41]	Pull/Merge Request	Diff, konfigurierbar	Konfigurationsdatei im Repository
BitsAI-CR [50]	internes Review-System	zweistufig: Regelprüfung, Filter	kuratierter interner Regelsatz
RovoDev Code Reviewer [52]	Pull Request (Bitbucket)	Diff, PR-Titel und -Beschreibung, verknüpfte Jira-Vorgänge	unternehmensweite Review-Guidelines; kein projektspezifischer Mechanismus

Die vorangegangene Übersicht zeigt, dass fast alle modernen Werkzeuge Möglichkeiten

zur Konfiguration projektspezifischer Vorgaben bieten. Eine pauschale Klassifizierung als „kontextlose“ Systeme ist demnach nicht mehr zutreffend. Dennoch offenbart eine differenzierte Betrachtung drei zentrale Defizite, die den Ausgangspunkt für den in dieser Arbeit gewählten Ansatz darstellen.[1]

1. **Freitext statt strukturiertem Entscheidungswissen.** Alle Mechanismen erwarten eine frei formulierte Anweisungsdatei. Eine Anbindung an die Architekturdokumentation, die im Repository ohnehin schon liegt, sieht keines der Werkzeuge vor. Wer ADRs pflegt, muss deren Inhalte also von Hand in eine zweite Datei übertragen und hält damit dieselbe Information doppelt vor. Damit entstehen genau die Konsistenzprobleme, die die Documentation-as-Code-Philosophie eigentlich verhindern soll [5].

2. **Kein Bewusstsein für den Entscheidungslebenszyklus.** ADRs kennen Zustände wie *proposed*, *accepted* und *superseded* [37]. Eine Anweisungsdatei kennt diese Unterscheidung nicht. Überholte Vorgaben lassen sich dort nur löschen, und mit ihnen verschwindet die Entscheidungshistorie.

3. **Keine Relevanzauswahl.** Sämtliche hinterlegten Anweisungen wandern in den Kontext, ganz gleich ob sie für den vorliegenden Änderungssatz überhaupt einschlägig sind. Mit wachsender Zahl an Vorgaben wird das zum Problem, denn wie in Abschnitt 2.2.8 gezeigt, verlieren LLMs durch irrelevanten Kontext messbar an Präzision [47] und verwerten Informationen in der Mitte langer Eingaben deutlich schlechter [32].

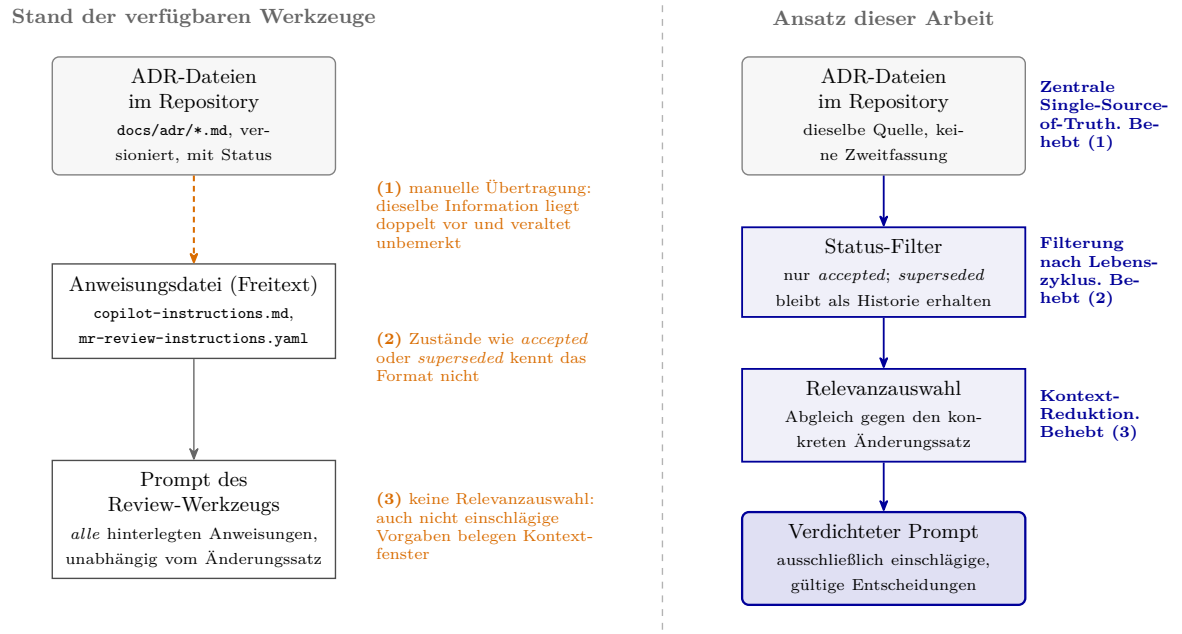


Abbildung 3.1.: Einbindung projektspezifischen Wissens in den Review-Prozess. Gegenüberstellung bestehender Werkzeuge (links) und des in dieser Arbeit verfolgten Ansatzes (rechts).

[1] Stand der geprüften Herstellerdokumentation: Juli 2026.

Abbildung 3.1 stellt beide Wege gegenüber. Links wird das ADR-Wissen von Hand in eine zweite Datei überführt und wandert von dort ungefiltert in den Prompt. Rechts bleibt das ADR die einzige Quelle, wird aber vor der Übergabe zweimal gefiltert, einmal nach Status und einmal nach Relevanz für den Änderungssatz.
Ein wesentlicher Mangel des gegenwärtigen Stands ist das gänzliche Fehlen unabhängiger empirischer Evaluationen für diese Mechanismen. Ob hinterlegte Architekturvorgaben die Review-Qualität verbessern und wie sich dieser Effekt quantifizieren lässt, ist bislang nicht hinreichend belegt.

3.2. Aktueller Stand der Forschung im Bereich agentischer Code-Reviews

3.2.1. Einführung agentischer Ansätze

Die frühe Forschung behandelte den automatisierten Code-Review als reines Übersetzungsproblem: Ein Modell bekommt einen Diff und erzeugt daraus einen Kommentar oder eine überarbeitete Fassung des Codes [30]. Khan et al. ordnen diese Entwicklung in ihrer Übersicht über 99 Publikationen aus den Jahren 2015 bis 2025 ein und stellen eine deutliche Verschiebung hin zu generativen Ende-zu-Ende-Verfahren fest. Zugleich bemängeln sie, dass es an konsolidierten Bewertungsmaßstäben fehlt [22], ein Punkt, auf den diese Arbeit später zurückkommt.
Der Übergang zu agentischen Systemen hängt am ReAct-Muster von Yao et al., das in Abschnitt 2.2 bereits beschrieben wurde [59]. Die breite Entwicklung agentischer Systeme im Bereich des Software Engineering wird von Liu et al. anhand von 124 Publikationen systematisiert [31]. Für den Code-Review selbst ist vor allem *CodeAgent* von Tang et al. relevant, ein Mehr-Agenten-System, in dem ein überwachender *QA-Checker* dafür sorgt, dass die beteiligten Rollen die ursprüngliche Prüffrage nicht aus den Augen verlieren [51]. Genau diese Aufteilung greift der in dieser Arbeit entwickelte Aufbau aus einem lokalisierenden und einem urteilenden Agenten wieder auf (siehe Abschnitt 5.1).

3.2.2. Empirische Befunde zum produktiven Einsatz

Die rasche Verbreitung agentischer Review-Werkzeuge steht jedoch in einem deutlichen Kontrast zu einer ernüchternden empirischen Befundlage. Grundlage gleich mehrerer aktueller Untersuchungen ist der AIDev-Datensatz von Hao Li et al., der 932.791 von autonomen Agenten erzeugte Pull Requests aus 116.211 Repositories umfasst [28]. Chowdhury et al. stellen darauf aufbauend fest: Pull Requests, die ausschließlich von Review-Agenten begutachtet wurden, werden um 23,17 Prozentpunkte seltener gemerged, und bei über 60 % der verworfenen Fälle liegt das Verhältnis von Signal zu Rauschen unter 30 % [7]. Zhong et al. kommen auf einer davon unabhängigen Datenbasis von 278.790 Review-Konversationen aus 300 Projekten zum selben Schluss: Agenten-Vorschläge werden seltener übernommen als die menschlicher Reviewer und über die Hälfte der abgelehnten Vorschläge ist entweder schlicht falsch oder wird anders gelöst [63].
Zhang et al. gießen diese Beobachtungen in einen Benchmark. Ihr aus echten menschlichen Reviews konstruierter Datensatz *c-CRAB* zeigt zweierlei: Gängige Review-Agenten lösen nur rund 40 % der Aufgaben, und die von ihnen erzeugten Reviews betonen systematisch andere Aspekte als die menschlichen Vergleichsreviews [60]. Vor allem der zweite

Befund stützt die inzwischen breit geteilte Schlussfolgerung, dass agentische Reviews die menschliche Begutachtung ergänzen, aber derzeit nicht ersetzen können [7, 63].
Nun berichten die industriellen Eigenentwicklungen deutlich positivere Zahlen [50, 52]. Dieser Widerspruch ist kein Zufall, sondern eine Frage des Untersuchungsaufbaus. Die akademischen Studien betrachten generische Agenten quer über heterogene Open-Source-Projekte hinweg. BitsAI-CR und RovoDev sind dagegen auf einen eng umrissenen Kontext zugeschnitten und um Filterstufen ergänzt, die die Präzision auf Kosten der Abdeckung nach oben ziehen. Daraus folgt die vielleicht wichtigste Erkenntnis dieses Abschnitts: Der Nutzen agentischer Reviews hängt weniger vom eingesetzten Modell ab als von der Passung zwischen Werkzeug und Einsatzkontext. Damit rückt genau das ins Zentrum, was diese Arbeit untersucht, nämlich die Frage, wie man einem Agenten den richtigen projektspezifischen Kontext beschafft.

3.2.3. Architekturkonformität als Sonderfall der Prüfung

Die Prüfung von Code gegen architektonische Vorgaben ist ein eigenes Forschungsfeld, das lange vor dem Aufkommen von LLMs bearbeitet wurde. Knodel und Popescu vergleichen die drei etablierten Verfahrensklassen der statischen Konformitätsprüfung, also Reflexionsmodelle, Beziehungskonformitätsregeln und Komponentenzugriffsregeln, entlang von 13 Dimensionen [23]. Passos et al. liefern dazu einen anwendungsnahen Überblick [39]. Wie in Abschnitt 2.3 beschrieben, teilen alle diese Verfahren dieselbe konstruktionsbedingte Grenze: Sie sehen nur, was sich als strukturelle Abhängigkeit ausdrücken lässt.
Um diese Limitierungen zu überwinden, eignen sich LLM-basierte Verfahren. Die beiden für die vorliegende Arbeit entscheidenden Studien zeigen, dass diese Methodik in der Praxis erfolgreich anwendbar ist. Ruiyin Li et al. untersuchen die automatisierte Erkennung von Verletzungssymptomen der Architekturerosion aus Review-Kommentaren: GPT-4o übertrifft dort mit einem F1-Wert von 0,851 die besten klassischen Verfahren, und in einem ergänzenden Experiment stieg die Erkennungsleistung der beteiligten Entwickler von rund 26 % auf 65 %, sobald ihnen die erkannten Symptome vorlagen [29]. Su et al. übertragen diesen Ansatz auf ADRs und prüfen anhand von 980 ADRs aus 109 Repositories, wie zuverlässig LLMs Verletzungen dokumentierter Architekturentscheidungen erkennen. Ihr Befund ist zweigeteilt: sehr treffsicher bei expliziten, direkt aus dem Code ableitbaren Entscheidungen, deutlich schwächer bei impliziten oder deployment-bezogenen [49].
Diese zweite Studie ist der unmittelbare Ausgangspunkt der vorliegenden Arbeit. Ihr Replication-Package dient in Kapitel 6 als Vergleichsbasis, an der sich der eigene agentische Ansatz messen lassen muss.

3.2.4. Forschungslücke

Die kritische Betrachtung des aktuellen Forschungsstandes offenbart drei wesentliche Defizite, an welche die Forschungsfragen aus Abschnitt 1.3 direkt anknüpfen. Zur systematischen Einordnung visualisiert Abbildung 3.2 die diskutierten Ansätze anhand der beiden Dimensionen, die den Kern dieser Arbeit bilden.

Die Art der Evidenzbeschaffung wurde nie als Variable behandelt Su et al. evaluieren die Erkennung von ADR-Verletzungen ausschließlich in einer statischen, retrievalbasierten Konfiguration [49]. Ob ein agentischer Zugriff, bei dem sich das Modell den Suchraum selbst erschließt, zu anderen Ergebnissen führt, hat für diese Aufgabe bislang



Abbildung 3.2.: Einordnung der besprochenen Arbeiten nach Art der Evidenzbeschaffung und Art der Kontextquelle. Der Quadrant oben rechts ist bislang unbesetzt und markiert die Forschungslücke.

niemand untersucht. Umgekehrt betrachten die einschlägigen empirischen Arbeiten zu agentischen Reviews [7, 63, 60] zwar Agenten, aber nur deren allgemeine Review-Qualität und nicht die Konformität gegenüber dokumentierten Architekturentscheidungen. Beide Stränge laufen also aneinander vorbei, und in dieser Lücke liegt Forschungsfrage 1.

Strukturierte Architekturdokumentation wird als Kontextquelle nicht genutzt Die verfügbaren Werkzeuge bieten Freitext-Anweisungsdateien an (Tabelle 3.1), aber keinen Mechanismus, der vorhandene ADRs unter Berücksichtigung ihres Status und ihrer Relevanz für den konkreten Änderungssatz einbindet. Dass die Art der Kontextaufbereitung überhaupt einen Unterschied macht, legen die Distraktor-Befunde von Shi et al. [47] und die Positionseffekte von Liu et al. [32] deutlich nahe. Die Feldstudie von Aðalsteinsson et al. weist in dieselbe Richtung, denn die beteiligten Entwickler bevorzugten retrievalgestützt aufbereitete Reviews sogar gegenüber menschlichen [2]. Weil dort allerdings beide Prototypen dieselbe Kontextbeschaffung verwendeten, lässt sich der Beitrag der Anreicherung nicht isolieren. Der kausale Effekt der Kontextaufbereitung auf die Reviewqualität bleibt damit offen [2], denn ein gemessener Vorsprung kann ebenso gut an der veränderten Aufgabenformulierung wie an der tatsächlich besseren Evidenzlage liegen. Genau diese Trennung leistet Forschungsfrage 2.

Es fehlt eine belastbare Evaluationsgrundlage Khan et al. konstatieren das Fehlen konsolidierter Bewertungsmaßstäbe [22]. Für die spezifische Frage der ADR-Konformitätsprüfung existiert mit dem Datensatz von Su et al. immerhin eine Grundlage, deren Reproduzierbarkeit und Belastbarkeit aber nie unabhängig geprüft wurde. Diese Arbeit

holt das nach. Dieser Punkt ist keine eigene Forschungsfrage, sondern die Voraussetzung dafür, dass sich die Antworten auf Forschungsfrage 1 und 2 überhaupt belastbar messen lassen. Für Forschungsfrage 3, also für die praxisgerechte Einbettung eines solchen Verfahrens in den Merge-Request-Workflow und für dessen Grenzen, liefert die Literatur bislang nur indirekte Anhaltspunkte: die Übernahmequoten der Feldstudien [50, 52], die Präferenzbefunde von Aðalsteinsson et al. [2] und das dokumentierte Misstrauen der Entwickler gegenüber KI-Ausgaben [38]. Eine Untersuchung eines mit Architekturwissen versorgten Agenten im laufenden DevOps-Prozess fehlt dagegen vollständig.

4. Anforderungsanalyse

TODO: Die Anforderungen müssen überabeitet werden, sobald das Kapitel technisccche Umsetzung fertig ist. Dieses Kapitel leitet die funktionalen und nichtfunktionalen Anforderungen an das System ab. Anschließend bestimmt es, welche Metriken für dessen Bewertung erhoben werden.

4.1. Anforderungen an das System

Die Anforderungen gehen auf drei Quellen zurück: die Grenzen des menschlichen Reviews (Abschnitt 2.1.3, FA-1 bis FA-4), die Lücke bestehender Werkzeuge (Abschnitt 3.2.4, FA-5 und NFA-3) sowie die organisatorischen Rahmenbedingungen des Unternehmenskontexts (NFA-1 und NFA-5). Sie legen damit aber nicht nur fest, was das System können soll. Zugleich fixieren sie die Versuchsbedingungen des Vergleichs in Kapitel 6: Von den elf Anforderungen an den Prüfkern beschreibt eine einzige die untersuchte Variable (FA-6), die übrigen zehn sind die konstant gehaltenen Bedingungen, unter denen sie gemessen wird.
Die funktionalen Anforderungen sind nach den beiden Bestandteilen des Systems getrennt. Der **Prüfkern** entscheidet, ob eine einzelne Architekturregel auf einem gegebenen Codestand eingehalten ist; auf ihm beruhen sämtliche Messungen. Die **Integration in den Entwicklungsprozess** bringt dieses Urteil an den Merge Request. Sie ist als Proof-of-Concept ausgelegt. Ein erfüllter Punkt aus dem ersten Block ist damit durch eine Messung belegt, einer aus dem zweiten lediglich durch eine Demonstration.

4.1.1. Funktionale Anforderungen

Die folgenden Anforderungen beschreiben den Weg von der dokumentierten Architekturentscheidung bis zum belegten Einzelurteil über eine Regel.

FA-1 – Zerlegung in atomare Prüfregeln Das System muss ein ADR vor der Prüfung in atomare, einzeln entscheidbare Regeln zerlegen. Die Prüfeinheit ist damit nicht das Dokument und nicht der Merge Request, sondern das Paar aus einer Regel und einem Codestand (Abschnitt 4.2.1).

FA-2 – Validierungsoptimierte Aufbereitung des Architekturwissens Das System muss die im Repository abgelegten ADRs (Abschnitt 2.4) als Prüfgrundlage heranziehen, sie dem Modell aber nicht unverändert übergeben. Der maschinell auswertbare Anteil der Entscheidung ist von ihrer für Menschen geschriebenen Historie zu trennen, Letztere zu entfernen. Grundlage ist der Befund zum Distraktor-Kontext aus Abschnitt 2.2.8.

FA-3 – Regelbasis mit übergeordnetem Rückfall Ist im Projekt selbst kein Architekturwissen hinterlegt, muss auf eine organisationsweite Regelbasis zurückgegriffen werden

können. Damit ist ein Repository ohne Vorbereitung prüfbar, und die Koexistenz von abteilungsweiten Basisregeln und dienstspezifischen Vorgaben ist abgebildet.

FA-4 – Selektive Regelauswahl Das System muss vor der eigentlichen Prüfung entscheiden, welche Regeln für die geänderten Dateien überhaupt in Frage kommen, und die übrigen ausschließen. Auch dies folgt aus Abschnitt 2.2.8: Irrelevanter Kontext verschlechtert die Urteilsqualität messbar.

FA-5 – Kontrollierter Suchraum Der Ausschnitt des Quellbaums, auf den die Prüfung zugreifen darf, muss explizit festgelegt und für alle verglichenen Ansätze identisch sein. Die Architekturdokumente selbst dürfen nicht Teil davon sein, da sich die zu prüfende Regel sonst als eigener Beleg wiederfindet. Der Zuschnitt bestimmt zugleich mit, welche Regeln überhaupt entscheidbar sind: 60 % der als unentscheidbar annotierten Fälle haben ihren Beleg außerhalb des gewählten Suchraums (Anhang A.3.2).

FA-6 – Eigenständige Evidenzbeschaffung Das System muss über die eingereichte Änderung hinaus gezielt weitere Codestellen nachschlagen können, da Architekturverletzungen sich oft erst im Zusammenspiel mehrerer Dateien zeigen. Der Zugriff erfolgt ausschließlich lesend auf einem eingefrorenen Stand des Quellbaums, wobei Auffinden und Lesen getrennte Schritte sind. Diese Anforderung ist die unabhängige Variable der Arbeit: Ihr Vorhandensein unterscheidet den agentischen vom statischen Ansatz.

FA-7 – Begrenztes Werkzeugbudget mit erzwungenem Endurteil Die eigenständige Suche muss nach oben begrenzt sein. Ist das Budget erschöpft, muss das System ein definiertes Urteil abgeben, statt abzubrechen, da Laufzeit und Aufwand je Regel sonst nicht kalkulierbar sind. Die geeignete Höhe des Budgets untersucht Abschnitt 6.6.

FA-8 – Dreiwertiges Urteil mit Belegpflicht Das System muss je Regel ein eindeutiges Urteil abgeben und dabei neben Konformität und Verstoß auch den Fall abbilden, dass die Evidenz für eine Entscheidung nicht ausreicht. Jedes Urteil ist mit Begründung und konkreter Belegstelle zu versehen; insbesondere darf ein Konformitätsurteil nicht ohne benannte Fundstelle ergehen.

FA-9 – Nachgelagerte Validierung des Urteils Das Urteil des Prüfers muss durch eine zweite Instanz gegengeprüft werden, die auf derselben Evidenzgrundlage arbeitet. Genau daran scheitert die Ableitung von y_{true} im Originaldatensatz: Dort erhalten die prüfenden Modelle das Urteil samt Begründung vorgelegt, statt unabhängig zu urteilen (Anhang A.2.3).

FA-10 – Maschinenlesbare Ergebnis- und Verlaufsausgabe Je geprüfter Regel müssen Urteil, Begründung, Belegstelle und der vollständige Verlauf der Evidenzbeschaffung maschinenlesbar vorliegen. Fälle ohne verwertbares Ergebnis sind als solche zu kennzeichnen. Dieses Format ist Voraussetzung für die Auswertung in Abschnitt 5.5 und die Nennerkonvention in Abschnitt 5.5.

FA-11 – Vergleichs- und Stapelbetrieb Das System muss dieselbe Regelmenge über verschiedene Strategien der Evidenzbeschaffung und über ganze Datensätze hinweg ausführen können, ohne dass sich etwas anderes als die untersuchte Variable ändert. Ein Lauf muss sich zudem auf eine benannte Teilmenge beschränken lassen. Ohne diese Anforderung wären weder die Wiederholungsläufe (Abschnitt 6.2.4) noch die Prompt-Variante bei sonst identischer Konfiguration (Abschnitt 6.4) durchführbar gewesen.

4.1.2. Funktionale Anforderungen an die Integration

Die folgenden Anforderungen betreffen den Weg des fertigen Urteils zu den Entwickelnden. Sie werden vom Proof-of-Concept bedient, tragen aber keine der berichteten Messungen.

FA-12 – Anbindung an den Merge-Request-Workflow Das System muss ereignisgesteuert bei Merge-Request-Ereignissen auslösen und sich in bestehende CI/CD-Prozesse einfügen, ohne deren Laufzeit zu blockieren (vgl. Abschnitt 2.5).

FA-13 – Verlustfreie Aufnahme des Änderungssatzes Das System muss die Änderungen eines Merge Requests vollständig einlesen, auch bei umfangreichen Diffs. Generierte oder irrelevante Artefakte wie Sperrdateien der Paketverwaltung oder Binärformate sind herauszufiltern, bevor sie in den Modellkontext gelangen.

FA-14 – Rückmeldung im Quellcodeverwaltungssystem Die erzeugten Befunde müssen direkt am betroffenen Merge Request platziert und dabei der verletzten Architekturentscheidung eindeutig zugeordnet werden.

4.1.3. Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen beschreiben die Qualitätsmerkmale, von denen Akzeptanz und Betreibbarkeit des Systems abhängen.

NFA-1 – Modell- und Anbieterunabhängigkeit Der Betrieb muss über eine generische Schnittstelle möglich sein, ohne Bindung an ein bestimmtes Sprachmodell, ohne Feinabstimmung eines eigenen Modells und ohne eigene Beschleunigerhardware. Das Referenzsystem aus Abschnitt 5.2 setzt genau dies voraus und ist deshalb nicht ohne Weiteres nachnutzbar.

NFA-2 – Robustheit gegenüber nicht formatgerechten Antworten Das System muss fehlerhafte Eingaben und nicht formatgerechte Modellantworten abfangen, ohne abzubrechen. Fehlschläge müssen dabei als solche erkennbar bleiben und dürfen nicht stillschweigend als gültiges Urteil in die Auswertung eingehen (Abschnitt 5.5).

NFA-3 – Nachvollziehbarkeit Jeder Zwischenschritt, insbesondere jeder Werkzeugaufruf und dessen Rückgabe, muss maschinell auswertbar protokolliert werden. Sämtliche Aussagen über den *Mechanismus* hinter dem gemessenen Unterschied beruhen ausschließlich auf diesen Protokollen (Abschnitte 6.3.1 bis 6.3.3).

NFA-4 – Reproduzierbarkeit im erreichbaren Rahmen Vergleichsläufe müssen unter identischen Bedingungen wiederholbar sein, was einen eingefrorenen Quellbaum, einen unveränderlichen Regelsatz und eine dokumentierte Modell- und Parameterkonfiguration erfordert. Da die Modellausgabe selbst nicht deterministisch ist, zielt die Anforderung auf die Kontrolle aller übrigen Bedingungen. Der verbleibende Rest wird über Wiederholungsläufe gemessen (Abschnitt 6.2.4).

NFA-5 – Vertraulichkeit der Codedaten Die Modellinteraktion muss über einen organisationsintern betriebenen Zugang erfolgen, sodass Quellcode die Kontrollsphäre der Organisation nicht verlässt. Das schließt die in Abschnitt 3.1 betrachteten geschlossenen Dienste aus.

NFA-6 – Minimalrechte der Werkzeuge Die dem Modell zugänglichen Werkzeuge dürfen ausschließlich lesend arbeiten und müssen auf den Projektbaum beschränkt bleiben; ein allgemeiner Kommandozeilenzugriff scheidet aus.

NFA-7 – Beschränkter und messbarer Aufwand Modellaufrufe, Tokenverbrauch und Laufzeit müssen je Regel erfassbar und nach oben begrenzt sein. Die Latenz muss niedrig genug bleiben, dass das Feedback noch innerhalb des laufenden Review-Vorgangs verwertbar ist.

NFA-8 – Unabhängigkeit von Repositorygröße und Umfang der Änderung Weder die Größe der Codebasis noch der Umfang eines Änderungssatzes dürfen zu einem harten Abbruch führen. Das schließt aus, den vollständigen Quellcode oder den vollständigen Diff in den Modellkontext zu laden.

NFA-9 – Inbetriebnahme ohne Projektvorbereitung Die Aufnahme eines weiteren Repositorys darf keine Anpassung an diesem Repository voraussetzen, da der Betriebsaufwand sonst jede Ausweitung über das Pilotprojekt hinaus begrenzt.

NFA-10 – Nützlichkeit und Akzeptanz Die erzeugten Befunde müssen relevant, sachlich korrekt und konstruktiv sein. Entscheidend ist dabei weniger die Zahl gefundener Verstöße als die Rate der Fehlalarme.

NFA-11 – Verständlichkeit Der Verweis auf die verletzte Entscheidung und die konkrete Belegstelle muss ausreichen, um die notwendige Änderung ohne zusätzliche Recherche abzuleiten.

NFA-12 – Wartbarkeit und Erweiterbarkeit Regelaufbereitung, Evidenzbeschaffung, Urteilsbildung und Ausgabeaufbereitung sind klar zu trennen und einzeln austauschbar zu halten. Erst das erlaubt es, künftig weitere Analysearten neben der Architekturkonformität aufzunehmen (Abschnitt 8.2.1).

4.1.4. Bewusst nicht aufgenommene Anforderungen

Fünf für ein produktives Review-System naheliegende Punkte sind bewusst nicht als Anforderung geführt, da sie im Rahmen dieser Arbeit weder umgesetzt noch nachgewiesen werden.

- **Statusgebundene Regelauswahl.** Der Prototyp wertet den ADR-Status nicht aus, stützt sich also nicht ausschließlich auf ADRs mit dem Status *accepted*. Der Punkt ist als offene Frage in Abschnitt 7.4 beschrieben. **TODO: wenn noch zeit bleibt, wird das auch implementiert**

- **Priorisierung nach Schweregrad.** Eine Sortierung der Befunde nach Kritikalität setzt ein Schweregradmodell voraus, das weder in den ADRs noch im Prüfkern existiert.

- **Sprachspezifische Analyse.** Der Prüfkern arbeitet rein textuell und damit sprachunabhängig, kann syntaktische Eigenheiten einzelner Sprachen aber nicht ausnutzen. Eine nach Sprachen getrennte Auswertung findet nicht statt.

- **Erfassung von Rückmeldungen der Entwickelnden.** Der Prototyp bietet keine Möglichkeit, einen einzelnen Befund als zutreffend oder fehlerhaft zu bewerten. Er prüft ausschließlich ADRs und meldet das Ergebnis am Merge Request zurück. Eine solche Erhebung setzt einen produktiven Einsatz über mehrere Teams voraus, den diese Arbeit nicht leisten kann.

- **Automatisierte Testbarkeit.** Eine Abdeckung durch Unit- und Integrationstests besteht nicht. Die Absicherung erfolgte empirisch über die Auswertung ganzer Läufe gegen eine Referenz.

4.2. Auswahl der Evaluationsmetriken

Der vorangegangene Abschnitt beschreibt, was das System leisten soll. Dieser Abschnitt klärt, woran sich messen lässt, ob es das auch tut. Die formalen Definitionen der genannten Maße stehen bereits in Abschnitt 2.6 und werden hier nicht wiederholt, ihre konkrete Berechnung beschreibt Abschnitt 5.5. Hier geht es allein um die Frage, warum gerade diese Maße erhoben werden und andere eben nicht.

4.2.1. Messgegenstand

Bewertet wird ein einzelnes Urteil: Hält der Code eine bestimmte Regel ein oder nicht, also *C*, *NC* oder *CIA*. Eine Messung ist damit ein Paar aus einer Regel und einem eingefrorenen Codestand. Der Grund dafür ist einfach. Nur für dieses Urteil gibt es eine Referenz (Abschnitt 5.3), an der sich Richtigkeit festmachen lässt. Wie wenig selbstverständlich das ist, zeigt Anhang A.2.3: Eine vorhandene Referenz ist noch keine unabhängige. Wie gut eine Rückmeldung formuliert ist, bleibt hier ungemessen, denn dafür fehlt ein vergleichbarer Maßstab.
Zwischen den Läufen ändert sich nur eines, die Art der Evidenzbeschaffung (FA-6). Modell, Regelsatz, Urteilskategorien und Datei-Suchraum bleiben gleich. Die Maße müssen also auf genau diesen Unterschied ansprechen und nicht auf Nebeneffekte.

4.2.2. Begründung der gewählten Maße

Accuracy Su et al. berichten durchgängig die Accuracy und sonst kaum etwas [49]. Sie wird deshalb miterhoben, denn nur über sie lassen sich die eigenen Läufe an den publizierten Wert anschließen. Für sich genommen sagt sie hier wenig, weil die Klassen

ungleich verteilt sind (Abschnitt 2.6) und sich eine hohe Quote schon mit der häufigsten Klasse erreichen lässt. Sie steht deshalb nie allein, sondern immer neben mindestens einem klassenweisen Maß.

Recall je Klasse Die drei Klassen stehen für unterschiedliche Fähigkeiten, deshalb wird der Recall für jede getrennt ausgewiesen. Beim *NC*-Recall geht es um den übersehenen Verstoß, also um den Fehler, dessen Vermeidung das System überhaupt erst rechtfertigt. Der *CIA*-Recall ist für diesen Vergleich noch wichtiger. Er zeigt, ob ein System merkt, dass seine Evidenz nicht ausreicht, statt zu raten, und genau darin sollten sich vorab fixierter Kontext und selbstgesteuerte Suche unterscheiden. Ein zusammenfassender Wert würde die Unterschiede zwischen den Klassen wieder einebnen.

Precision je Klasse *C* ist mit rund 59 % der Fälle in y_{true} und 60 % in der unabhängigen Referenz die häufigste Klasse. Ohne die Precision von *C* bliebe ein System unauffällig, das seine Trefferquote vor allem aus vorschnellen Konformitätsurteilen zieht. Die Precision von *NC* beziffert die Fehlalarme und ist damit die Messgröße zu NFA-10. Ein Verfahren mit zu vielen Fehlalarmen lässt sich nicht in einen Merge-Request-Workflow einbauen, was direkt die dritte Forschungsfrage betrifft (Abschnitt 1.3).

Makro-Mittelung und F_1 Um zwei Konfigurationen nebeneinanderstellen zu können, braucht es trotzdem eine einzelne Zahl. Die Wahl fällt auf die Makro-Mittelung, weil die seltenen Klassen dort nicht von *C* überdeckt werden. Zusätzlich wird der Abstand zwischen makro-gemitteltem und gewichtetem F_1-Wert berichtet. Ist er groß, stammt die Leistung überwiegend aus der häufigsten Klasse, und genau das zeigt die Accuracy allein nicht.

Matthews Correlation Coefficient Die Klassen sind deutlich schief verteilt. In der maßgeblichen Zufallsstichprobe stehen 36 *C*- und 19 *CIA*-Fällen nur 5 *NC*-Fälle gegenüber; in y_{true} sind es etwa 59 % *C*, 21 % *NC* und 20 % *CIA*. Bei dieser Verteilung ist der MCC das Korrektiv zur Accuracy. Laufen beide auseinander, beruht ein scheinbarer Vorsprung eher auf der Klassenverteilung als auf besseren Urteilen.

Übereinstimmung der Verfahren untereinander Zwei Verfahren können fast gleich genau sein und trotzdem bei ganz anderen Regeln richtig liegen. Korrektheit und Übereinstimmung sind also verschiedene Dinge, und Letztere wird getrennt erhoben, über die gepaarte Vierfeldertafel beider Verfahren, aus der sich übereinstimmende und abweichende Fälle direkt ablesen lassen (Abschnitt 6.2.3). Sie ist zugleich die Grundlage des Signifikanztests und der Frage, in welche Richtung die Verfahren voneinander abweichen (Abschnitt 6.3.3). Cohens κ [11] misst dasselbe als eine einzige Zahl und wird deshalb nicht berichtet, denn für den Vergleich ist die Tafel nützlicher, weil sie die *Richtung* der Abweichungen erhält.

Streuungs- und Signifikanzmaße Die Stichprobe ist klein, 60 Fälle im maßgeblichen Zufallsteil und 81 insgesamt. Ohne Angabe zur Streuung wären Unterschiede von wenigen Prozentpunkten nicht vom Zufall zu trennen. Erhoben werden deshalb zwei Größen für zwei verschiedene Fehlerquellen. Der **exakte McNemar-Test** [35] prüft, ob ein beobachteter Unterschied zwischen zwei Verfahren trägt; er passt hier, weil beide auf denselben

Fällen laufen und die Vergleiche damit gepaart sind (Abschnitt 2.6.6). Eine zweite Quelle deckt er nicht ab: Die Modelle sind auch bei festen Parametern nicht deterministisch, zwei Läufe derselben Konfiguration fallen also auseinander. Diese Streuung wird über **Wiederholungsläufe** bei identischer Konfiguration gemessen (Abschnitt 6.2.4). Deren Standardabweichung gibt die Schwelle, ab der eine Differenz als Differenz zählt.

Aufwand der Evidenzbeschaffung Neben der Urteilsgüte wird erfasst, was ein Urteil kostet. Das sind keine Klassifikationsmetriken, ohne sie lässt sich ein Genauigkeitsgewinn aber nicht einordnen, denn der agentische Ansatz beschafft seine Evidenz über wiederholte Modellaufrufe. Gemessen wird auf zwei Ebenen. Beim **Werkzeugverhalten** zählt das Auswertungsskript je Fall die Aufrufe von `list_files`, `grep` und `read_file`, die erfolglosen Suchen und die Menge des gefundenen Materials. Diese Zahlen sind zugleich die Datengrundlage der Mechanismus-Analyse, denn erst mit ihnen lässt sich unterscheiden, ob ein Ausweichurteil daher kommt, dass gar nicht gesucht wurde, oder daher, dass die Suche nichts fand (Abschnitte 6.3.1 bis 6.3.3). Bei den **Betriebskosten** stehen Tokenverbrauch, Modellaufrufe je Regel und Laufzeit (Anhang A.3.4). Was Aufwand und Nutzen miteinander zu tun haben, zeigt allerdings erst die Budget-Ablation, die das Werkzeugbudget variiert und die Genauigkeit dagegen aufträgt (Abschnitt 6.6). Zusammen belegen diese Größen NFA-7 und NFA-8.

Bewusst nicht erhobene Maße ROC- und Precision-Recall-Kurven entfallen, weil die Modelle harte Labels liefern und keine kalibrierten Konfidenzwerte. Es gibt also keinen Schwellenwert, der sich variieren ließe. Die Mikro-Mittelung fällt bei genau einem Label je Fall mit der Accuracy zusammen und bringt nichts Zusätzliches. Der F_1-Score wird berichtet, ist hier aber nicht das Leitmaß, denn bei dieser Klassenverteilung sagt der MCC mehr.

5. Versuchsaufbau

Die zentrale Hypothese dieser Arbeit besagt, dass ein Prüfsystem Architekturverstöße zuverlässiger erkennt, wenn es sich die notwendige Evidenz aus dem Quellcode eigenständig beschafft. An einem produktiven System lässt sich dies nicht direkt nachweisen. Das vorliegende Kapitel beschreibt daher einen kontrollierten Versuchsaufbau zur Isolierung genau dieses Wirkmechanismus.
Dieses Kapitel definiert die methodischen Rahmenbedingungen. Dazu gehören die implementierten Verfahren (Abschnitt 5.1), der verwendete Datensatz (Abschnitt 5.2) und die Referenz (Abschnitt 5.3). Darauf bauen die Basiskonfiguration (Abschnitt 5.4) und die Metrikberechnung (Abschnitt 5.5) auf. Abschnitt 5.6 erläutert das Prinzip der Experimentreihe. Die Ergebnisse dieser Reihe befinden sich in Kapitel 6.
Dieser Aufbau dient der systematischen Beantwortung der Forschungsfragen. Der Hauptvergleich zwischen statischer und agentischer Evidenzbeschaffung adressiert die erste Forschungsfrage. Für die zweite Frage erhält die Baseline einen angepassten Prompt. Dadurch lässt sich ein reiner Formulierungseffekt vom eigentlichen Mehrwert der selbst beschafften Evidenz trennen. Die dritte Forschungsfrage zur Integration wird in Kapitel 7 beantwortet.
Beide Verfahren wurden eigens implementiert, um alle methodischen Anforderungen zu erfüllen. Die Art der Evidenzbeschaffung stellt die zentrale unabhängige Variable dar. Daher muss der Kontextaufbau vollständig kontrollierbar sein und jeder Werkzeugaufruf maschinenlesbar protokolliert werden. Bestehende Dienste oder etablierte Frameworks eignen sich hierfür nicht. Sie kapseln die Ausführung intern und verwehren somit den für diese Untersuchung zwingend erforderlichen Zugriff.

5.1. Die beiden verglichenen Verfahren

Verglichen werden zwei Verfahren, die dieselbe Aufgabe lösen und sich in genau einer Eigenschaft unterscheiden, nämlich darin, wie sie an die Codeevidenz gelangen, auf der ihr Urteil beruht (FA-6). Beide erhalten denselben Regeltext, arbeiten auf demselben Codestand und geben dasselbe dreiwertige Urteil ab.
Dieser Regeltext ist der Eingang beider Abläufe und deshalb in beiden folgenden Abbildungen der erste Kasten. Er entsteht in keinem der beiden Verfahren, sondern ist bereits Teil des Datensatzes: Su et al. haben den *Decision*-Abschnitt jeder ADR-Datei über einen vorgelagerten LLM-Aufruf in einzeln entscheidbare Aussagen zerlegt (FA-1) und diese Regeltexte werden hier unverändert übernommen, statt sie neu zu erzeugen (Tabelle 5.2). Der Vergleich setzt also erst hinter der Regelgewinnung an und ein Unterschied zwischen den Verfahren kann nicht aus unterschiedlich formulierten Regeln stammen.

5.1.1. Baseline: der statische RAG-Judge

Als Baseline dient das Verfahren der Originalarbeit [49], nachgebildet und über denselben Modellzugang betrieben wie der agentische Judge. Es ist zugleich der Bezugspunkt, gegen

den alle Experimente in Kapitel 6 gemessen werden. Es folgt dem klassischen Muster der Retrieval-Augmented Generation [27], wie es Abschnitt 2.2.4 beschreibt:

- Der Quellcode wird vorab in Chunks von 800 Zeichen mit 200 Zeichen Überlappung zerlegt.
- Die Chunks werden mit dem Sentence-Transformer-Modell *all-MiniLM-L6-v2* [42] vektorisiert.
- Für jede Regel liefert eine FAISS-Ähnlichkeitssuche [21] die fünf ähnlichsten Chunks (top_k = 5), zusammen im Mittel 3389 Zeichen Kontext, um ein gesamtes Repository zu beurteilen. Dieser Wert stammt aus der Originalarbeit und ist zugleich die Größe, die Experiment E6 variiert (Abschnitt 6.8).
- Der Judge sieht ausschließlich diese fünf Ausschnitte und urteilt in einem einzigen Prompt-Aufruf, mit dem unveränderten System-Prompt des Originals.

Zwei Eigenschaften dieses Aufbaus sind für den Vergleich entscheidend, weil der agentische Judge sich genau in ihnen unterscheidet. Der Judge kann nicht nachschlagen. Was das Retrieval nicht geliefert hat, existiert für ihn nicht. Er kann außerdem nicht feststellen, dass etwas fehlt, denn die Ähnlichkeitssuche gibt stets fünf Ausschnitte zurück, auch wenn sie inhaltlich irrelevant sind. Ein Ausweichurteil dieses Verfahrens ist deshalb primär eine Aussage über seinen Kontext und nicht über den geprüften Code.
Abbildung 5.1 zeigt den Ablauf, von der einmaligen Vorverarbeitung der Codebasis bis zum Urteil über die Einzelregel.

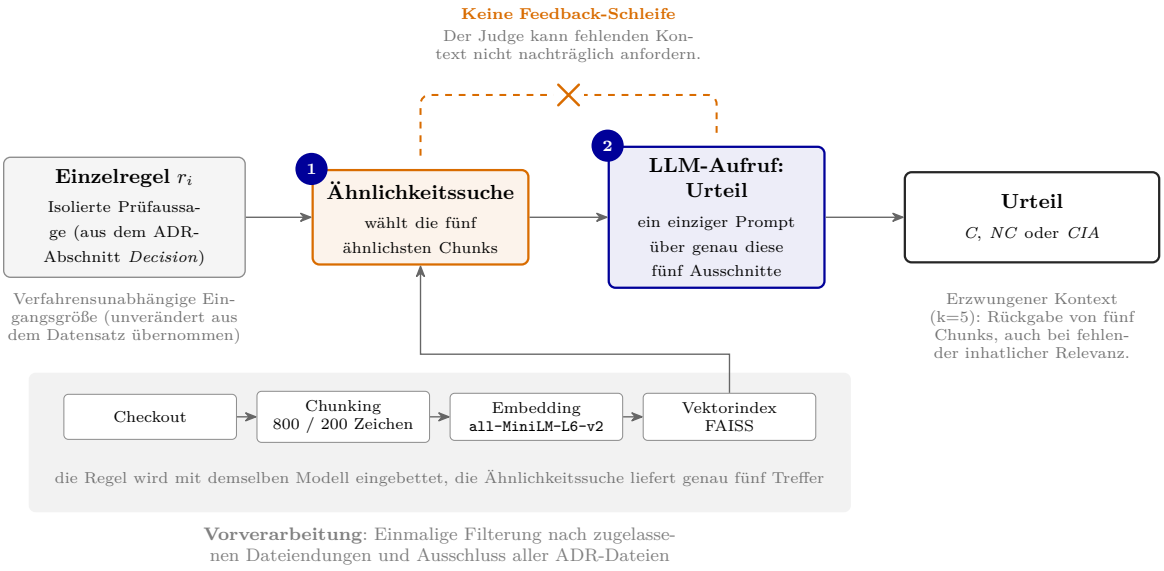


Abbildung 5.1.: Ablauf der statischen RAG-Pipeline. Eigene Darstellung nach dem Replication-Package von Su et al. [49].

5.1.2. Der agentische Judge

Das Modell erhält stattdessen Live-Zugriff auf das ausgecheckte Repository und drei schreibgeschützte Werkzeuge dafür, `list_files`, `grep` (Regex-Suche mit optionalem Pfad-

Filter) und `read_file`. Die Werkzeuge laufen ohne Shell-Zugriff und mit Schutz vor Pfad-Ausbrüchen außerhalb des Projektbaums (NFA-6). Das Modell sucht seine Evidenz selbst und iterativ und urteilt erst danach.
Die Verarbeitung ist dabei auf zwei Rollen aufgeteilt. Ein *Locator* verfügt nur über `list_files` und `grep`, navigiert iterativ und meldet am Ende präzise Kandidatenstellen in der Form `{path, start_line, end_line, why}`. Ein *Reader* verfügt ausschließlich über `read_file`, liest diese Stellen und fällt das Urteil. Auffinden und Lesen einer Belegstelle sind damit getrennte Schritte, was zugleich FA-6 entspricht und die Werkzeugprotokolle auswertbar hält. Aus ihnen lässt sich später ablesen, ob eine Suche erfolglos blieb oder ob gefundene Evidenz nicht gelesen wurde.
Diese Rollentrennung ist das Ergebnis mehrerer Entwicklungsschritte und damit selbst eine Entwurfsentscheidung, die sich prüfen lässt. Sie ist deshalb der Basiswert einer eigenen Variationsdimension und wird in Experiment E3 (Abschnitt 6.5) gegen eine Fassung ohne Rollentrennung gestellt. Für alle übrigen Läufe ist die hier beschriebene Zwei-Rollen-Fassung maßgeblich.
Abbildung 5.2 zeigt den Ablauf mit beiden Agenten, der Werkzeugschicht und der Nachsuche, die der Orchestrierungs-Code auslöst.

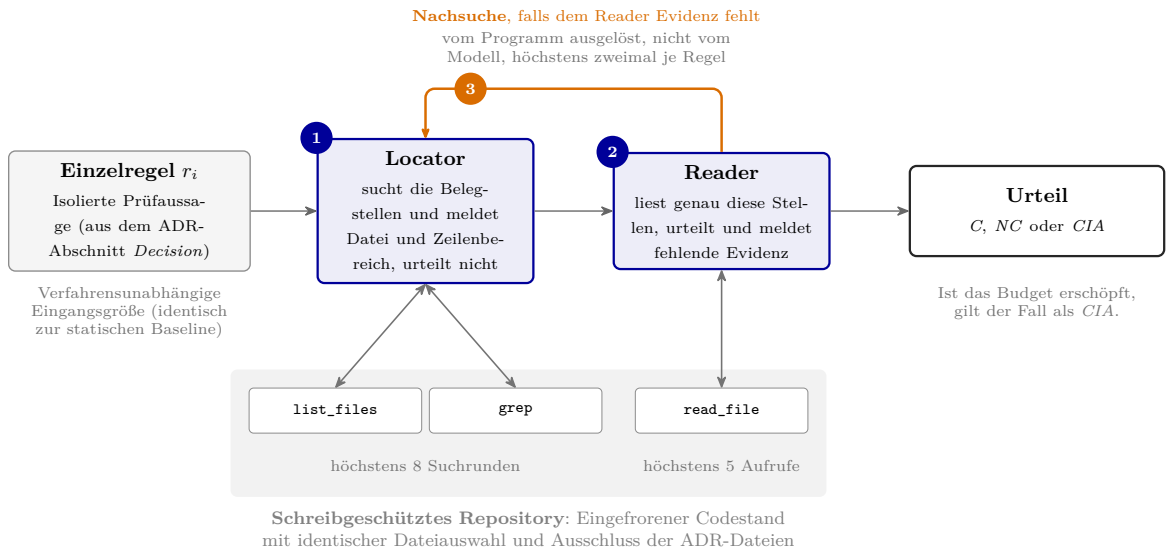


Abbildung 5.2.: Ablauf der agentischen Pipeline aus Locator und Reader. Eigene Darstellung.

5.1.3. Gemeinsame Suchraumbeschränkung

Ein Vergleich zweier Verfahren ist nur dann aussagekräftig, wenn beide auf dieselben Dateien zugreifen können. Andernfalls misst er nicht die Art der Evidenzbeschaffung, sondern die Größe des Suchraums.
Maßgeblich ist deshalb die Datei-Endungs-Whitelist des Originalverfahrens, eine feste Liste von rund 40 Programmiersprachen-Endungen (`.js`, `.py`, `.java`, `.go`, `.ts` und weitere). Der Retrieval-Index der Baseline ist über genau diese Dateien gebaut, und die Werkzeuge des agentischen Judge sind auf genau dieselbe Menge beschränkt (FA-5). Zusätzlich sind für beide Verfahren die ADR-Dateien selbst ausgeschlossen, damit sich die zu prüfende Regel nicht als eigener Beleg wiederfinden lässt.

Zwei Eigenschaften dieser Liste sind für die spätere Interpretation wichtig. Markdown steht nicht darauf, und zwar nicht aufgrund eines gezielten ADR-Filters, den das Original gar nicht besitzt, sondern schlicht, weil Dokumentation nie in der Dateiauswahl enthalten war. Ebenso wenig enthalten sind YAML, Build- und Konfigurationsdateien. Damit legt die Whitelist nicht nur fest, was die Verfahren sehen, sondern auch, welche Regeln aus dem Code überhaupt beantwortbar sind. Anhang A.3.2 zeigt, dass daraus ein erheblicher Teil der unentscheidbaren Fälle folgt.

5.1.4. Gemeinsames Urteilsformat

Beide Verfahren geben dasselbe dreiwertige Urteil ab. Möglich sind *Compliant* (*C*), *Not Compliant* (*NC*) und *Code is Insufficient to Answer* (*CIA*). Ein Lauf erzeugt je Fall eine Zeile mit Urteil, Begründung, Belegstelle und Ausfallkennzeichen (FA-10).
Beide Verfahren sind dabei bewusst als Messinstrumente zugeschnitten, nicht als Review-Werkzeuge. Sie prüfen ein ganzes Repository gegen einen festen Regelsatz statt einen einzelnen Änderungssatz. Auf jede Aufbereitung der Ausgabe verzichten sie. Mit dem später gebauten Prototyp teilen sie allein den zu untersuchenden Kern: die Art, wie ein Modell an seine Belege gelangt.
Abbildung 5.3 stellt beide Abläufe nebeneinander. Sichtbar wird dabei, wo genau sie auseinandergehen. Eingabe, Suchraum und Urteilskategorien sind identisch, und auch das Ergebnis hat in beiden Fällen dieselbe Form. Verschieden ist allein das Mittelstück, also der Weg von der Regel zur Evidenz.

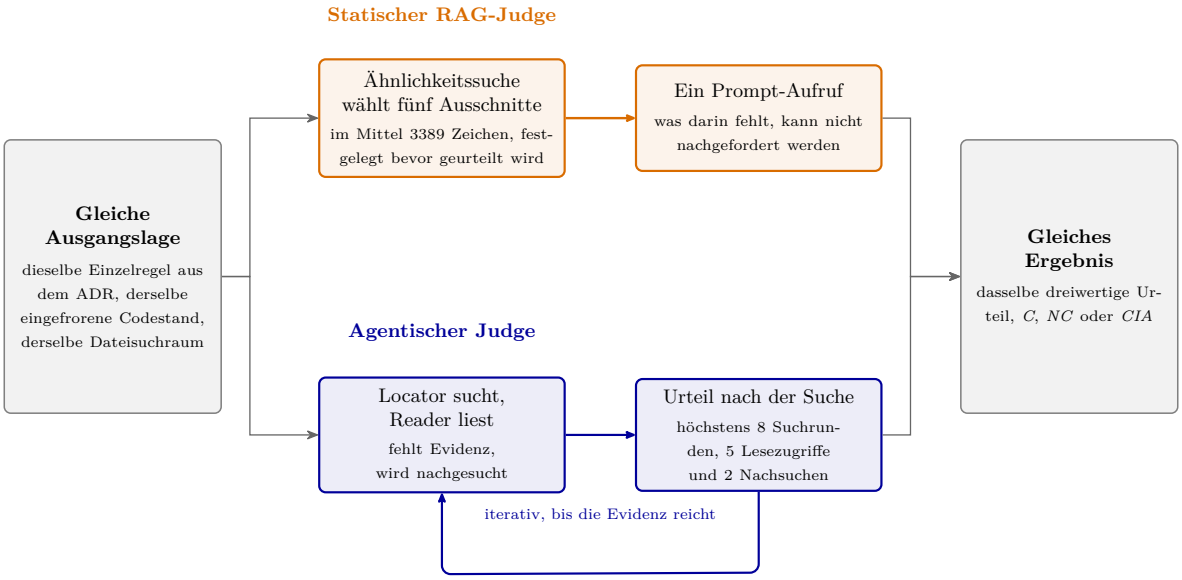


Abbildung 5.3.: Gegenüberstellung beider Verfahren. Gemeinsame Eingabe, gemeinsamer Suchraum und gemeinsame Urteilskategorien. Verschieden ist ausschließlich die Art der Evidenzbeschaffung. Eigene Darstellung.

5.2. Datengrundlage

5.2.1. Testfälle und Repositories

Grundlage sind 305 Fälle aus 43 GitHub-Repositories aus dem Replication-Package von Su et al. [49]. Jeder Fall verknüpft eine ADR-Datei, den daraus abgeleiteten atomaren Regeltext (FA-1) und das Label y_{true} der Originalarbeit. Die 305 Fälle sind eine Teilmenge eines größeren Original-Laufs über 1471 Regeln in 109 Repositories, weshalb die Fall-IDs nicht fortlaufend sind.
Der Datensatz wird hier ausschließlich als *Fallsammlung* genutzt, also für die Regeltexte und die zugehörigen Repositories. Sein Label y_{true} dient nicht als Maßstab. Anhang A.2.3 weist nach, dass dieses Label überwiegend aus den Ausgaben eines der bewerteten Verfahren abgeleitet ist und den Vergleich deshalb systematisch verzerren würde.

5.2.2. Modellauswahl

Alle Läufe wurden über den Hochschul-Gateway KI-Connect-NRW ausgeführt, der die Modellauswahl vorgibt (NFA-5). Der agentische Judge setzt dabei ein striktes JSON-Werkzeugaufruf-Format voraus, und dafür waren von den zunächst freigeschalteten Modellen nur `gpt-5.4-mini` und `gpt-5.4-nano` durchgehend zuverlässig. Als drittes Modell kommt `gpt-5.6-luna` hinzu.
Der Basiswert für alle Läufe ist `gpt-5.4-mini`. `gpt-5.4-nano` und `gpt-5.6-luna` sind die beiden weiteren Stufen dieser Dimension und kommen in Experiment E5 (Abschnitt 6.7) zum Einsatz. Beide Verfahren erhalten in jedem Vergleich dasselbe Modell, sodass die Modellwahl den Vergleich innerhalb eines Laufs nicht beeinflusst.
`gpt-5.6-luna` hält das strikte JSON-Werkzeugaufruf-Format zuverlässig ein und konnte deshalb in beiden Bedingungen laufen. Es verfügt zugleich über das mit Abstand größte Kontextfenster der drei Modelle, was es für die Frage nach dem Verhältnis von Modellgröße und Evidenzzugang besonders aufschlussreich macht (Abschnitt 6.7). Anders als für die beiden übrigen Stufen liegt für dieses Modell allerdings nur ein Lauf je Bedingung vor, weshalb sein Abstand nicht der Größe nach mit den Mittelwerten der anderen Stufen verglichen wird. **TODO: Die Größe des Kontextfensters von `gpt-5.6-luna` in Tokens ergänzen, sonst bleibt „mit Abstand am größten“ unbelegt.** **frage: Reichen hier erstmal 2 Modelle?**

5.3. Die Referenz

Wie in Abschnitt 5.2.1 bereits begründet, scheidet das ursprüngliche Label y_{true} als objektiver Maßstab aus. Um beide Verfahren fair bewerten zu können, wurde für diese Arbeit stattdessen eine neue, von den Modellen völlig unabhängige Referenz manuell erstellt.

5.3.1. Erstellung der unabhängigen Referenz

Mit festem Seed (42) wurden zunächst 60 Fälle rein zufällig aus der Gesamtmenge von 305 Fällen gezogen. Diese Reihenfolge ist methodisch zwingend, da nur eine Ziehung aus dem vollständigen Datensatz eine repräsentative Stichprobe garantiert. Diese bildet die tatsächliche Verteilung leichter und schwerer Fälle unverzerrt ab. Zusätzlich wurden gezielt 27 spezielle Fälle aufgenommen, in denen die menschliche Validierung der Originalarbeit

das Modellurteil überstimmt hat. Diese Beispiele zeigen die Grenzen des Originalverfahrens auf. Eine vorherige Aussortierung dieser fehleranfälligen Fälle hätte die Basis der Zufallsstichprobe künstlich vereinfacht und den Systemvergleich durch eine Auswahlverzerrung verfälscht. Da sechs dieser speziellen Fälle bereits in der zufälligen Ziehung enthalten waren, ergeben sich abzüglich dieser Überschneidungen insgesamt 81 einzigartige Fälle aus 29 Repositories.
Beide Teile werden getrennt ausgewertet. Maßgeblich für den fairen Verfahrensvergleich ist allein der Zufallsteil ($n = 60$). Die Gesamtmenge von 81 Fällen enthält überproportional viele anspruchsvolle Fälle, bei denen das Originalmodell nachweislich Fehler gemacht hat. Eine Auswertung über diese Gesamtmenge würde die statische Baseline daher methodisch benachteiligen.
Um eine unbewusste Beeinflussung durch die ursprünglichen Labels zu vermeiden, enthält die Arbeitsdatei weder y_{true} noch die vorherigen Modellantworten. Die Reihenfolge der Fälle ist zufällig gemischt und die exakte Zuordnung wurde erst nach Abschluss der Annotation aufgedeckt. Dadurch war während der manuellen Prüfung nicht ersichtlich, aus welchem Teil der Stichprobe ein Fall stammt. Ein eigens entwickeltes Skript (`gt_repo_tool.py`) beschränkt den Lesezugriff exakt auf den Suchraum der beiden Verfahren (Abschnitt 5.1.3). Da andere Dateien technisch blockiert sind, ist die strikte Einhaltung der Suchraumgrenzen systemseitig garantiert und nicht von manueller Sorgfalt abhängig. Für eine transparente Überprüfung dokumentiert jedes vergebene Label den genauen Dateipfad sowie die Zeilennummer der jeweiligen Fundstelle.
Die drei Urteilskategorien sind dabei enger gefasst als im Original-Prompt, vor allem *CIA* (Tabelle 5.1).

Tabelle 5.1.: Label-Semantik der unabhängigen Referenz.

Label	Bedeutung
C	Untersuchter Code zeigt die Regel umgesetzt.
NC	Untersuchter Code ist einschlägig und zeigt die Regel nicht umgesetzt oder widerspricht ihr.
CIA	Auf dieser Evidenzbasis nicht entscheidbar, entweder grundsätzlich keine Code-Frage (Kosten, Prozess, Absichtserklärung, externe Infrastruktur) oder die belegenden Artefakte liegen außerhalb des Suchraums.

5.3.2. Grenzen einer Einzelannotation

Die Annotation erfolgte ausschließlich durch den Verfasser. Eine Gegenprüfung durch eine zweite Person fand nicht statt. Daraus ergeben sich zwei methodische Einschränkungen. Zum einen fehlt ein statistisches Maß für die Beurteilerübereinstimmung. Zum anderen ist dem Verfasser die zu prüfende Hypothese bekannt. Eine unbewusste Voreingenommenheit lässt sich daher nicht vollständig ausschließen. Die zuvor beschriebenen Vorkehrungen minimieren dieses Risiko zwar, können es aber nicht gänzlich beseitigen.
Dennoch ist diese neue Referenz methodisch belastbarer als das ursprüngliche Label y_{true}. Sie basiert auf einer umfassenden Suche im gesamten Repository anstelle von lediglich fünf vorselektierten Ausschnitten. Zudem ist sie völlig unabhängig von den Ausgaben der evaluierten Verfahren entstanden. Jede Entscheidung lässt sich zudem anhand einer konkreten Belegstelle transparent nachprüfen. Dadurch wird die methodische Zirkularität der

Originalarbeit (Anhang A.2.3) erfolgreich aufgelöst. Die Einzelannotation bleibt dennoch eine Limitation dieser Untersuchung und wird in Abschnitt 8.3 nochmals aufgegriffen.

5.4. Basiskonfiguration der Vergleichsläufe

Jeder Lauf dieser Arbeit startet von derselben Grundeinstellung, die im Folgenden *Basiskonfiguration* heißt. Sie legt fest, mit welchem Modell, welchem Prompt, welcher Architektur und welchem Suchbudget gearbeitet wird und auf welchen Fällen. Tabelle 5.2 führt sie vollständig auf. Die letzte Spalte hält bereits fest, welche der Dimensionen in Kapitel 6 gezielt verändert wird und welche über alle Läufe konstant bleibt.

Tabelle 5.2.: Basiskonfiguration aller Läufe. Jedes Experiment in Kapitel 6 verändert genau eine der oberen fünf Dimensionen und lässt alle übrigen unangetastet.

Dimension	Basiswert	Variation
Variierbare Dimensionen		
Evidenzbeschaffung	statisch: fünf vorab per Ähnlichkeitssuche gewählte Ausschnitte. Agentisch: selbstgesteuerte Suche über `list_files`, `grep` und `read_file`	E1
Prompt	Original-Prompt (restriktiv) vs. optimierter Prompt (permissiv) für beide Verfahren	E2
Agentische Architektur	zwei getrennte Rollen, Locator sucht, Reader liest und urteilt	E3
Suchbudget	8 Suchrunden, 5 `read_file`-Aufrufe, 2 Nachsuchen. E4 senkt und erhöht diese Grenzen über vier Stufen	E4
Modell	`gpt-5.4-mini` mit `reasoning_effort=high`. E5 wiederholt beide Verfahren zusätzlich mit `gpt-5.4-nano` und `gpt-5.6-luna`	E5
Kontextmenge der Baseline	fünf Chunks je Regel (top_k = 5), im Mittel 3389 Zeichen. E6 hebt auf 20, 50 und 100 Chunks an	E6
Über alle Läufe konstant		
Fälle	dieselben 81 annotierten Fälle, berichtet wird der Zufallsteil ($n = 60$)	—
Suchraum	dieselbe Datei-Whitelist, ADR-Pfade ausgeschlossen (Abschnitt 5.1.3)	—
Codestand	derselbe eingefrorene Checkout je Repository	—
Regeltexte	unverändert aus dem Datensatz übernommen, nicht neu generiert	—
Urteilskategorien	*C*, *NC*, *CIA*	—
Referenz	Manuell erstellte Referenz (Abschnitt 5.3)	—
Auswertung	Verwendete Metriken und deren Berechnung **TODO: wird hier klar was ich meine?**	—
Wiederholungen	drei Läufe je Konfiguration, sofern nicht anders vermerkt	—

Damit sich jeder Lauf nachfahren lässt (NFA-4), hält Tabelle 5.3 die vollständigen Aufrufparameter fest. Sie gelten für beide Verfahren gleichermaßen, denn der statische Judge

spricht dasselbe Modell über denselben Gateway an und unterscheidet sich nur darin, welchen Kontext er mitschickt.

Tabelle 5.3.: Allgemeine Hyperparameter sowie verfahrensspezifische Konfigurationen der Judges.

Parameter	Wert
`temperature`	0,1
`top_p`	0,9
`max_tokens`	4096
`seed`	42
`reasoning_effort`	`high` (Modelle der `gpt-5`-Familie)
Timeout je Anfrage	120 Sekunden
Wiederholversuche	5
Suchbudget (agentisch)	8 Suchrunden, 5 `read_file`-Aufrufe, 2 Nachsuchen
Chunking (statisch)	800 Zeichen, 200 Zeichen Überlappung
Embedding-Modell (statisch)	*all-MiniLM-L6-v2*

TODO: Die vollständigen Prompt-Texte (statischer Original-Prompt, permissive Variante, Locator-Prompt, Reader-Prompt und der Prompt der Ein-Agenten-Fassung aus E3) gehören in den Anhang und müssen von hier aus referenziert werden, jeweils mit hervorgehobener Differenz zur Ausgangsfassung.

Die Vergleichsläufe wurden dabei eigens für die annotierten Fälle durchgeführt und nicht nachträglich aus einem größeren Lauf herausgefiltert. Nur so ist sicher, dass beide Verfahren dieselbe Fallmenge unter denselben Bedingungen bearbeitet haben.

Der agentische Judge protokolliert außerdem jeden Werkzeugaufruf in ein JSONL-Event-Log (NFA-3) und zwar mit Zeitpunkt, Phase, übergebenen Argumenten, Trefferzahl, gefundenen Dateien, Ergebnisgröße und Dauer. Aus diesem Log stammen sämtliche Werkzeugkennzahlen, die Kapitel 6 berichtet. Erst diese Protokollierung erlaubt Aussagen darüber, warum ein Urteil so ausfiel, statt nur festzustellen, dass es so ausfiel.

5.5. Erhebung und Auswertung der Metriken

Ermittlung der auswertbaren Fälle In jedem Lauf bleiben einzelne Fälle ohne verwertbares Urteil, weil die Antwort des Modells nicht als JSON lesbar war oder eine Anfrage nie beantwortet wurde. Solche Fälle sind aus allen berichteten Prozentwerten herausgenommen, aus dem Zähler wie aus dem Nenner. Der Nenner ist also die Zahl der gültigen Fälle und für beide Verfahren gleich gebildet. Zählt man die Ausfälle stattdessen als Fehlurteil, verschiebt sich der Wert bei Verfahren mit vielen Ausfällen deutlich.

5.5.1. Berechnung der Klassifikationsmetriken

Am Ende eines jeden Laufs entsteht für jeden Fall eine Ausgabezeile, die das gewählte Urteil, die Begründung, die Belegstelle und einen Statusvermerk für eventuelle Fehlversuche (FA 10) enthält. Anhand der Fall-ID ordnet das Auswertungsskript dieser Zeile das passende Referenzlabel zu und vergleicht beide Werte. Mithilfe der Bibliothek `scikit-learn` werden daraus die in Abschnitt 4.2 begründeten Kennzahlen berechnet. Dazu gehören

Precision, Recall und der F_1-Wert für jede der drei Klassen sowie die klassenübergreifende Accuracy. Ebenso umfasst die Auswertung das Makro-Mittel, das gewichtete Mittel, den Matthews Correlation Coefficient [33] und die Konfusionsmatrix. Aufgrund der kleinen Stichprobe erfolgt eine zusätzliche statistische Absicherung. Beide Verfahren haben exakt dieselben Fälle bearbeitet. Daher fließen für diesen Test nur jene Fälle in die Betrachtung ein, bei denen genau ein Verfahren das richtige Urteil traf. Für diese Teilmenge berechnet der exakte McNemar-Test einen Binomialtest [35]. Nach dem Abzug der übereinstimmenden Ergebnisse bleiben nur wenige Datenpaare übrig. Aus diesem Grund wird zwingend die exakte Variante des Tests anstelle der χ^2-Näherung verwendet.
Ein zweites Skript extrahiert die Werkzeugkennzahlen direkt aus den Ereignisprotokollen. Es zählt für jeden Fall die Aufrufe von `list_files`, `grep` und `read_file`. Zudem ermittelt es die Gesamtzahl aller getätigten Werkzeugaufrufe. Das Skript erfasst außerdem die erfolglosen Suchanfragen und die angeforderten Nachsuchen. Auch die Gesamtzahl der gesammelten Trefferzeilen wird für jeden Fall berechnet. Für jede betrachtete Gruppe wird abschließend der Mittelwert berichtet.

5.6. Prinzip der Experimentreihe

Ein einzelner Genauigkeitsvergleich hat wenig Aussagekraft. Er lässt offen, worauf ein gemessener Unterschied beruht. Die Untersuchung ist deshalb als systematische Reihe aufgebaut. Jeder Schritt verändert exakt eine Dimension der Basiskonfiguration und vergleicht das Ergebnis mit dem unveränderten Zustand.
Der erste Schritt ist der Hauptvergleich (E1). Er ersetzt die statische Evidenzbeschaffung der Baseline durch die agentische Variante. Alles Übrige bleibt gleich. Ein eventueller Unterschied erklärt sich damit aber noch nicht vollständig. Die folgenden Schritte prüfen deshalb systematisch naheliegende Alternativerklärungen. Sie verändern jeweils genau eine weitere Dimension. Dazu gehören der Prompt (E2), die agentische Architektur (E3), das Suchbudget (E4), das Sprachmodell (E5) und die Kontextmenge der Baseline (E6). Jeder dieser Schritte erfordert einen eigenen Satz neuer Läufe.
Ein Experiment definiert sich über die manipulierte Größe und nicht über die reine Anzahl der Läufe. Wenn ein Schritt dieselbe Dimension über mehrere Ausprägungen hinweg bewegt, bleibt dies ein einziges Experiment mit mehreren Stufen. Beispielsweise verstellt E4 das Suchbudget über vier Stufen, während E5 den Hauptvergleich mit zwei weiteren Modellen wiederholt und E6 die Kontextmenge der Baseline über drei Stufen anhebt. Der Verlauf über diese Stufen hinweg bildet jeweils den eigentlichen Befund. Eine Aufteilung in mehrere Experimente würde diesen Zusammenhang künstlich trennen, weshalb erst der Wechsel auf eine andere Dimension ein neues Experiment begründet.
Das Experiment E6 geht explizit einer wichtigen Alternativerklärung nach. Der gemessene Vorsprung des Agenten könnte schlicht an einer größeren Textmenge liegen. Das Originalverfahren darf immer nur exakt fünf Codeausschnitte lesen. Der Agent kann sich jedoch eigenständig ein Vielfaches dieser Menge beschaffen. Dieser systematische Vorteil lässt sich durch die vorherigen Experimente nicht ausschließen. Aus diesem Grund erhöht E6 schrittweise die erlaubte Evidenzmenge für die statische Baseline. Das Experiment testet somit, ob allein mehr Lesematerial das statische Verfahren auf das Niveau des Agenten hebt.
Neben den eigentlichen Experimenten gibt es zwei ergänzende Auswertungen. Ein reiner Kontrolllauf wiederholt eine bereits getestete Konfiguration. Dieser Lauf ermittelt die natürliche Messstreuung. Diese Streuung dient als Vergleichsmaßstab für alle gemesse-

nen Unterschiede der anderen Experimente. Zusätzlich werten Skripte die bestehenden Werkzeugprotokolle aus. Diese Protokolle erklären das Suchverhalten des Agenten in bereits abgeschlossenen Läufen. Beide Ergänzungen verändern keine Variablen und schaffen keine neuen Testbedingungen. Sie tauchen in Kapitel 6 daher lediglich als methodische Absicherung oder vertiefende Auswertung auf.
Die Zuordnung der Experimente zu den Forschungsfragen folgt einem klaren Schema. Die erste Forschungsfrage wird durch E1 beantwortet, wobei der Kontrolllauf und E5 dieses Ergebnis absichern. E2 liefert im Zusammenspiel mit E3, E4 sowie den Protokollauswertungen die Erkenntnisse für die zweite Frage. Zur dritten Forschungsfrage steuert die Experimentreihe lediglich zwei Kennzahlen bei, nämlich die Laufzeit und den Aufwand je Regel aus E4 (NFA-7) sowie die Zahl der Fehlalarme als Precision der Klasse *NC* (NFA-10). Die vollständige Beantwortung dieser dritten Frage erfolgt in Kapitel 7.

6. Experimente und Ergebnisse

TODO: hier müssen unbedingt noch mehr grafiken rein, damit man es besser verstehen kann.

Kapitel 5 hat den Messstand eingerichtet und mit Tabelle 5.2 eine Basiskonfiguration festgelegt. Dieses Kapitel bewegt diese Basis, und zwar in sechs Schritten, von denen jeder genau eine Dimension verändert und alle übrigen unangetastet lässt. Jeder Schritt ist ein eigener Satz Läufe, wird zuerst in seinem Aufbau beschrieben und anschließend gegen den unveränderten Zustand gehalten.

Die Reihe beginnt mit dem Hauptvergleich E1, der die statische Evidenzbeschaffung durch die agentische ersetzt (Abschnitt 6.2). Erst danach lässt sich fragen, worauf der gemessene Unterschied beruht. Genau das prüfen die folgenden fünf Experimente, indem sie der Reihe nach den Prompt (E2, Abschnitt 6.4), die agentische Architektur (E3, Abschnitt 6.5), das Suchbudget (E4, Abschnitt 6.6), das Modell (E5, Abschnitt 6.7) und die Kontextmenge der Baseline (E6, Abschnitt 6.8) verändern. Abschnitt 6.9 führt zusammen, was von den Erklärungen am Ende übrig bleibt, und Abschnitt 6.10 beantwortet damit die drei Forschungsfragen. Was die Zahlen tragen und wo sie an Grenzen stoßen, klärt Kapitel 8.

Diese Reihenfolge ist keine Aufzählung, sondern ein Ausschlussverfahren. E1 stellt einen Unterschied fest, für den zunächst fünf Erklärungen in Frage kommen: die Formulierung des Prompts, die Bauart des Agenten, die Menge an Evidenz, die er sammeln darf, das verwendete Modell und schlicht die Menge an Text, die er im Vergleich zur Baseline zu sehen bekommt. Jedes der folgenden Experimente nimmt sich genau eine dieser Erklärungen vor und versucht, sie zu bestätigen. Was am Ende keiner von ihnen zugeschrieben werden kann, bleibt der Art der Evidenzbeschaffung. Tabelle 6.1 zeigt die Reihe im Überblick, samt der Konfiguration, unter der jeder Schritt gelaufen ist.

Tabelle 6.1.: Überblick über die Experimentreihe. Die letzte Spalte nennt die Erklärung, die der jeweilige Schritt prüft. Alle Läufe verwenden `reasoning_effort=high`, soweit das Modell die Einstellung unterstützt. Alles, was nicht in der Spalte „Variierte Größe“ steht, entspricht der Basiskonfiguration aus Tabelle 5.2.

Exp.	Variierte Größe und Stufen	Modell	Läufe	Geprüfte Erklärung
E1	Evidenzbeschaffung: fünf vorselektierte Ausschnitte → selbstgesteuerte Suche	`gpt-5.4-mini`	6	— stellt den zu erklärenden Unterschied erst her
E2	Prompt: „keine Evidenz → *NC*“ gegen „→ *CIA*“, statisch und agentisch	`gpt-5.4-mini`	6	Der Vorsprung stammt nur aus einer Prompt-Formulierung
E3	Architektur: Locator und Reader → ein einzelner Agent	`gpt-5.4-mini`	3	Er hängt an der Rollentrennung, nicht am Werkzeugzugang
E4	Suchbudget: 2/1/0, 5/3/1, **8/5/2**, 12/8/3	`gpt-5.4-mini`	9	Er kommt nur daher, dass der Agent mehr sammeln durfte
E5	Modell: mini, nano, luna (Tabelle 6.22)	die drei Stufen selbst	8	Er hängt an diesem einen Modell
E6	Kontextmenge der Baseline: top_k 5, 20, 50, 100	`gpt-5.4-mini`	9	Der Agent urteilt nicht gezielter, er sieht nur mehr Text

Zwei Arten von Auswertung stehen bewusst außerhalb dieser Reihe und werden deshalb nicht als Experiment gezählt. Das eine ist der Kontrolllauf in Abschnitt 6.2.4, der eine Konfiguration lediglich wiederholt, um die Messstreuung zu beziffern. Das andere sind die Auswertungen der Werkzeugprotokolle in Abschnitt 6.3. Sie kommen ohne einen einzigen zusätzlichen Modellaufruf aus, betrachten die Logdateien bereits durchgeführter Läufe und erklären deren Ergebnis, statt eine neue Bedingung herzustellen.
Als Bewertungsmaßstab dient durchgehend die selbst erhobene Referenz und nicht das ursprüngliche Label y_{true}. Letzteres stammt nämlich größtenteils direkt aus den Antworten eines der hier verglichenen Verfahren und würde das Ergebnis verzerren (siehe Anhang A.2.3). Die genaue Zusammensetzung dieser eigenen Referenz klärt Anhang A.3.1. Sofern nicht explizit anders angegeben, beziehen sich alle Werte auf den Zufallsteil der Stichprobe ($n = 60$) und auf das Modell `gpt-5.4-mini` mit der Einstellung `reasoning_effort=high` (siehe Abschnitt 5.4).

6.1. Vorbemerkung zu den Bezugsgrößen

Drei Konventionen gelten für das gesamte Kapitel und müssen vorab feststehen, weil sonst benachbarte Zahlen verwechselbar werden.

Mittelwerte tragen die Befunde Jede Konfiguration wurde, sofern nicht ausdrücklich anders vermerkt, dreimal ausgeführt. Alle inhaltlichen Aussagen beziehen sich auf den

Mittelwert dieser drei Läufe, nicht auf einen Einzellauf. Konfusionsmatrizen und Werkzeugkennzahlen stammen dagegen durchgehend aus Lauf 1, weil sie fallweise Strukturen zeigen, die sich nicht sinnvoll mitteln lassen. Wo beides nebeneinandersteht, ist es entsprechend gekennzeichnet. Für den agentischen Judge treten dadurch drei eng benachbarte Werte auf: 76,7 % ist Lauf 1 über alle 60 Fälle, 76,3 % derselbe Lauf beschränkt auf die 59 Fälle mit beiderseits verwertbarem Urteil (Abschnitt 5.5), und 75,6 % der Mittelwert über drei Läufe. Beim statischen Judge fällt in jedem Lauf genau ein Fall aus, jeweils bei einer anderen Regel, weshalb sich seine 55,9 % auf $n = 59$ beziehen. Der maßgebliche Abstand beträgt damit 19,6 Prozentpunkte.

Signifikanz wird auf Fallebene geprüft Bei drei Wiederholungen je Bedingung liegt für jeden Fall nicht ein Urteilspaar vor, sondern drei. Diese einfach zusammenzuzählen wäre falsch, denn es sind dieselben 60 Fälle dreimal und nicht 180 unabhängige Beobachtungen; der p-Wert fiele dadurch deutlich zu günstig aus. Berichtet werden deshalb zwei Tests, die den Fall als Analyseeinheit behalten. Der *Mehrheitstest* wertet je Fall, ob ein Verfahren in mindestens zwei von drei Läufen richtig lag, und rechnet den exakten McNemar-Test auf diesen zusammengefassten Urteilen. Der *Vorzeichentest* vergleicht je Fall die Zahl richtiger Läufe beider Verfahren und ist dadurch empfindlicher für Fälle, die nur knapp kippen. Der Test auf einem einzelnen Laufpaar wird nur zur Transparenz mitgeführt. Stimmen beide Tests überein, ist der Befund eindeutig; weichen sie voneinander ab, wird das ausdrücklich benannt, statt sich den günstigeren Wert auszusuchen.

Maßstab Bewertet wird durchgehend gegen die unabhängige Handannotation. Als Schwelle für die Deutbarkeit einer Differenz dient die Messstreuung aus dem Kontrolllauf (Abschnitt 6.2.4).

6.2. E1 – Agentische statt statischer Evidenzbeschaffung

6.2.1. Aufbau des Experiments

E1 ist der Hauptvergleich dieser Arbeit und verändert gegenüber der Basiskonfiguration genau eine Dimension, nämlich die Art der Evidenzbeschaffung. Gegeneinander laufen:

- **Bedingung A (Baseline, statisch).** Der nachgebildete RAG-Judge aus Abschnitt 5.1.1. Je Regel liefert die FAISS-Ähnlichkeitssuche die fünf ähnlichsten Chunks (800 Zeichen, 200 Zeichen Überlappung, Embeddings aus *all-MiniLM-L6-v2*), und das Modell urteilt in einem einzigen Aufruf über genau diese fünf Ausschnitte. Verwendet wird der unveränderte Judge-Prompt der Originalarbeit.
- **Bedingung B (agentisch).** Der Judge aus Abschnitt 5.1.2 mit Live-Zugriff auf denselben Checkout. Locator (`list_files`, `grep`) und Reader (`read_file`) arbeiten im Standardbudget von 8 Suchrunden, 5 `read_file`-Aufrufen und 2 Nachsuchen.

Konstant gehalten sind in beiden Bedingungen das Modell `gpt-5.4-mini` mit `reasoning_effort=high`, die 81 annotierten Fälle mit dem Zufallsteil $n = 60$ als Berichtsbasis, der Suchraum aus Datei-Whitelist ohne ADR-Pfade, der eingefrorene Codestand, die unverändert übernommenen Regeltexte und das Auswertungsskript. Jede Bedingung wurde

dreimal ausgeführt; berichtet wird zunächst Lauf 1, die übrigen beiden tragen den Kontrolllauf in Abschnitt 6.2.4.
Zwei weitere Werte kommen ohne eigenen Lauf hinzu, weil sie als Dateien im Replication-Package vorliegen. Das ist erstens die unmittelbare Einschätzung des dort eingesetzten Modells Marco-o1 (*clf1*) und zweitens der daraus erstellte Labelsatz y_{true}. Beide werden hier nicht als eigene Bedingung geführt, sondern nur zur Einordnung mitgemessen, wie weit das Originalverfahren gegen die unabhängige Referenz kommt.
Die vollständigen Aufrufparameter beider Bedingungen, also Temperatur, `top_p`, `max_tokens`, Seed, Timeout und Wiederholversuche, stehen in Tabelle 5.3 und gelten unverändert für alle Läufe dieses Kapitels.

6.2.2. Ergebnis des Hauptvergleichs

Tabelle 6.2 stellt die Ergebnisse nebeneinander, alle gemessen an der unabhängigen Referenz auf dem Zufallsteil ($n = 60$). Mitgeführt sind bereits die beiden Prompt-Varianten aus Experiment E2, damit die Größenordnungen von Anfang an vergleichbar sind. Beim *permissiven Prompt* darf der statische Judge einen Fall als *CIA* kennzeichnen, wenn er in den Ausschnitten keine Evidenz findet, statt ihn wie im Original-Prompt verlangt als *NC* zu werten. Beim *restriktiven Prompt* bekommt umgekehrt der agentische Reader dieselbe Vorgabe wie das Original, muss also bei fehlender Evidenz auf *NC* erkennen. Ausgewertet werden beide erst in Abschnitt 6.4; an dieser Stelle stehen sie nur zur Einordnung. Abbildung 6.1 zeigt dieselben Werte grafisch und Tabelle 6.3 ergänzt für die drei eigenen Verfahren die über die drei Klassen gemittelten Kennzahlen.

Tabelle 6.2.: Genauigkeit gegen die unabhängige Referenz, Zufallsteil ($n = 60$, $C = 36$, $NC = 5$, $CIA = 19$). Maßgeblich ist die Spalte „Mittel“; der MCC bezieht sich auf Lauf 1. Die beiden Verfahren der Originalarbeit liegen als feste Dateien vor und haben deshalb keine Wiederholungen.

Verfahren	Lauf 1	Mittel (3)	MCC
Agentisch, `gpt-5.4-mini`	76,7 %	**75,6 %**	**0,569**
Marco-o1 (*clf1*, Original-Baseline)	71,7 %	—	0,460
Statisch, permissiver Prompt (keine Evidenz → *CIA*)	65,0 %	66,7 %	0,471
y_{true} der Originalarbeit	66,7 %	—	0,387
Agentisch, restriktiver Prompt (keine Evidenz → *NC*)	58,3 %	60,3 %	0,260
Statisch, Original-Prompt (keine Evidenz → *NC*)	55,9 %	55,9 %	0,337

TODO: Abbildung `abb_erg_hauptergebnis.pdf` muss neu erzeugt werden. Sie zeigt noch die Einzellaufwerte und insbesondere den permissiven Prompt mit 65,0 % statt mit dem Mittelwert von 66,7 %; außerdem fehlt die Zeile zum restriktiven agentischen Prompt.

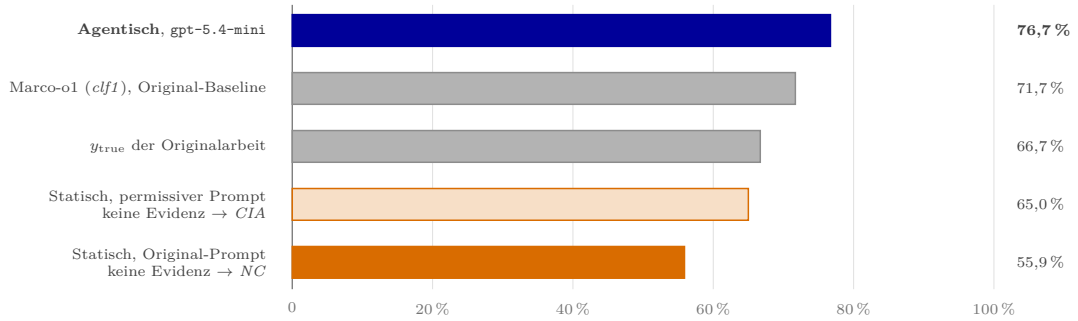


Abbildung 6.1.: Genauigkeit der verglichenen Verfahren, gemessen an der unabhängigen Referenz. Die Werte stammen aus Tabelle 6.2. Ob der Abstand belastbar ist, prüft Abschnitt 6.2.3. Eigene Darstellung.

Tabelle 6.3.: Makro-gemittelte Kennzahlen der drei eigenen Verfahren gegen die unabhängige Referenz, jeweils Lauf 1. Gemittelt wird über die drei Klassen, nicht über Wiederholungsläufe.

	Statisch, Original	Agentisch	Statisch, permissiv
Precision (Makro)	0,528	**0,683**	0,797
Recall (Makro)	0,599	**0,698**	0,625
F_1 (Makro)	0,493	**0,676**	0,627
F_1 (gewichtet)	0,596	**0,771**	0,651

Der agentische Judge erreicht im Mittel dreier Läufe eine Accuracy von 75,6 % und liegt damit 19,6 Prozentpunkte vor dem statischen Ansatz mit Original-Prompt (55,9 %). Im Einzellauf sind es 76,7 gegenüber 55,9 %. Der Vorsprung bestätigt sich beim MCC mit 0,569 gegenüber 0,337. Selbst die beiden Referenzwerte der Originalarbeit bleiben hinter dem neuen Ergebnis zurück, denn das unmittelbare Urteil von Marco-o1 erzielt 71,7 % und der daraus abgeleitete Labelsatz y_{true} 66,7 %. Wie sich dieser Abstand über die drei Urteilskategorien verteilt, zeigen die Konfusionsmatrizen in Abschnitt 6.4, wo sie zusammen mit den Prompt-Varianten gedeutet werden.

6.2.3. Absicherung gegen die Fallauswahl

Ein Abstand von rund 20 Prozentpunkten ist bei 60 Fällen noch kein belastbarer Beleg dafür, dass das agentische Verfahren überlegen ist. Die Auswahl der Fälle könnte den Ausschlag gegeben haben. Beide Verfahren haben dieselben Fälle bearbeitet, weshalb sich das gepaart prüfen lässt. Sortiert man alle 59 auswertbaren Fälle gepaart ein, ergibt sich die Vierfeldertafel in Tabelle 6.4.

Tabelle 6.4.: Gepaarte Vierfeldertafel beider Verfahren gegen die unabhängige Referenz ($n = 59$).

	Agent richtig	Agent falsch
Statisch richtig	31	2
Statisch falsch	**14**	12

In 43 Fällen urteilen beide gleich, nämlich 31-mal richtig und 12-mal falsch, weshalb diese Fälle zum Vergleich nichts beitragen. Es bleiben 16 Streitfälle, von denen 14 zugunsten des Agenten und 2 zugunsten des statischen Judge ausgehen. Der exakte McNemar-Test ergibt für dieses Laufpaar $p = 0{,}0042$.
Maßgeblich ist allerdings nicht ein einzelnes Laufpaar, sondern die Auswertung über alle drei Läufe mit dem Fall als Analyseeinheit (Abschnitt 6.1). Tabelle 6.5 führt beide Verfahren auf.

Tabelle 6.5.: Signifikanzprüfung des Hauptvergleichs über alle drei Läufe, Analyseeinheit ist der Fall ($n = 58$). Die diskordanten Paare der Einzelläufe dürfen nicht aufsummiert werden, weil es dieselben Fälle mehrfach sind.

Verfahren	diskordant	p (exakt)
Mehrheitsurteil je Fall (mindestens 2 von 3 richtig), konservativ	15 : 2	**0,0023**
Vorzeichentest auf der Zahl richtiger Läufe, empfindlicher	22 : 7	**0,0081**
einzelne Laufpaare, nur zur Transparenz	14:2, 14:3, 16:5	0,0042, 0,0127, 0,0266

Befund Beide Tests stimmen überein: Der Unterschied ist auf dem Fünf-Prozent-Niveau signifikant, und zwar mit $p = 0{,}0023$ im konservativeren der beiden Verfahren. Wären die Verfahren gleich gut, käme ein solches Ergebnis in weniger als einem von vierhundert Fällen zustande. Die Fallauswahl scheidet damit als Erklärung für den gemessenen Abstand aus.
Die Tafel zeigt zugleich, wie weit die beiden Verfahren überhaupt auseinanderliegen. Bei identischer Eingabe und identischem Suchraum sind 16 der 59 Fälle strittig, sie urteilen also nicht nur unterschiedlich gut, sondern in erheblichem Umfang schlicht verschieden.

6.2.4. Kontrolllauf: Streuung bei reiner Wiederholung

Die verwendeten Modelle erzeugen Reasoning-Tokens und sind auch bei fixer Temperatur nicht deterministisch. Zwei Läufe derselben Konfiguration liefern deshalb nicht dasselbe Ergebnis. Bevor sich irgendeine Differenz dieses Kapitels deuten lässt, muss also feststehen, wie groß die reine Messstreuung ist. Dieser Kontrolllauf verändert nichts an der Konfiguration, sondern wiederholt beide Bedingungen aus E1 lediglich dreimal; einziger Unterschied ist der Zeitpunkt der Ausführung. Er ist deshalb kein eigenes Experiment, sondern die Eichung des Maßstabs, an dem alle folgenden Differenzen zu messen sind.

Ergebnis Je drei Läufe pro Verfahren bei vollständig identischer Konfiguration ergeben die Werte in Tabelle 6.6.

Tabelle 6.6.: Wiederholungsläufe bei identischer Konfiguration.

Verfahren	Lauf 1	Lauf 2	Lauf 3	Mittel	Standardabw.	Spanne
Statisch	55,9 %	54,2 %	57,6 %	55,9 %	**1,69 pp**	3,4 pp
Agentisch	76,7 %	73,3 %	76,7 %	75,6 %	**1,92 pp**	3,3 pp

Befund Die Mittelwertdifferenz beträgt 19,6 Prozentpunkte und damit das gut Zehnfache der Messstreuung. Selbst der schlechteste agentische Lauf liegt mit 73,3 % noch 15,7 Punkte über dem besten statischen mit 57,6 %, sodass sich die Wertebereiche der beiden Verfahren nicht überschneiden. Gegen bloßes Wiederholungsrauschen ist das Hauptergebnis damit abgesichert. Zugleich folgt daraus eine Schwelle für alles Weitere, denn Differenzen unterhalb von rund zwei Prozentpunkten lassen sich bei dieser Fallzahl nicht mehr deuten.

Stabilität der Einzelurteile **TODO: ist das überhaupt relevant für den roten faden?** Die reine Standardabweichung zeigt nur, wie die Gesamtgenauigkeit schwankt. Sie verrät aber nicht, ob das System bei denselben Regeln auch immer zum gleichen Ergebnis kommt. Zwei Läufe können schließlich die gleiche Genauigkeit erzielen, sich aber bei völlig anderen Code-Stellen irren. Deshalb wurden die Urteile zusätzlich auf Fallebene verglichen. Wichtig vorab: Diese Auswertung misst nur die Stabilität, nicht die Richtigkeit. Auch ein Fall, der dreimal identisch falsch beurteilt wird, zählt hier als vollkommen stabil. Um die Schwankungen zu messen, wurden die drei Durchläufe zunächst paarweise verglichen. Beim agentischen Judge flossen dabei alle 60 Fälle ein, beim statischen Judge waren es 58 auswertbare Fälle. Die Auswertung zeigt: Vergleicht man je zwei Läufe miteinander, stimmen die Urteile beim statischen Verfahren in 83,3 % der Fälle überein. Beim agentischen Ansatz liegt die Quote bei 80,0 %. Anschaulich bedeutet das, dass bei einer bloßen Wiederholung des Tests etwa jedes sechste (statisch) beziehungsweise jedes fünfte Urteil (agentisch) in eine andere Kategorie kippt. Welche Kategorien das genau betrifft, schlüsselt Tabelle 6.7 auf.
Verlangt man stattdessen ein durchgehend identisches Label in *allen drei* Läufen, wird der Maßstab noch strenger. Hier liefern 44 von 58 Fällen (75,9 %) beim statischen Judge in jedem Durchlauf dasselbe Ergebnis. Beim agentischen Judge sind es 42 von 60 Fällen (70,0 %). Dieser Wert ist die maßgebliche Kennzahl für die Anforderung NFA-4, da er die tatsächliche Reproduzierbarkeit bei reiner Wiederholung beziffert. **wichtig: muss das nachvollziehbarer gemacht werden?**

Welche Urteile wechseln Aufschlussreicher als die Zahl der Wechsel ist, *welche* Urteile wechseln. Tabelle 6.7 schlüsselt sämtliche Wechsel über alle drei Laufpaare nach dem betroffenen Kategorienpaar auf.

Tabelle 6.7.: Urteilswechsel zwischen den Wiederholungsläufen, summiert über alle drei Laufpaare.

Wechselpaar	Statisch	Agentisch
CIA ↔ *NC*	**17**	8
C ↔ *CIA*	9	10
C ↔ *NC*	3	**18**
Wechsel gesamt	29	36

Beim statischen Judge betreffen 17 der 29 Wechsel das Paar *CIA* ↔ *NC*, also genau die Grenze, die der Original-Prompt verwischt, indem er „keine Evidenz gefunden“ und „widerspricht der Regel“ unter demselben Label zusammenfasst. Die Streuung hat an dieser Stelle also eine benennbare Ursache und ist kein reines Modellrauschen. Damit ist Experiment E2 bereits vorgezeichnet (Abschnitt 6.4).

Beim agentischen Judge liegt der Schwerpunkt woanders, denn dort entfallen 18 der 36 Wechsel auf das Paar $C \leftrightarrow NC$. Unsicher ist er also nicht darin, ob überhaupt Evidenz vorliegt, sondern darin, wie sie zu bewerten ist. Wer im Code nachsehen kann, muss seltener zwischen „nichts gefunden“ und „Verstoß“ raten, dafür aber häufiger entscheiden, ob das Gefundene die Regel nun erfüllt oder verletzt.

6.3. Auswertung der Werkzeugprotokolle des Hauptlaufs

E1 belegt, *dass* der agentische Judge besser abschneidet, aber nicht, *wie* er zu seinen Urteilen kommt. Dieser Abschnitt schließt diese Lücke ohne einen einzigen zusätzlichen Modellaufruf. Ausgewertet werden allein die JSONL-Event-Logs der bereits durchgeführten E1-Läufe (Abschnitt 5.4). Es handelt sich deshalb nicht um Experimente, sondern um drei Blicke auf dieselben Rohdaten: einmal nach dem gewählten Urteil gruppiert, einmal nach der Korrektheit des Urteils und einmal beschränkt auf die Fälle, in denen sich beide Verfahren uneinig waren. Was diese Auswertungen nur beobachten können, führt Experiment E4 anschließend gezielt herbei (Abschnitt 6.6).
Für das Verständnis der folgenden Auswertungen ist eine Kennzahl besonders wichtig: Ein Drittel aller Suchanfragen des Agenten liefert keinen einzigen Treffer (beim schwächeren Modell sind es sogar 40 %). Das ist keine Fehlfunktion, sondern ein normales Resultat, wenn man mit Regex in einer unvertrauten Codebasis sucht. Für das Suchbudget hat das allerdings direkte Konsequenzen: Hat der Agent beispielsweise nur zwei Versuche, läuft im Schnitt bereits einer davon ins Leere. Die vollständigen Metriken zur Werkzeugnutzung finden sich in Anhang A.3.

6.3.1. Werkzeugnutzung je nach gewähltem Urteil

Ergebnis Wie sich die 60 Fälle auf die drei vom Agenten gewählten Label verteilen und wie viel er dabei jeweils gesucht hat, zeigt Tabelle 6.8.

Tabelle 6.8.: Mittlere Werkzeugnutzung des agentischen Judge, gruppiert nach dem gewählten Label.

Gewählt	n	`list_files`	`grep`	davon leer	`read_file`	Nachsuchen	Tools ges.	Trefferzeilen
C	38	1,03	6,16	**1,26**	**2,16**	0,08	9,34	173,55
NC	8	1,88	9,00	3,00	2,75	0,88	13,62	196,25
CIA	14	1,29	7,50	**4,29**	**0,64**	0,29	9,43	88,21

Befund Der agentische Judge vergibt seltener *CIA* als die Baseline. Das kann zweierlei heißen: Entweder hat er gründlich gesucht und nichts gefunden, oder er hat früh aufgegeben. Nur im ersten Fall trägt die Annahme, dass aktiver Evidenzzugang die Urteilsqualität verbessert. Ein Blick auf die Logdaten (Abbildung 6.2) entscheidet die Frage. Wenn der Agent ein unklares *CIA*-Urteil fällt, sucht er vorher mit durchschnittlich 7,50 `grep`-Aufrufen sogar *häufiger* als bei einem klaren *C*-Urteil (6,16 Aufrufe). Das Problem ist nicht mangelnde Anstrengung, sondern fehlender Erfolg. Bei *CIA*-Fällen laufen 4,29 Suchen komplett ins Leere, 3,4-mal so viele wie bei *C*-Fällen (1,26). Dementsprechend

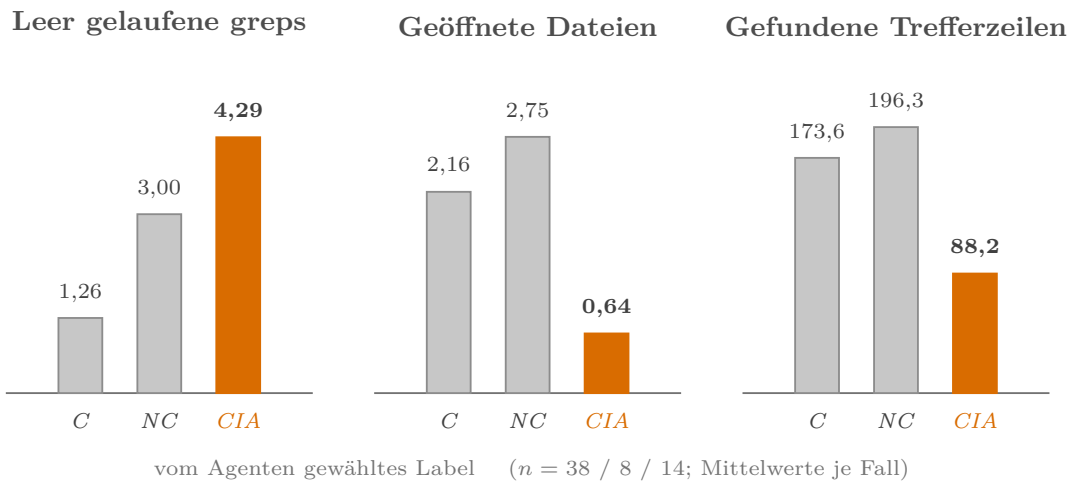


Abbildung 6.2.: Drei Kennzahlen aus Tabelle 6.8 nach dem vom Agenten gewählten Label. Jedes Panel hat seine eigene Einheit und seinen eigenen Maßstab. Vergleichen lässt sich deshalb nur innerhalb eines Panels. Eigene Darstellung.

öffnet der Agent kaum noch Dateien (0,64 statt 2,16) und die Menge der gefundenen Codezeilen halbiert sich von 173,6 auf 88,2.
Das belegt genau, wie der Agent arbeitet: Er wählt *CIA* erst dann, wenn er den Code intensiv, aber erfolglos durchsucht hat. Sobald er brauchbare Belege findet, legt er sich auf *C* oder *NC* fest. Dieser aktive Zugriff auf den Quellcode ist der Hauptgrund, warum der agentische Ansatz deutlich weniger unklare *CIA*-Fälle produziert.

6.3.2. Zusammenhang zwischen Suche und Korrektheit

Ergebnis Aus der vorigen Auswertung ließe sich vorschnell folgern, mehr Suchen führe zu besseren Urteilen. Sie gruppiert aber nach Antworten und nicht nach richtigen Antworten, und ein Agent kann ausgiebig suchen und trotzdem danebenliegen. Tabelle 6.9 schlüsselt dieselben Kennzahlen deshalb neu auf: Hier sind die Daten nicht nach der gewählten Kategorie gruppiert, sondern danach, ob das Urteil tatsächlich richtig oder falsch war.

Tabelle 6.9.: Mittlere Werkzeugnutzung des agentischen Judge, gruppiert nach Korrektheit des Urteils.

	n	`list_files`	`grep`	davon leer	`read_file`	Nachsuchen	Tools ges.	Trefferzeilen
Urteil richtig	46	1,07	6,17	1,78	1,72	0,11	**8,96**	153,70
Urteil falsch	14	1,64	**9,07**	3,57	2,43	0,64	**13,14**	166,43

Befund Die Auswertung belegt ganz deutlich, dass viel Suchen nicht automatisch zu richtigen Ergebnissen führt. Tatsächlich nutzt der Agent bei falschen Urteilen seine Werkzeuge spürbar öfter. Er greift im Schnitt 13,14 Mal auf seine Hilfsmittel zu, während er bei korrekten Urteilen nur 8,96 Aufrufe benötigt. Auch die regulären Suchanfragen steigen bei Fehlern von 6,17 auf 9,07 an. Besonders auffällig ist die Zahl der Nachsuchen. Sie liegt

bei falschen Antworten mit 0,64 mehr als dreimal so hoch wie bei korrekten Antworten mit 0,11.
Dieses Verhalten lässt sich logisch erklären. Ist die Sachlage eindeutig, findet der Agent die passende Stelle im Quellcode schnell und entscheidet richtig. Ist die Lage jedoch unklar, sucht er immer weiter, stößt auf thematisch ähnliche Inhalte und verrennt sich darin. Eine auffällig hohe Zahl an Werkzeugaufrufen ist daher kein Zeichen für besondere Gründlichkeit, sondern eher ein Indiz für eine schwierige oder unklare Ausgangslage.
Diese Erkenntnis ordnet die vorherige Auswertung in einen neuen Kontext ein. Die Vermutung „viel Evidenz bringt bessere Urteile“ trifft pauschal einfach nicht zu. Am Ende ist ausschließlich entscheidend, ob eine Suche *erfolgreich* war, und nicht, wie *ausgiebig* sie betrieben wurde. Man kann daher nicht schlussfolgern, dass der Agent den Code inhaltlich besser versteht. Die Daten beweisen lediglich, dass er die passenden Codezeilen im Erfolgsfall effektiv aufspürt.

6.3.3. Analyse der abweichenden Urteile

Ergebnis Fälle, in denen beide Verfahren dasselbe antworten, sagen über den Unterschied nichts aus, selbst dann nicht, wenn die gemeinsame Antwort falsch war. Aufschlussreich sind die Streitfälle, und dort vor allem die Richtung. Von den 15 Fällen, in denen sich die Verfahren uneinig waren, lag in 7 Fällen ausschließlich der agentische Judge richtig. In den verbleibenden 8 Fällen traf nur das statische Modell die korrekte Entscheidung. Tabelle 6.10 schlüsselt diese beiden Gruppen auf und zeigt, welche Code-Beweise (Evidenz) den Modellen jeweils zur Verfügung standen.

Tabelle 6.10.: Streitfälle beider Judges, nach Richtung getrennt, mit der jeweiligen Evidenzlage.

	n	`read_file`	leere greps	Trefferzeilen
Richtung 1, statisch sagt CIA, Agent sagt C oder NC (7 Fälle)				
Agent recht	4	2,50	2,00	260,75
Agent unrecht	3	2,67	2,67	254,33
Richtung 2, statisch sagt C oder NC, Agent sagt CIA (8 Fälle)				
Agent recht	6	**0,17**	**4,50**	**75,17**
Agent unrecht	2	1,50	6,00	92,00

Der Vorteil bei fehlenden Informationen Die erste Konfliktsituation tritt auf, wenn der statische Judge ein klares Urteil trifft und der Agent sich stattdessen mit *CIA* zurückhält. Die Auswertung zeigt hier ein absolut eindeutiges Bild. Der Agent behält in 6 von 8 dieser Fälle recht. Die Logdaten bestätigen dabei exakt das Suchmuster aus Abschnitt 6.3.1. In diesen Situationen liefen durchschnittlich 4,5 Suchanfragen komplett ins Leere und der Agent hat fast gar keine Dateien geöffnet. Er hat also aktiv gesucht, schlichtweg nichts gefunden und folgerichtig auf *CIA* entschieden. Genau hier liegt der allergrößte Gewinn des direkten Zugriffs auf den Quellcode. Der Agent kann zweifelsfrei erkennen, dass bestimmte Informationen im Code einfach nicht existieren. Das statische Verfahren liefert stattdessen bei jeder Anfrage blind irgendwelche fünf Ausschnitte und zwingt das Modell so zu einer Entscheidung.
Der exakt umgekehrte Fall liefert hingegen keine verwertbaren Erkenntnisse. Wenn der Agent entscheidet und das statische Modell auf *CIA* ausweicht, steht es bei insgesamt

nur 7 Fällen lediglich 4 zu 3 für den Agenten. Auch das Suchverhalten beider Gruppen unterscheidet sich in diesen Momenten praktisch gar nicht. Die Menge der gefundenen Trefferzeilen liegt sehr nah beieinander (260,75 im Vergleich zu 254,33) und auch die Zahl der geöffneten Dateien ist mit 2,50 und 2,67 beinahe identisch. Die geringe Datenmenge lässt für diese Konstellation also keine belastbaren Schlüsse zu.
Testet man das exakt gleiche Szenario mit dem schwächeren Modell `gpt-5.4-nano` (Experiment E5), wird der klare Vorteil des Agenten beim Erkennen fehlender Informationen sogar noch wesentlich deutlicher. Die detaillierten Einzelzahlen zu diesem Durchlauf sind in Anhang A.3 aufgelistet.

Suchaufwand als Indikator für Fallkomplexität Wenn man zwei Verfahren direkt miteinander vergleichen möchte, lassen sich aus den reinen Logdaten keine verlässlichen Gütekriterien ableiten. Betrachtet man die Konfliktfälle, zeigen sich oft kaum Unterschiede im Suchverhalten der Systeme und klare Tendenzen ergeben sich teilweise nur beim Einsatz schwächerer Modelle. Dieses Bild ändert sich jedoch komplett, sobald man nicht mehr die Verfahren vergleicht, sondern die Komplexität der Einzelfälle selbst betrachtet. Gruppiert man alle 60 Fälle danach, wie stabil sie in den drei agentischen Durchläufen beurteilt wurden, wird ein klares Muster sichtbar. Tabelle 6.11 belegt einen deutlichen Zusammenhang zwischen der Schwierigkeit eines Falls und dem investierten Suchaufwand.

Tabelle 6.11.: Suchaufwand nach Fallschwierigkeit, gemittelt über die drei agentischen Wiederholungsläufe.

Fallgruppe	n	`grep`-Aufrufe je Fall
In allen drei Läufen richtig	37	**5,91**
Gemischt	15	8,67
In allen drei Läufen falsch	8	8,38

Fälle mit durchgehend korrekten Urteilen kommen im Schnitt mit 5,91 Suchanfragen aus. Bei gemischten oder durchgehend falschen Urteilen steigt dieser Wert spürbar auf 8,67 und 8,38 an. Die reine Anzahl der Suchanfragen beweist zwar nicht, ob ein konkretes Urteil am Ende stimmt, sie ist aber ein exzellenter Indikator für die generelle Schwierigkeit eines Falls. Daraus lässt sich eine direkte Regel für die Praxis ableiten. Wenn der Agent für ein Urteil auffällig lange suchen musste, sollte diese Meldung im Merge Request nur mit entsprechender Vorsicht oder im Zweifel gar nicht erst angezeigt werden.

6.3.4. Suchfehler oder Urteilsfehler

Der agentische Judge liegt in rund einem Viertel der Fälle falsch. Offen ist, wie viel davon durch bessere Suche überhaupt noch zu holen wäre. Ursprünglich war dafür ein eigener Lauf vorgesehen, der dem Modell die annotierte Belegstelle unmittelbar vorlegt. Er ist nicht durchführbar, weil die Referenz nur für 19 der 41 entscheidbaren Fälle einen Pfad mit Zeilenangabe führt. Dieselbe Frage lässt sich aber ohne zusätzlichen Modellaufruf aus den Protokollen beantworten.
Aus dem Event-Log ist je Fall rekonstruierbar, welche Dateien der Agent überhaupt gesehen hat, nämlich die Treffer jedes `grep` und der Pfad jedes `read_file`. Enthält diese Menge die annotierte Belegdatei, war die Suche erfolgreich; ein trotzdem falsches Urteil

ist dann kein Such-, sondern ein Urteilsfehler. Die Fehler lassen sich so in zwei Gruppen teilen, von denen nur die erste durch besseres Retrieval behebbar wäre.

TODO: Zahlen nachtragen, erzeugt mit `Scripts/03_data_analysis/analyze_such_vs_urteilsfehler.py` über die drei agentischen Läufe. Zu berichten sind die Aufteilung der Fehlurteile in Such- und Urteilsfehler sowie der Anteil beider an allen 60 Fällen.

Eine Einschränkung gehört zwingend dazu. Das Kriterium ist dateigenau und nicht zeilengenau. Hat der Agent die richtige Datei geöffnet, aber nicht die entscheidenden Zeilen gelesen, zählt das hier bereits als Urteilsfehler. Der so bestimmte Suchfehleranteil ist deshalb eine *Untergrenze* für den Anteil der Fehler, den perfekt vorgegebene Evidenz beheben könnte.

6.4. E2 – Variation des Prompts

Der Abstand aus E1 ist statistisch belegt, aber noch nicht erklärt. Die naheliegendste Alternativerklärung ist der Prompt. E2 verändert deshalb gegenüber der Basiskonfiguration ausschließlich diese eine Dimension und beantwortet damit die zweite Forschungsfrage: Es trennt den reinen Effekt der Formulierung von dem Vorteil, den das Modell durch die selbst gesuchte Evidenz hat.

6.4.1. Aufbau des Experiments

Der Judge-Prompt der Originalarbeit verlangt ein *NC*-Urteil, wenn die abgerufenen Ausschnitte überhaupt keine Evidenz enthalten. Damit fallen zwei grundverschiedene Sachverhalte unter dasselbe Label, nämlich „der Code widerspricht der Regel“ und „es wurde nichts gefunden“. Für den zweiten Fall gäbe es *CIA*. Rechnet man das nicht heraus, sieht der agentische Vorsprung größer aus, als er ist, denn ein Teil davon käme dann nur aus dem Prompt.

Variiert wird deshalb die Anweisung für den Fall fehlender Evidenz, und zwar in beiden Verfahren. Erst dadurch entsteht ein vollständiges Feld aus zwei Verfahren und zwei Prompt-Fassungen:

- **E2a, statisch permissiv.** Der statische Judge darf einen Fall als *CIA* kennzeichnen, wenn die Ausschnitte keine Evidenz enthalten, statt wie im Original-Prompt verlangt auf *NC* zu erkennen.
- **E2b, agentisch restriktiv.** Der Reader-Prompt des agentischen Judge trennt „nichts gefunden“ und „widerspricht der Regel“ bereits von sich aus. Ihm wird diese Trennung genommen, er erhält also dieselbe restriktive Vorgabe wie das Original. Die Richtung ist damit gegenläufig zu E2a, variiert wird aber dieselbe Größe.

Identisch bleiben in beiden Fällen das Modell `gpt-5.4-mini` mit `reasoning_effort=high`, dieselben 60 Fälle, der Codestand und das Auswertungsskript. Bei E2a bleibt zusätzlich der Retrieval-Index unverändert, sodass exakt dieselben fünf Ausschnitte je Regel vorliegen; bei E2b bleiben Architektur und Suchbudget unverändert. Jede Fassung wird dreimal ausgeführt.

TODO: Der vollständige Wortlaut aller vier Prompt-Fassungen gehört in den Anhang, mit hervorgehobener Differenz. Von hier aus referenzieren.

6.4.2. Ergebnis

Der permissive Prompt hebt den statischen Judge von 55,9 auf 66,7 %. Der restriktive Prompt senkt den agentischen Judge von 75,6 auf 60,3 %. Beide Verfahren reagieren also deutlich, und zwar so stark, dass der agentische Judge unter der restriktiven Vorgabe fast auf das Niveau der ursprünglichen Baseline zurückfällt, obwohl er weiterhin vollen Zugriff auf das Repository hat. Tabelle 6.12 stellt alle vier Bedingungen nebeneinander.

Tabelle 6.12.: Accuracy im Feld aus Verfahren und Prompt-Fassung, jeweils Mittel aus drei Läufen gegen die unabhängige Referenz.

	restriktiver Prompt	permissiver Prompt	Prompt-Effekt
statisch	55,9 %	66,7 %	+10,7 pp
agentisch	60,3 %	**75,6 %**	+15,2 pp
Architektureffekt	+4,4 pp	+8,9 pp	

Aus diesem Feld lassen sich beide Effekte getrennt ablesen. Der Prompt bringt dem statischen Judge 10,7 Prozentpunkte und dem agentischen 15,2. Die Architektur bringt unter der restriktiven Vorgabe 4,4 Prozentpunkte und unter der permissiven 8,9. Keiner der beiden Effekte ist also eine feste Größe, denn jeder hängt davon ab, wie der andere Faktor gesetzt ist. Die Differenz zwischen den beiden Architektureffekten beträgt 4,5 Prozentpunkte und ist genau diese Wechselwirkung.

Warum die Lücke sich nicht in Anteile zerlegen lässt Eine Aufteilung in Prozentanteile, wie sie ein früherer Auswertungsstand vorsah, setzt voraus, dass sich beide Effekte unabhängig voneinander beziffern lassen. Genau das ist nicht der Fall. Der Weg vom statischen Judge mit Original-Prompt zum agentischen Judge in der Basiskonfiguration führt über 10,7 Prozentpunkte Prompt-Wechsel und 8,9 Prozentpunkte Verfahrenswechsel, zusammen die Gesamtlücke von 19,6 Prozentpunkten. Diese 8,9 Punkte sind aber nicht *der* Architekturanteil, sondern der Architektureffekt *unter dem permissiven Prompt*. Mit dem restriktiven Prompt als Ausgangspunkt ergäbe dieselbe Rechnung 4,4 Punkte. Anteile, die zusammen hundert Prozent ergeben, würden die Wechselwirkung von 4,5 Punkten verdecken, statt sie auszuweisen. Berichtet werden deshalb die vier Zellen und nicht zwei Prozentzahlen.

Absicherung Der Abstand zwischen dem agentischen Judge mit restriktivem und mit permissivem Prompt ist signifikant. Das Mehrheitsurteil über drei Läufe ergibt 15 zu 3 diskordante Fälle bei $p = 0{,}0075$, der Vorzeichentest 19 zu 5 bei $p = 0{,}0066$ ($n = 59$). Beide Tests stimmen überein. Auf dem einzelnen Laufpaar liegt der Wert bei $p = 0{,}0127$.

Was daraus folgt Der Werkzeugzugang macht die Urteilsanweisung nicht entbehrlich, sondern beide wirken zusammen. Der agentische Judge nutzt seine selbst beschaffte Evidenz nur dann gut, wenn der Prompt ihm erlaubt, fehlende Evidenz als solche zu benennen. Nimmt man ihm diese Möglichkeit, bricht seine Genauigkeit um 15,2 Punkte ein, und er reagiert damit sogar *stärker* auf die Formulierung als der statische Judge. Das widerspricht der Erwartung, die diesem Experiment zugrunde lag, nämlich dass ein Verfahren mit Live-Zugriff die Unterscheidung ohnehin über die Evidenzlage trifft und auf den Prompt kaum noch angewiesen ist. Die Werkzeugdaten zeigen auch, woran das liegt:

Unter der restriktiven Vorgabe sucht der Agent praktisch unverändert weiter (6,53 statt 6,85 `grep`-Aufrufe, 2,15 statt 1,88 geöffnete Dateien), er darf das Ergebnis dieser Suche nur nicht mehr korrekt benennen. Elf der 19 unentscheidbaren Fälle landen dadurch bei *NC* (Tabelle 6.16).

6.4.3. Wohin die Fehler wandern

Die Genauigkeiten allein sagen noch nicht, was die beiden Prompt-Fassungen anrichten. Die vier Konfusionsmatrizen (Tabellen 6.13 bis 6.16, alle aus Lauf 1) zeigen, dass keine der Fassungen das Grundproblem löst. Sie verlagern es nur, und zwar jeweils in die entgegengesetzte Richtung.

Tabelle 6.13.: Statisch, Original-Prompt (Accuracy 55,9 %, $n = 59$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	22	7	6	62,9 %
NC	1	4	0	80,0 %
CIA	3	9	7	36,8 %
Precision	84,6 %	20,0 %	53,8 %	

Tabelle 6.14.: Statisch, permissiver Prompt (Accuracy 65,0 % in Lauf 1, $n = 60$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	19	0	**17**	52,8 %
NC	1	2	2	40,0 %
CIA	1	0	18	**94,7 %**
Precision	90,5 %	100,0 %	**48,6 %**	

Tabelle 6.15.: Agentisch (Accuracy 76,7 %, $n = 60$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	31	3	2	86,1 %
NC	2	3	0	60,0 %
CIA	5	2	12	63,2 %
Precision	81,6 %	37,5 %	85,7 %	

Tabelle 6.16.: Agentisch, restriktiver Prompt (Accuracy 58,3 %, $n = 60$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	32	3	1	88,9 %
NC	4	1	0	20,0 %
CIA	6	**11**	2	**10,5 %**
Precision	76,2 %	**6,7 %**	66,7 %	

Beim statischen Judge findet das Modell mit dem permissiven Prompt deutlich mehr unklare Fälle, denn der *CIA*-Recall springt von 36,8 auf 94,7 %. Das geht jedoch auf

Kosten der übrigen Klassen: 17 von 36 regelkonformen Fällen landen jetzt fälschlich bei *CIA*, der *C*-Recall fällt von 62,9 auf 52,8 % und die *CIA*-Precision von 53,8 auf 48,6 %. Der statische Judge antwortet also nicht mehr zu restriktiv, sondern zu großzügig.
Beim agentischen Judge wirkt die restriktive Vorgabe spiegelbildlich und noch schärfer. Der *CIA*-Recall bricht von 63,2 auf 10,5 % ein, weil 11 der 19 unentscheidbaren Fälle als *NC* ausgegeben werden. Die *NC*-Precision fällt dadurch auf 6,7 %, das heißt von rund fünfzehn gemeldeten Verstößen trifft einer zu. Für ein System, dessen Zweck das Melden von Verstößen ist (NFA-10), ist das der ungünstigste aller vier Zustände, obwohl das Verfahren dabei vollen Repository-Zugriff hat.
Betrachtet man alle Klassen, ist allein der agentische Judge mit permissivem Prompt einigermaßen ausgewogen (Recall 86,1 %, 60,0 %, 63,2 %). Jede der drei übrigen Bedingungen bricht in mindestens einer Klasse ein. Der eigentliche Unterschied liegt damit nicht in der Trefferquote, sondern in der Fehlerverteilung, und diese wird erst durch das Zusammenspiel von Werkzeugzugang und passender Urteilsanweisung stabil.

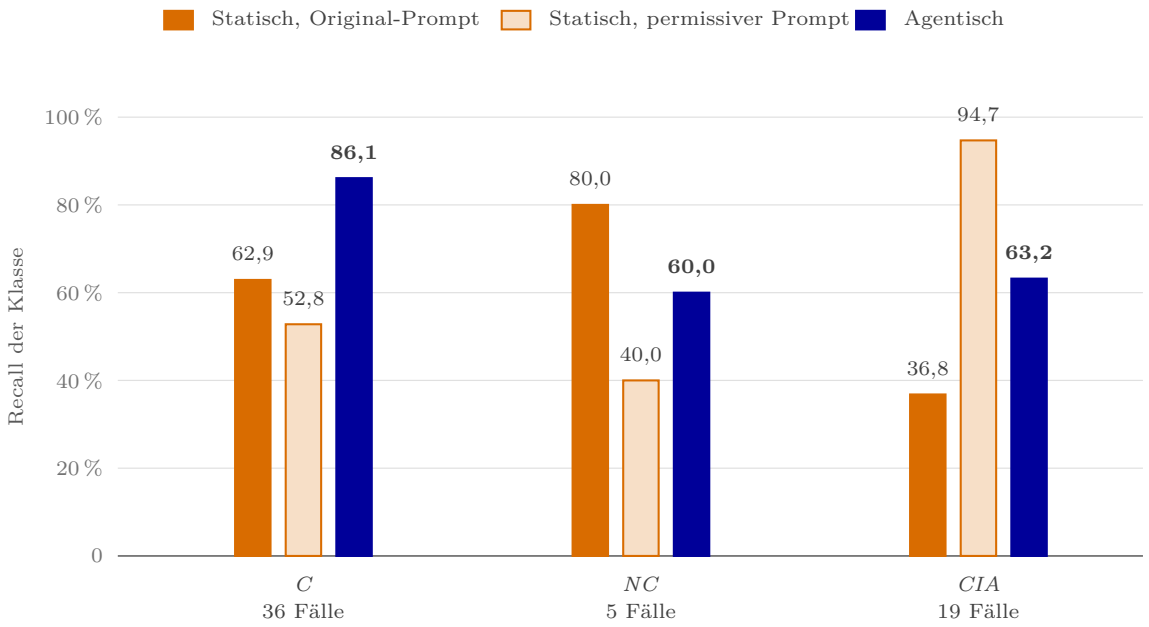


Abbildung 6.3.: Recall je Klasse, aus den Tabellen 6.13 bis 6.16. Die Werte für *NC* beruhen auf fünf Fällen und sind nicht belastbar. Eigene Darstellung.

TODO: Abbildung `abb_erg_urteilsprofil.pdf` muss neu erzeugt werden. Sie zeigt bisher nur drei Verfahren; die vierte Bedingung (agentisch mit restriktivem Prompt) fehlt und ist gerade die aussagekräftigste, weil sie in der Klasse *CIA* auf 10,5 % einbricht.

Diese Ausgewogenheit gilt allerdings nur für den Recall. Nimmt man den Abstand zwischen makro-gemitteltem und gewichtetem F_1-Wert als Maß, liegt der permissive statische Prompt mit 0,024 vor dem agentischen Judge mit 0,096. Über Recall und Precision zusammen verteilt er seine Leistung also gleichmäßiger, obwohl er in der Genauigkeit rund neun Punkte zurückliegt. Die Werte für alle Läufe stehen in Anhang A.3.

6.5. E3 – Variation der agentischen Architektur

E2 hat gezeigt, dass rund die Hälfte des Vorsprungs nicht am Prompt hängt. Damit bleibt offen, ob der verbleibende Anteil tatsächlich an der selbst beschafften Evidenz liegt oder an der besonderen Bauart des agentischen Judge, also an der Trennung in Locator und Reader. E3 verändert deshalb gegenüber der Basiskonfiguration ausschließlich diese Dimension.

6.5.1. Aufbau des Experiments

Verglichen werden zwei agentische Fassungen, die beide vollen Werkzeugzugriff auf denselben Checkout haben:

- **Bedingung A (Basiskonfiguration).** Zwei getrennte Rollen. Der Locator verfügt nur über `list_files` und `grep` und meldet Kandidatenstellen, der Reader verfügt nur über `read_file`, liest diese Stellen und urteilt.
- **Bedingung B (Variation).** Ein einzelner Agent mit allen drei Werkzeugen, ohne Rollentrennung und ohne Übergabeformat. Er sucht, liest und urteilt in derselben Schleife.

Konstant bleiben das Modell `gpt-5.4-mini` mit `reasoning_effort=high`, das Standardbudget von 8 Suchrunden, 5 `read_file`-Aufrufen und 2 Nachsuchen, dieselben 60 Fälle, der Suchraum, der Codestand und der inhaltliche Prompt-Kern. Verändert wird allein, ob dieser Kern auf zwei Rollen aufgeteilt ist oder nicht. Bedingung B wird dreimal ausgeführt, Bedingung A liegt aus E1 bereits in drei Läufen vor. Der Vergleich beantwortet zwei Fragen zugleich. Erstens, ob die Rollentrennung überhaupt Genauigkeit bringt oder nur die Protokolle besser auswertbar macht. Zweitens, wie sich der Werkzeugaufwand verändert, wenn ein einzelner Agent frei zwischen Suchen und Lesen wechseln darf.
Eine Eigenheit der Ein-Agenten-Fassung ist dabei vorab festzuhalten, weil sie die Auswertung einschränkt. Ohne Rollentrennung lässt sich aus dem Protokoll nicht mehr unterscheiden, ob eine Belegstelle gefunden, aber nicht gelesen wurde, denn Suchen und Lesen liegen in derselben Schleife. Die Nachsuche entfällt zwangsläufig, weil sie nur dem Rücksprung zwischen den beiden Rollen dient.

6.5.2. Ergebnis

Der Verzicht auf die Rollentrennung kostet 6,7 Prozentpunkte, denn die Ein-Agenten-Fassung erreicht im Mittel dreier Läufe 68,9 % gegenüber 75,6 % der Basiskonfiguration (Tabelle 6.17).

Tabelle 6.17.: Beide agentischen Fassungen gegen die unabhängige Referenz, Mittel aus je drei Läufen ($n = 60$).

Bedingung	Accuracy	Standardabw.	MCC (Lauf 1)
A — Locator und Reader getrennt (Basis)	**75,6 %**	1,92 pp	0,569
B — ein Agent, keine Rollentrennung	68,9 %	2,55 pp	0,532

Dieser Abstand liegt zwar über der Messstreuung, hält der Signifikanzprüfung aber nicht stand. Das Mehrheitsurteil über drei Läufe ergibt nur 6 zu 1 diskordante Fälle bei $p =$

0,1250, der Vorzeichentest 15 zu 8 bei $p = 0{,}2100$. Beide Tests stimmen darin überein, dass der Genauigkeitsunterschied bei dieser Fallzahl *nicht* abgesichert ist. Auf dem einzelnen Laufpaar liegt der Wert bei $p = 0{,}2668$.

Der Mechanismus ist eindeutiger als die Genauigkeit Aufschlussreich ist deshalb nicht die Accuracy, sondern das Verhalten dahinter. Tabelle 6.18 stellt die Werkzeugnutzung beider Fassungen gegenüber.

Tabelle 6.18.: Mittlere Werkzeugnutzung je Fall, Lauf 1 aus dem Event-Log. Die Spalte „mit `read_file`“ nennt den Anteil der Fälle, in denen überhaupt eine Datei geöffnet wurde.

Bedingung	`list_files`	`grep`	davon leer	`read_file`	mit `read_file`	Tools ges.	Trefferzeilen
A — getrennt	1,20	6,85	2,20	1,88	**83 %**	9,93	157
B — ein Agent	1,02	3,28	0,85	0,57	**40 %**	4,87	104

Ohne Rollentrennung sucht der Agent nur noch halb so viel, nämlich 3,28 statt 6,85 `grep`-Aufrufe, und er öffnet in 60 % der Fälle *überhaupt keine Datei*. Er urteilt dann allein aus den Kontextzeilen, die `grep` ohnehin mitliefert. Die Konfusionsmatrix zeigt, wohin das führt (Tabelle 6.19): 11 der 36 regelkonformen Fälle werden als *NC* ausgegeben, die *NC*-Precision fällt auf 23,5 %.

Tabelle 6.19.: Ein Agent, keine Rollentrennung (Accuracy 68,3 % in Lauf 1, $n = 60$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	22	**11**	3	61,1 %
NC	1	4	0	80,0 %
CIA	2	2	15	78,9 %
Precision	88,0 %	**23,5 %**	83,3 %	

Befund Die Rollentrennung trägt einen konsistenten, statistisch aber nicht gesicherten Genauigkeitsvorteil. Die Aussage dieses Experiments ruht deshalb nicht auf der Accuracy, sondern auf dem Mechanismus, und der ist eindeutig: Ein einzelner Agent mit denselben drei Werkzeugen und demselben Budget nutzt sie nur zur Hälfte und liest die gefundenen Stellen mehrheitlich gar nicht. Der Werkzeugzugang allein genügt also nicht, es braucht eine Architektur, die das Lesen erzwingt. Damit ist die zweite der beiden Lesarten eingetreten, die dieses Experiment vorab benannt hatte: Der in E2 verbleibende Vorteil hängt an der Rollentrennung und nicht am Werkzeugzugang als solchem. Für den Prototyp ist das die wichtigste Entwurfsvorgabe der ganzen Arbeit (Kapitel 7).
Nebenbei fällt die Ein-Agenten-Fassung deutlich billiger aus, mit 54 statt 74 Sekunden und 73 589 statt 125 898 Tokens je Regel. Wer den Aufwand senken will, zahlt dafür also mit genau dem Verhalten, das den Vorteil trägt.

6.6. E4 – Variation des Suchbudgets

E1 bis E3 verändern, *wie* der Agent an seine Evidenz kommt. E4 verändert, *wie viel* er davon holen darf. Praktisch geht es darum, ab wann zusätzliche Aufrufe nichts mehr bringen, denn das schlägt direkt auf Laufzeit und Kosten je Regel durch (NFA-7). Methodisch

geht es um die Frage, ob der Vorteil an der Bauart des Verfahrens hängt oder schlicht daran, dass der Agent mehr Material sammeln durfte.

6.6.1. Aufbau des Experiments

Variiert werden allein die drei Obergrenzen des agentischen Judge, also die Zahl der Suchrunden, die Zahl der `read_file`-Aufrufe und die Zahl der Nachsuchen. Tabelle 6.20 nennt die vier Stufen. Sie sind über Umgebungsvariablen gesetzt, sodass zwischen den Läufen keine Codeänderung nötig ist. Die Standardstufe ist der Basiswert aus Tabelle 5.2 und entspricht damit exakt dem agentischen Lauf aus E1. Unverändert bleiben über alle vier Stufen das Modell `gpt-5.4-mini` mit `reasoning_effort=high`, dieselben 60 Fälle, der Suchraum, der Codestand und der Prompt. Jede Stufe wurde dreimal ausgeführt; die Standardstufe ist mit dem agentischen Lauf aus E1 identisch.

Tabelle 6.20.: Budgetstufen der Ablation E4. Die Standardstufe entspricht der Basiskonfiguration und damit dem agentischen Lauf aus E1.

Stufe	Suchrunden	`read_file`-Aufrufe	Nachsuchen
min	2	1	0
mid	5	3	1
Standard	8	5	2
max	12	8	3

Vier Stufen, ein Experiment Zwei der Stufen liegen unterhalb des Basiswerts und eine darüber, und beide Richtungen beantworten verschiedene Fragen. Nach unten geht es darum, ob ein Mindestbudget konstitutiv ist, ob das Verfahren also ohne ausreichend viele Aufrufe überhaupt noch etwas leistet. Nach oben geht es um den Sättigungspunkt, also um die Frage, ab wann zusätzliche Aufrufe nur noch Laufzeit kosten. Getrennte Experimente werden daraus trotzdem nicht, denn manipuliert wird in beiden Richtungen dieselbe Größe. Der Befund ist nicht ein einzelner Stufenvergleich, sondern der Verlauf über alle vier Stufen hinweg, und diesen Verlauf würde eine Aufteilung in ein Absenkungs- und ein Erhöhungsexperiment genau in der Mitte zerschneiden. Berichtet werden die beiden Richtungen deshalb getrennt, gezählt werden sie als ein Experiment.
Drei Lesarten sind vorab denkbar und werden durch die Messung entschieden. Zeigt sich der agentische Vorteil erst bei hohem Budget, kommt er nur daher, dass der Agent mehr Material sammeln durfte. Fällt der Agent auf der kleinsten Stufe auf das Niveau der Baseline zurück, braucht das Verfahren ein Mindestbudget, um überhaupt zu funktionieren. Sinkt die Zahl der *CIA*-Urteile zugleich mit jeder größeren Stufe weiter, dann ist der Zusammenhang, den Abschnitt 6.3.1 nur beobachten kann, hier gezielt herbeigeführt.

TODO: Bisher werden die drei Grenzen nur gemeinsam als Paket verstellt. Damit lässt sich nicht sagen, welche der drei den Effekt trägt. Nachzuholen wären Läufe, die jeweils *eine* Grenze allein verändern, also etwa Suchrunden 2/5/8/12 bei festen 5 `read_file`-Aufrufen und 2 Nachsuchen, und analog für die beiden anderen Dimensionen. **wichtig: muss hier noch die einzelnen stufen getestet werden? weil gerade wurden immer 3 pakete getestet**

6.6.2. Ergebnis

Die zwölf Läufe über die vier Budgetstufen aus Tabelle 6.20 ergeben die Werte in Tabelle 6.21.

Tabelle 6.21.: Genauigkeit, Werkzeugnutzung und Urteilsverteilung über die vier Budgetstufen ($n = 60$). Die Accuracy ist das Mittel aus drei Läufen, Werkzeugnutzung und Urteilsverteilung stammen aus Lauf 1.

Stufe	Accuracy	SD	`grep`	davon leer	`read_file`	Tools ges.	Urteile *C*/*NC*/*CIA*
min (2/1/0)	56,1 %	2,55	1,07	0,37	0,57	2,53	16 / 2 / **42**
mid (5/3/1)	70,0 %	1,67	4,62	1,65	1,57	7,27	31 / 8 / 21
Standard (8/5/2)	**75,6 %**	1,92	6,85	2,20	1,88	9,93	38 / 8 / 14
max (12/8/3)	76,1 %	**4,19**	7,78	2,82	1,92	10,83	41 / 8 / 11

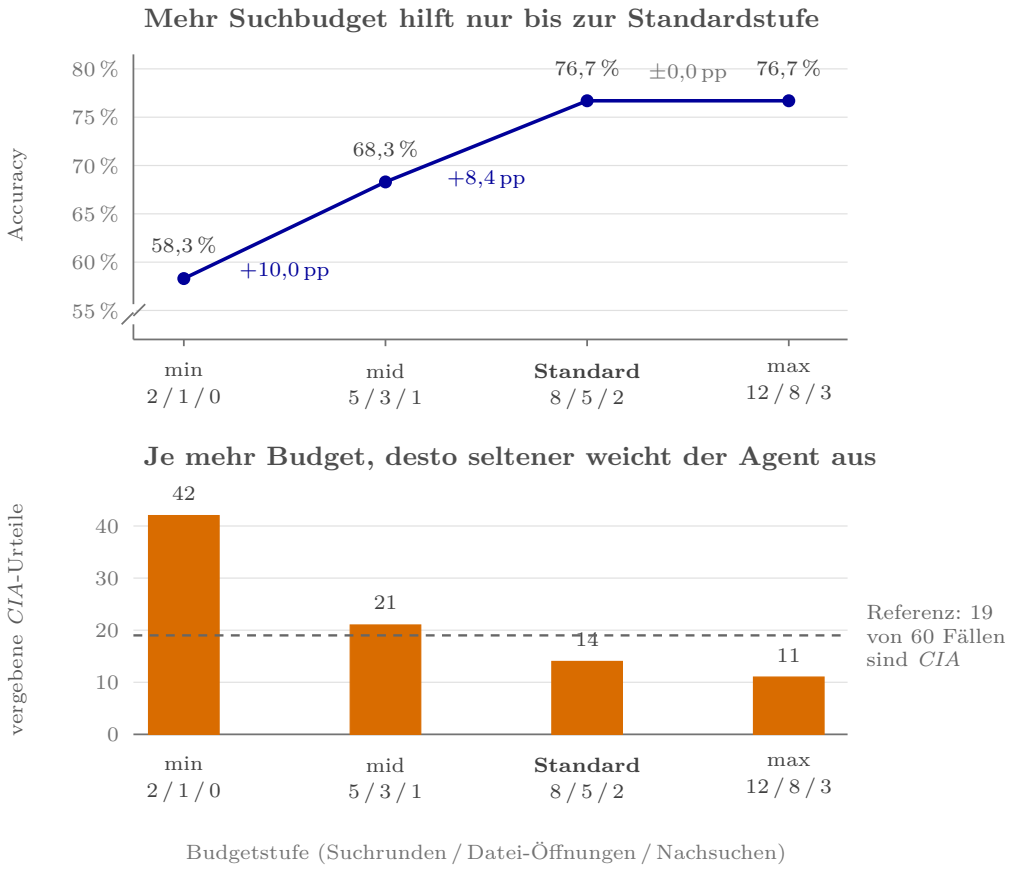


Abbildung 6.4.: Genauigkeit und Zahl der *CIA*-Urteile über die vier Budgetstufen aus Tabelle 6.21, jeweils mit beschrifteter x-Achse für die vier Stufen. Die Genauigkeitsachse ist beschnitten und beginnt bei 55 % statt bei null (Achsenbruch-Symbol am Fuß der Achse), die Achse der *CIA*-Urteile beginnt bei null. Die gestrichelte Linie markiert die 19 *CIA*-Fälle der Referenz. Eigene Darstellung.

TODO: Abbildung `abb_erg_budget.pdf` muss neu erzeugt werden. Sie zeigt die Einzellaufwerte 58,3 / 68,3 / 76,7 / 76,7 %; maßgeblich sind jetzt die

Mittelwerte 56,1 / 70,0 / 75,6 / 76,1 %. Sinnvoll wären zusätzlich Fehlerbalken über die drei Läufe, weil die Streuung auf der höchsten Stufe stark zunimmt.
Der Zuwachs von Stufe zu Stufe beträgt +13,9 Prozentpunkte von min auf mid, +5,6 von mid auf Standard und +0,5 von Standard auf max.

Richtung 1, Absenkung unter den Basiswert Gibt man dem Agenten nur minimale Ressourcen, erreicht er 56,1 % und liegt damit gerade noch 0,2 Prozentpunkte über dem statischen Ansatz mit 55,9 %. Der Vorsprung verschwindet also vollständig, denn 0,2 Punkte liegen weit unterhalb der Messstreuung von rund zwei Punkten. Der Grund wird an der Werkzeugnutzung sofort sichtbar: Mit 1,07 Suchanfragen und 0,57 geöffneten Dateien je Fall kann der Agent kaum mehr Evidenz sammeln, als die Baseline ihm ohnehin vorlegt. Der Leistungsvorteil entsteht demnach nicht dadurch, dass Werkzeuge vorhanden sind, sondern erst dadurch, dass sie oft genug benutzt werden dürfen. Ein Mindestbudget ist für das agentische Verfahren konstitutiv, und unterhalb davon ist es schlicht eine teurere Baseline.

Richtung 2, Erhöhung über den Basiswert Nach oben wächst die Genauigkeit dagegen kaum noch. Ein doppelt so großes Budget von 12/8/3 bringt 0,5 Prozentpunkte und damit weniger, als zwei identische Läufe ohnehin auseinanderliegen. Der Agent nutzt das größere Budget durchaus, denn die Zahl der Suchen steigt von 6,85 auf 7,78, doch für das Urteil zahlt sich das nicht mehr aus. Bemerkenswert ist dabei die Streuung: Auf der höchsten Stufe beträgt die Standardabweichung 4,19 Prozentpunkte und damit mehr als das Doppelte der Standardstufe. Mehr Budget macht das Verfahren also nicht nur nicht besser, sondern zugleich unzuverlässiger.
Erst beide Richtungen zusammen ergeben den eigentlichen Befund. Der Zusammenhang zwischen Budget und Genauigkeit ist weder linear noch beliebig skalierbar, sondern eine Sättigungskurve mit einem Knie, und der Basiswert der Konfiguration liegt genau auf diesem Knie. Hätte man nur eine der beiden Richtungen gemessen, wäre entweder der Schwellencharakter oder die Sättigung unsichtbar geblieben. Weil jede Stufe dreimal gelaufen ist, ruht insbesondere die Sättigungsaussage nicht auf einem Einzelwert.

Bestätigung des beobachteten Suchmusters Ist das Budget stark limitiert, weicht der Agent bei 42 von 60 Fällen auf die Kategorie *CIA* aus. Das sind mehr als doppelt so viele Ausweichurteile, wie die Referenzdaten eigentlich vorsehen. Sobald man das Budget jedoch schrittweise erhöht, sinkt diese Zahl rapide ab auf 21, dann auf 14 und ganz am Ende auf 11. Das untere Panel in Abbildung 6.4 stellt diesen Verlauf dar. Die Logik dahinter ist simpel. Je mehr der Agent suchen darf, desto seltener muss er mangels Informationen kapitulieren, da er handfeste Beweise für *C* oder *NC* aufspürt. Die Protokollauswertung in Abschnitt 6.3.1 hatte diese Dynamik bereits beobachtet, hier ist sie nun gezielt herbeigeführt.
Besonders aufschlussreich ist dabei die höchste Budgetstufe. Hier vergibt das System mit 11 Einstufungen in *CIA* sogar weniger unklare Urteile als die Referenz. Gleichzeitig ordnet es 41 Fälle der Kategorie *C* zu, obwohl in Wirklichkeit nur 36 korrekt wären. Die Gesamtgenauigkeit steigt trotzdem kaum, da sich neue Treffer und neue Fehler gegenseitig aufheben. Das zeigt, was ab dem Sättigungspunkt passiert. Der Agent beginnt, gefundene Codezeilen zu überinterpretieren. Er verfällt exakt in das Muster, das bereits in Abschnitt 6.3.2 bei der Analyse der falschen Urteile aufgefallen war.

Fazit für den praktischen Einsatz Aus all diesen Daten ergibt sich eine klare Empfehlung für den Prototyp. Das Standardbudget bietet den besten Kompromiss. Ein kleineres Budget kostet den gesamten Vorteil, während ein größeres nur die Laufzeit in die Höhe treibt. Die maximale Stufe benötigt 158 Sekunden je Fall, das Standardbudget dagegen 74, was mit Blick auf die Anforderung NFA-7 den Ausschlag gibt.
Zur vollständigen Wahrheit gehört ein Blick auf die Gesamtkosten. Vergleicht man den agentischen Ansatz im Standardbudget mit dem statischen System, verbraucht der Agent je Regel rund 51-mal so viele Tokens (125 898 gegenüber 2456), und die Laufzeit steigt um den Faktor 18 auf 74 statt 4 Sekunden. Diese Werte zur Anforderung NFA-7 sind der Preis für die knapp 20 Prozentpunkte Genauigkeit. Sämtliche Einzelwerte aller Durchläufe stehen in Anhang A.3.

6.7. E5 – Variation des Modells

E4 hat gezeigt, dass der Vorsprung nicht einfach an der Menge gesammelter Evidenz hängt, denn oberhalb des Standardbudgets bringt zusätzliche Suche nichts mehr. Damit sind Prompt, Architektur und Suchmenge als alleinige Erklärungen abgearbeitet. Was bleibt, ist das Modell: Alle bisherigen Läufe verwenden `gpt-5.4-mini`, und ein Befund, der nur mit diesem einen Modell auftritt, wäre keine Aussage über das Verfahren. E5 verändert deshalb gegenüber der Basiskonfiguration ausschließlich das Modell und tut das, wie E4 beim Budget, über mehrere Stufen derselben Dimension.

6.7.1. Aufbau des Experiments

Beide Bedingungen aus E1, also der statische und der agentische Judge, werden erneut über dieselben 60 Fälle ausgeführt, einmal je zusätzlicher Modellstufe. Tabelle 6.22 nennt die drei Stufen. Wo das Modell es zulässt, bleibt `reasoning_effort=high` gesetzt. Unverändert bleiben außerdem Prompt, Architektur, Suchbudget, Suchraum, Codestand und Auswertung. Beide Verfahren erhalten je Stufe dasselbe Modell, sodass die Modellwahl den Vergleich innerhalb eines Laufs nicht beeinflusst. Liegt der agentische Judge auf jeder Stufe vorn, liegt der Befund an der Bauart. Verschwindet der Vorsprung auf einer Stufe oder dreht er sich um, lag er am Modell.

Tabelle 6.22.: Modellstufen der Rotation E5. Die Basisstufe entspricht der Basiskonfiguration und damit den Läufen aus E1.

Stufe	Modell	Einordnung
Basis	`gpt-5.4-mini`	Referenzstufe, identisch mit den Läufen aus E1
kleiner	`gpt-5.4-nano`	schwächeres Modell derselben Familie, prüft, ob der Befund auf Modellstärke beruht
drittes Modell	`gpt-5.6-luna`	größtes Kontextfenster der drei Modelle, prüft, ob ein großes Fenster den Evidenzzugang ersetzen kann. Nur ein Lauf je Bedingung

Für `gpt-5.4-nano` liegen drei Läufe je Bedingung vor, für `gpt-5.6-luna` dagegen nur einer. Der Abstand der luna-Stufe wird deshalb nicht der Größe nach mit den Mittelwerten

der beiden anderen Stufen verglichen, sondern nur daraufhin gelesen, ob er in dieselbe Richtung zeigt.

6.7.2. Ergebnis

Tabelle 6.23 stellt alle drei Stufen nebeneinander.

Tabelle 6.23.: Beide Verfahren über die drei Modellstufen, gegen die unabhängige Referenz. Für `mini` und `nano` ist die Accuracy das Mittel aus drei Läufen, MCC und F_1 stammen aus Lauf 1. Für `luna` liegt nur ein Lauf vor.

Verfahren	Modell	Accuracy	Lauf 1	MCC	F_1 (Makro)
Statisch	`gpt-5.4-nano`	46,1 %	42,4 %	0,187	0,355
Agentisch	`gpt-5.4-nano`	**64,4 %**	68,3 %	**0,482**	**0,583**
Statisch	`gpt-5.4-mini`	55,9 %	55,9 %	0,337	0,493
Agentisch	`gpt-5.4-mini`	**75,6 %**	76,7 %	**0,569**	**0,676**
Statisch	`gpt-5.6-luna`	—	48,3 %	0,268	0,380
Agentisch	`gpt-5.6-luna`	—	**75,0 %**	**0,557**	**0,649**

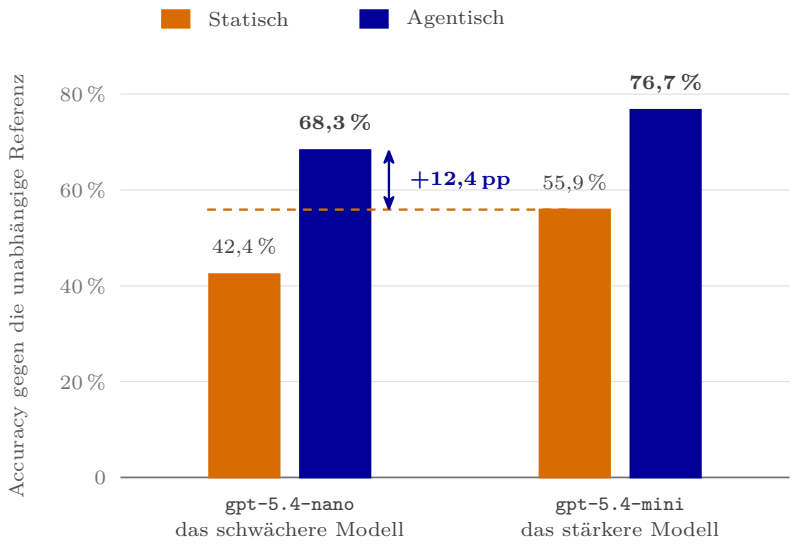


Abbildung 6.5.: Die Genauigkeiten aus Tabelle 6.23. Der agentische Vorsprung tritt bei beiden Modellen auf. Der agentische Lauf mit `gpt-5.4-nano` übertrifft sogar den statischen mit `gpt-5.4-mini`. Eigene Darstellung.

Der Vorsprung tritt auf jeder Stufe auf Auf allen drei Modellen liegt der agentische Judge vorn, und zwar deutlich: 18,3 Prozentpunkte bei `nano`, 19,6 bei `mini` und 26,7 im Einzellauf mit `luna`. Der Befund hängt damit nicht an einem einzelnen Modell. Der Größe nach lassen sich diese drei Werte allerdings nicht ordnen, denn die luna-Stufe beruht auf einem einzigen Laufpaar, während die beiden anderen Mittelwerte sind.

Stufe `gpt-5.4-nano` Auf dem einzelnen Laufpaar ist der Unterschied klar signifikant, denn der agentische Judge liegt in 20 Fällen als Einziger richtig, der statische nur in 4, was $p = 0{,}0015$ ergibt. Über alle drei Läufe widersprechen sich die beiden Tests jedoch: Das Mehrheitsurteil ergibt 17 zu 8 diskordante Fälle bei $p = 0{,}1078$, der Vorzeichentest 28 zu 10 bei $p = 0{,}0051$ ($n = 58$). Dieser Widerspruch ist selbst ein Befund. Er bedeutet, dass die Einzelurteile zwischen den Läufen so stark kippen, dass das auf richtig oder falsch gerundete Mehrheitsurteil Information verliert, die der Vorzeichentest noch sieht. Passend dazu streuen beide `nano`-Bedingungen mit 3,43 und 4,19 Prozentpunkten etwa doppelt so stark wie die entsprechenden `mini`-Läufe. Berichtet werden deshalb beide Werte, und der Vorsprung auf dieser Stufe gilt als deutlich, aber nicht sauber abgesichert.
Inhaltlich ist die Ursache eindeutig: Im statischen Lauf ist das schwächere Modell überfordert, wenn ihm Evidenz fehlt. Es stuft 15 von 19 eigentlich unentscheidbaren Fällen als *NC* ein und setzt die Prompt-Vorgabe „keine Evidenz → Not Compliant“ damit noch unkritischer um als `gpt-5.4-mini`. Sobald es den Code selbst durchsuchen darf, verschwindet dieses Verhalten, und das Suchmuster aus Abschnitt 6.3.1 reproduziert sich. Konfusionsmatrizen und Werkzeugdaten stehen in Anhang A.3.

Stufe `gpt-5.6-luna` Diese Stufe ist die aufschlussreichste der drei. `gpt-5.6-luna` verfügt über das mit Abstand größte Kontextfenster, ist statisch aber mit 48,3 % das *schwächste* der drei Modelle, schwächer noch als `gpt-5.4-mini` mit 55,9 %. Agentisch erreicht es dagegen 75,0 % und liegt damit praktisch gleichauf mit `gpt-5.4-mini`. Der Abstand zwischen beiden Bedingungen beträgt 26,7 Prozentpunkte bei 19 zu 3 diskordanten Fällen und $p = 0{,}0009$. Auch das Suchmuster reproduziert sich, denn mit 223 Trefferzeilen je Fall sammelt der agentische luna-Lauf sogar mehr Evidenz ein als `gpt-5.4-mini` mit 157.
Ein großes Kontextfenster nützt also nichts, wenn der Kontext die entscheidende Stelle gar nicht enthält. Genau das ist die Lage der statischen Baseline: Sie bekommt fünf per Ähnlichkeit gewählte Ausschnitte, und ob die einschlägige Stelle darunter ist, entscheidet nicht die Größe des Fensters, sondern das Retrieval.

Evidenzzugang gegen Modellstärke Der agentische `nano`-Lauf erreicht 64,4 % und übertrifft damit trotz des schwächeren Modells den statischen `mini`-Lauf mit 55,9 % um 8,5 Punkte (Abbildung 6.5). Die luna-Stufe verschärft diese Aussage noch, denn dort verliert das Modell mit dem größten Kontextfenster in der statischen Bedingung gegen zwei kleinere Modelle. Für die Praxis heißt das, dass ein besserer Evidenzzugang mehr bringt als ein größeres Modell oder ein größeres Kontextfenster.

TODO: Abbildung `abb_erg_modellrotation.pdf` muss neu erzeugt werden. Sie zeigt bisher nur zwei Modelle und die Einzellaufwerte; nötig sind drei Stufen und die Mittelwerte für `mini` und `nano`. Die luna-Balken sind als Einzellauf zu kennzeichnen.

6.8. E6 – Variation der Kontextmenge der Baseline

Nach E5 steht fest, dass der Befund nicht an einem einzelnen Modell hängt. Eine Erklärung ist damit aber immer noch unwidersprochen, und es ist die naheliegendste von allen. Der agentische Judge sammelt im Standardbudget ein Vielfaches dessen ein, was die Baseline zu sehen bekommt, denn diese ist auf fünf Ausschnitte und im Mittel 3389 Zeichen festgelegt. Vielleicht urteilt der Agent also gar nicht gezielter, sondern schlicht

auf mehr Text. Träfe das zu, wäre der Kern dieser Arbeit widerlegt, denn dann ließe sich derselbe Gewinn ohne jede Agentik erreichen, indem man einfach den Abrufparameter hochdreht.
E6 stellt der Baseline deshalb genau die Frage, die E4 dem Agenten gestellt hat, nämlich wie viel Evidenz sie überhaupt sehen darf. Erst beide Experimente zusammen trennen die Menge der Evidenz von der Art ihrer Beschaffung.

6.8.1. Aufbau des Experiments

Variiert wird allein top_k, also die Zahl der Chunks, die die Ähnlichkeitssuche je Regel zurückgibt. Unverändert bleiben Chunk-Größe und Überlappung (800 und 200 Zeichen), das Embedding-Modell, der Index selbst, der Original-Prompt, das Modell `gpt-5.4-mini` mit `reasoning_effort=high`, die Fälle, der Suchraum und der Codestand. Es wird also derselbe Index abgefragt, nur werden mehr Nachbarn zurückgegeben. Jede Stufe läuft dreimal.
Die Stufen müssen sich daran ausrichten, wie viel Evidenz der agentische Judge tatsächlich einsammelt. Gemessen über die Ergebnisgrößen aller `grep`- und `read_file`-Aufrufe sind das im Standardbudget im Mittel **67 802 Zeichen** je Regel (Median 59 181, $n = 81$). Tabelle 6.24 stellt dem die vier Stufen der Baseline gegenüber.

Tabelle 6.24.: Kontextstufen der Ablation E6 mit der tatsächlich übergebenen Textmenge. Die Basisstufe entspricht der Konfiguration der Originalarbeit und damit dem statischen Lauf aus E1. Bezugsgröße der letzten Spalte sind die 67 802 Zeichen, die der agentische Judge im Standardbudget je Regel einsammelt.

Stufe	top_k	Kontext je Regel	Anteil am Agenten	Wiederholungen
Basis	5	3389 Zeichen	5 %	3 (aus E1)
mittel	20	13 838 Zeichen	20 %	3
hoch	50	35 413 Zeichen	52 %	3
angeglichen	100	71 648 Zeichen	106 %	3

Ursprünglich waren nur drei Stufen bis top_k = 50 vorgesehen. Die Messung zeigte jedoch, dass diese Stufe erst gut die Hälfte der Agentenmenge erreicht, weshalb eine vierte Stufe bei top_k = 100 ergänzt wurde. Erst sie erfüllt die Bedingung, auf die es hier ankommt: Beide Verfahren sehen dann annähernd gleich viel Code, und verschieden bleibt allein, ob dieser Code vorab per Ähnlichkeit ausgewählt oder gezielt gesucht wurde.
Drei Ausgänge sind denkbar und jeder von ihnen wäre ein verwertbarer Befund. Bleibt die Lücke zum agentischen Judge über alle Stufen bestehen, ist die Menge der Evidenz nicht der wirksame Mechanismus. Schließt sie sich, geht ein erheblicher Teil des in E1 gemessenen Vorsprungs allein auf die Kontextmenge zurück. Sinkt die Genauigkeit bei der höchsten Stufe sogar wieder, verdünnt zusätzlicher Kontext die einschlägige Stelle, was zu dem Befund aus Abschnitt 6.3.2 passen würde.

6.8.2. Ergebnis

Die Baseline gewinnt allein durch mehr Chunks deutlich hinzu, aber sie holt den agentischen Judge nicht ein (Tabelle 6.25).

Tabelle 6.25.: Genauigkeit der Baseline über die vier Kontextstufen, Mittel aus je drei Läufen. Die letzte Zeile nennt zum Vergleich den agentischen Judge in der Basiskonfiguration.

Stufe	Accuracy	Standardabw.	MCC (Lauf 1)	Abstand zum Agenten
top_k = 5 (Basis)	55,9 %	1,69 pp	0,337	−19,6 pp
top_k = 20	62,0 %	2,23 pp	0,426	−13,6 pp
top_k = 50	64,2 %	2,13 pp	0,409	−11,4 pp
top_k = 100	66,7 %	3,12 pp	0,422	**−8,9 pp**
Agentisch, Standardbudget	**75,6 %**	1,92 pp	0,569	—

Die Menge erklärt gut die Hälfte Von den 19,6 Prozentpunkten Abstand aus E1 schließen sich 10,8 allein dadurch, dass die Baseline mehr Chunks bekommt. Das ist deutlich mehr, als diese Arbeit bis hierher angenommen hatte, und es relativiert den Befund aus E1 erheblich. Wer nur die Kontextmenge angleicht, ohne irgendetwas an der Art der Beschaffung zu ändern, holt also den größeren Teil des Rückstands auf.

Ein Rest bleibt, ist aber nicht abgesichert Bei annähernd gleicher Evidenzmenge, also top_k = 100 gegen das agentische Standardbudget, bleiben 8,9 Prozentpunkte Abstand. Dieser Rest ist bei 60 Fällen jedoch nicht signifikant. Das Mehrheitsurteil über drei Läufe ergibt 11 zu 4 diskordante Fälle bei $p = 0{,}1185$, der Vorzeichentest 14 zu 8 bei $p = 0{,}2863$ ($n = 58$); auf dem einzelnen Laufpaar liegt der Wert bei $p = 0{,}0768$. Alle drei Werte verfehlen die Fünf-Prozent-Schwelle. Der verbleibende Abstand wird deshalb ausdrücklich *deskriptiv* berichtet und nicht als gesicherter Effekt.

Zwei Einschränkungen Erstens hat die Kurve auf der höchsten Stufe noch nicht gesättigt, denn der Zuwachs von top_k = 50 auf 100 beträgt noch 2,5 Prozentpunkte. Ob die Baseline mit noch mehr Kontext weiter aufholt, ist damit offen. Zweitens wächst die Streuung mit der Stufe, von 1,69 auf 3,12 Prozentpunkte. Mehr Kontext macht die Baseline also, wie schon beim Agenten in E4, zugleich unzuverlässiger.

Befund Von den drei vorab benannten Ausgängen ist der zweite eingetreten, allerdings nur zur Hälfte. Die Alternativerklärung „der Agent sieht schlicht mehr Code“ erklärt nachweislich gut die Hälfte des in E1 gemessenen Vorsprungs. Sie erklärt ihn aber nicht vollständig, denn bei angeglichener Menge bleibt ein Abstand von 8,9 Punkten. Dass dieser Rest auf die gezielte Beschaffung entfällt, ist die plausibelste Deutung, mit diesen Daten aber nicht statistisch abgesichert.

TODO: Für diesen Abschnitt fehlt noch eine Abbildung analog zu Abbildung 6.4: top_k auf der x-Achse, Accuracy mit Fehlerbalken über die drei Läufe, dazu die agentische Genauigkeit als waagerechte Referenzlinie. Das macht den Befund schneller lesbar als die Tabelle.

6.9. Zusammenführung: was von den Erklärungen übrig bleibt

Die Reihe war als Ausschlussverfahren angelegt. E1 hat einen Unterschied von 19,6 Prozentpunkten festgestellt, für den fünf Erklärungen in Frage kamen, und die folgenden

fünf Experimente haben sich jede einzeln vorgenommen. Tabelle 6.26 hält fest, was dabei herausgekommen ist.

Tabelle 6.26.: Stand des Ausschlussverfahrens nach der Experimentreihe.

Exp.	Erklärung	Stand
E2	Prompt-Formulierung	**bestätigt, und mehr als erwartet.** Der Prompt bringt dem statischen Judge 10,7 pp und dem agentischen 15,2 pp. Beide Effekte greifen ineinander, eine Aufteilung in feste Anteile ist deshalb nicht definierbar
E3	Rollentrennung Locator/Reader	**bestätigt im Mechanismus, nicht in der Genauigkeit.** Ohne Trennung öffnet der Agent in 60 % der Fälle keine Datei (40 gegenüber 83 % Lesezugriff). Der Genauigkeitsabstand von 6,7 pp ist mit $p = 0{,}1250$ nicht signifikant
E4	Menge der gesammelten Evidenz	**verworfen.** Oberhalb des Standardbudgets bringt zusätzliche Suche nur noch 0,5 pp, und mehr Werkzeugaufrufe gehen mit *falschen* Urteilen einher
E5	Abhängigkeit vom Modell	**verworfen.** Der Vorsprung tritt auf allen drei Modellen auf, mit 18,3 pp bei `nano`, 19,6 pp bei `mini` und 26,7 pp im Einzellauf mit `luna`
E6	Kontextmenge der Baseline	**zur Hälfte bestätigt.** Allein durch mehr Chunks schließen sich 10,8 der 19,6 pp. Bei angeglichener Menge bleiben 8,9 pp, die mit $p = 0{,}1185$ aber nicht signifikant sind

Das Ergebnis ist nicht die schlichte Bestätigung der Ausgangsannahme, sondern ihre Verschiebung. Zwei der fünf Erklärungen sind ausgeräumt, drei haben sich ganz oder teilweise bestätigt. Der Vorsprung aus E1 besteht, aber er entsteht nicht durch den Werkzeugzugang als solchen.
Sichtbar wird das an den beiden Experimenten, die dem Werkzeugzugang etwas wegnehmen, ohne ihn abzuschaffen. Nimmt man dem Agenten die Urteilsanweisung, die fehlende Evidenz zulässt, fällt er auf 60,3 % und damit fast auf Baseline-Niveau, obwohl er weiterhin das ganze Repository durchsuchen darf (E2). Nimmt man ihm die Rollentrennung, nutzt er seine Werkzeuge nur noch zur Hälfte und liest in der Mehrzahl der Fälle überhaupt keine Datei (E3). Nimmt man ihm das Suchbudget, verschwindet der Vorsprung vollständig (E4). Der Werkzeugzugang ist also notwendig, aber allein nicht hinreichend.
Was bleibt, ist ein engerer und zugleich belastbarerer Befund. Der Vorsprung entsteht aus dem Zusammenspiel dreier Bedingungen: einer Urteilsanweisung, die „nichts gefunden“ von „widerspricht der Regel“ trennt; einer Architektur, die Suchen und Lesen voneinander trennt; und einem Suchbudget oberhalb der Mindestschwelle. Fällt eine davon weg, bleibt vom Vorsprung wenig übrig.

Was die Reihe nicht zeigen kann Ein Teil der Befunde ist der Größe nach messbar, aber nicht statistisch abgesichert. Signifikant sind der Hauptvergleich E1 ($p = 0{,}0023$), der Prompt-Effekt beim agentischen Judge E2b ($p = 0{,}0075$) und die Modellstufen. Nicht signifikant sind die Rollentrennung E3 ($p = 0{,}1250$) und der bei angeglichener Kontextmenge verbleibende Abstand E6 ($p = 0{,}1185$). Das ist kein Zufall, sondern eine Eigenschaft des

Aufbaus: Bei 60 Fällen und daraus resultierenden sieben bis fünfundzwanzig diskordanten Fällen lassen sich Differenzen in der Größenordnung von fünf bis neun Prozentpunkten nicht absichern. Die Studie ist für die großen Unterschiede ausreichend dimensioniert und für die mittleren nicht. Wo im Folgenden mit nicht signifikanten Differenzen argumentiert wird, geschieht das deshalb ausdrücklich beschreibend und gestützt auf die Werkzeugprotokolle, nicht auf die Genauigkeit allein.

6.10. Beantwortung der Forschungsfragen

Aus den Einzelbefunden lassen sich damit die drei Forschungsfragen aus Abschnitt 1.3 beantworten. Tabelle 6.27 zeigt vorab, welche Experimente jeweils tragen und mit welchen Zahlen.

Tabelle 6.27.: Beitrag der Experimente und Auswertungen zu den drei Forschungsfragen.

Frage	Gegenstand	Grundlage	Tragende Zahlen
FF1	Nutzen der agentischen Evidenzbeschaffung	E1, Kontrolllauf, E5	75,6 gegenüber 55,9 % Accuracy, MCC 0,569 gegenüber 0,337, $p = 0{,}0023$ im Mehrheitstest, Messstreuung 1,7–1,9 pp. Tritt auf allen drei Modellen auf (+18,3, +19,6, +26,7 pp)
FF2	Bessere Evidenzlage oder bloßer Prompting-Effekt	E2, E3, E4, E6 und die Protokollauswertungen	Prompt-Effekt +10,7 pp statisch und +15,2 pp agentisch, Wechselwirkung +4,5 pp. Kontextmenge erklärt 10,8 der 19,6 pp; die verbleibenden 8,9 pp sind mit $p = 0{,}1185$ nicht abgesichert. Ohne Rollentrennung 40 statt 83 % Lesezugriff
FF3	Praxisgerechte Integration und ihre Grenzen	E3, E4, Protokollauswertungen, Konfusionsmatrizen	74 gegenüber 4 Sekunden und das 51-Fache an Tokens je Regel. *NC*-Precision 37,5 %, unter restriktivem Prompt nur 6,7 %. Rollentrennung als Entwurfsvorgabe. Die Frage selbst beantwortet Kapitel 7

FF1 – Inwiefern verbessert eine agentische Evidenzbeschaffung die Erkennung von Architekturverstößen? Die Verbesserung ist deutlich und statistisch abgesichert. Der agentische Judge erreicht im Mittel dreier Läufe 75,6 %, die statische Baseline 55,9 %. Der Abstand von 19,6 Prozentpunkten übersteigt die Messstreuung um das Zehnfache und ist im konservativeren der beiden Tests mit $p = 0{,}0023$ signifikant. Er tritt auf allen drei geprüften Modellen auf und ist damit keine Eigenschaft eines bestimmten Modells.
Für den praktischen Einsatz bleibt eine gewichtige Einschränkung. Die Precision bei der Erkennung echter Regelverstöße der Kategorie *NC* liegt auch beim agentischen Judge bei lediglich 37,5 % (Tabelle 6.15). Mehr als die Hälfte der gemeldeten Verstöße sind also Fehlalarme, und da genau solche Warnungen der Zweck des Systems sind (NFA-10), ist das der schwächste Punkt des Verfahrens. Der Wert beruht allerdings auf nur fünf echten Fällen und ist statistisch nicht belastbar (Abschnitt 8.3) **frage: hier sind nur 5 fälle bei nc rausgekommen. reicht das wirklich? oder müssen für die belastbarkeit mehr fälle gesucht werden** . Zusammenfassend ist der agentische Judge das genaueste der

geprüften Verfahren, aber keines, dessen Verstoßmeldungen man ungeprüft übernehmen sollte.

FF2 – Worauf lässt sich der Leistungsunterschied zurückführen? Nicht auf den Werkzeugzugang allein. Die Experimentreihe zerlegt den Unterschied nicht in saubere Anteile, sondern zeigt drei Bedingungen, die zusammenwirken müssen.
Die erste ist die *Urteilsanweisung*. Der Prompt bringt dem statischen Judge 10,7 Prozentpunkte und dem agentischen sogar 15,2, und beide Effekte hängen voneinander ab: Der Architekturvorteil beträgt unter der restriktiven Vorgabe 4,4 Punkte und unter der permissiven 8,9 (Tabelle 6.12). Eine Aufteilung in Prozentanteile, wie ein früherer Auswertungsstand sie vorsah, ist deshalb nicht definierbar, denn sie würde diese Wechselwirkung von 4,5 Punkten gerade verdecken. Entgegen der Erwartung reagiert der agentische Judge sogar *stärker* auf die Formulierung als der statische; Werkzeugzugang macht die Anweisung also nicht entbehrlich.
Die zweite ist die *Menge der Evidenz*, und ihr Anteil ist größer als angenommen. Allein dadurch, dass die Baseline mehr Chunks erhält, schließen sich 10,8 der 19,6 Prozentpunkte (E6). Bei angeglichener Evidenzmenge bleiben 8,9 Punkte, die aber mit $p = 0{,}1185$ nicht abgesichert sind. Dass dieser Rest auf die gezielte Beschaffung entfällt, ist die plausibelste Deutung, mehr als das geben die Daten nicht her.
Die dritte ist die *Architektur*. Ohne Rollentrennung nutzt ein Agent mit denselben Werkzeugen und demselben Budget sie nur zur Hälfte und öffnet in 60 % der Fälle keine einzige Datei (E3). Der Genauigkeitsunterschied von 6,7 Punkten ist zwar nicht signifikant, das Verhaltensmuster dahinter aber eindeutig.
Worüber der Vorteil wirkt, zeigen die Protokolle: Der Agent erkennt, wenn Evidenz im Suchraum nicht existiert, weil seine Suchen ins Leere laufen (4,29 gegenüber 1,26 leere `grep`-Aufrufe bei *CIA*-Urteilen). Er urteilt dabei nicht besser, weil er mehr sucht, denn hohe Aufrufzahlen gehen mit *falschen* Urteilen einher. Entscheidend ist nicht der Umfang der Suche, sondern ihr Erfolg.

FF3 – Wie lässt sich der Ansatz praxisgerecht integrieren? Die vollständige Beantwortung erfolgt in Kapitel 7. Die Reihe liefert dafür vier Vorgaben. Erstens steht die Konfiguration samt Kosten fest: Im Standardbudget benötigt eine Regelprüfung 74 Sekunden und rund 126 000 Tokens, gegenüber 4 Sekunden und 2456 Tokens beim statischen Verfahren (NFA-7). Zweitens ist die Rollentrennung keine Feinheit, sondern konstitutiv, denn ohne sie liest das System die gefundenen Stellen mehrheitlich gar nicht. Drittens muss die Urteilsanweisung „nichts gefunden“ und „widerspricht der Regel“ strikt trennen; unter der restriktiven Fassung fällt die *NC*-Precision auf 6,7 %, das heißt vierzehn von fünfzehn Meldungen wären Fehlalarme. Viertens liefert der Suchaufwand ein Warnsignal: Ein Urteil, für das der Agent auffällig lange suchen musste, ist fehleranfälliger und sollte im Merge Request zurückhaltend oder gar nicht angezeigt werden.

7. Konzeption und technische Umsetzung

Kapitel 3 hat die Forschungslücke benannt, Kapitel 6 die tragende Annahme dieser Arbeit geprüft. Dieses Kapitel baut daraus ein lauffähiges System. Es setzt am Merge Request an, prüft den Code gegen die ADRs des Projekts und schreibt seine Befunde als Kommentar zurück. Die wichtigsten Entscheidungen dieses Systems sind nicht geraten, sondern gemessen. Tabelle 7.1 stellt jedem Messbefund die Entscheidung gegenüber, die daraus folgt. Im weiteren Verlauf ist jeweils genannt, worauf eine Festlegung beruht.

Tabelle 7.1.: Messbefunde aus Kapitel 6 und die daraus abgeleiteten Entscheidungen für den Prototyp.

Befund	Konsequenz für den Prototyp
Der agentische Judge übertrifft den statischen um 19,6 Prozentpunkte. Ohne Rollentrennung öffnet ein Agent mit denselben Werkzeugen in 60 % der Fälle keine einzige Datei (Abschnitte 6.2, 6.5)	Der Prüfschritt gibt dem Modell Werkzeuge über den Quellbaum und übernimmt die Trennung von Locator und Reader unverändert (Abschnitt 7.2.4)
60 % der unentscheidbaren Fälle sind Artefakte einer Datei-Whitelist, die nur Programmiersprachen erfasst (Anhang A.3.2)	Der Suchraum des Prototyps umfasst ausdrücklich auch CI-, Build- und Konfigurationsdateien (Abschnitt 7.2.4)
Eine einzelne Prompt-Anweisung, die fehlende Evidenz als Verstoß wertet, erklärt 46 % des gemessenen Unterschieds (Abschnitt 6.4)	Der Prototyp trennt „widerspricht der Regel“ und „nichts gefunden“ strikt und meldet nur Ersteres als Befund (Abschnitt 7.3)
Falsche Urteile gingen mit *mehr* Werkzeugaufrufen einher als richtige, und ab acht Suchrunden steigt die Genauigkeit nicht weiter (Abschnitte 6.3.2, 6.6)	Das Werkzeugbudget wird auf den gemessenen Sättigungspunkt gesetzt, seine Ausschöpfung als Zeichen von Unsicherheit gewertet (Abschnitt 7.2.4)
Der Vorteil reproduziert sich mit einem schwächeren Modell und fällt dort größer aus (Abschnitt 6.7)	Der Prototyp bindet sich an kein bestimmtes Modell. Die Gateway-Auswahl schränkt die Urteilsgüte damit nicht ein (Abschnitt 7.1)

Nicht jede Entscheidung folgt aus einer Messung. Die Zerlegung der ADRs in einzelne Regeln und die Vorauswahl der passenden Regeln (Abschnitte 7.2.2 und 7.2.3) stützen sich auf die Literatur zum Distraktor-Kontext. Die Anbindung an GitLab folgt schlicht aus dem Einsatzkontext. Beruht eine Festlegung nur auf Plausibilität, ist das im Text gesagt.

7.1. Auswahl der Werkzeugkette für den Proof-of-Concept

Drei Vorgaben stehen von vornherein fest. Erstens muss das System am Merge Request ansetzen, weil dort die Qualitätssicherung zusammenläuft (Abschnitt 2.5) [36]. Die CI/CD-Pipeline darf es dabei nicht aufhalten (FA-12). Zweitens muss es über den verfügbaren Hochschul-Gateway laufen, damit der Quellcode die Organisation nicht verlässt (NFA-5), und es darf an kein bestimmtes Modell gebunden sein (NFA-1). Drittens braucht es eine abgeschottete Ausführungsumgebung, denn der Prüfschritt klont einen fremden Quellbaum und führt darauf Suchbefehle aus (NFA-6).
Eine vierte Vorgabe stammt aus den Messungen. Das Modell muss seine Belege selbst im Repository suchen können. Abschnitt 6.2 weist für diesen Aufbau 19,6 Prozentpunkte Vorsprung nach, und die Experimentreihe zeigt zugleich, dass dieser Vorsprung an drei Bedingungen zugleich hängt, nämlich an der Urteilsanweisung, der Rollentrennung und einem ausreichenden Suchbudget (Abschnitt 6.9). Damit fällt jedes Werkzeug weg, das seinen Kontext vorab zusammenstellt und danach nicht mehr ändert. Zusammen mit der geforderten lückenlosen Protokollierung (NFA-3) fallen auch die fertigen Dienste und die Agenten-Frameworks weg, denn sie kapseln genau die Schleife, die hier kontrollierbar bleiben muss. Die Agentenschleife ist deshalb wieder direkt gegen die Chat-Completions-Schnittstelle des Gateways geschrieben. Tabelle 7.2 zeigt die gewählten Bausteine.

Tabelle 7.2.: Komponenten der Werkzeugkette des Proof-of-Concepts.

Aufgabe	Gewählte Komponente	Begründung
Auslösung des Reviews	GitLab-Webhook auf Merge-Request-Ereignisse [55]	Löst bei einem Ereignis aus und läuft neben der Pipeline. Der Review hält keinen Merge auf
Dienst und Oberfläche	Next.js (TypeScript)	Ein Prozess für Webhook-Endpunkt, Ergebnis-Callback und die Anzeige des Review-Verlaufs. Die Oberfläche dient der Nachvollziehbarkeit, nicht dem Produktivbetrieb
Anbindung an GitLab	`@gitbeaker/rest` [12] sowie die Discussions-Schnittstelle der REST-API	Typisierter Zugriff auf Diffs und Merge-Request-Daten. Inline-Diskussionen brauchen eine Positionsangabe, die der Client selbst nicht abbildet
Ausführungsumgebung der Prüfung	Ephemerer Container (Podman) je Merge Request	Eigener Dateibaum ohne Zugriff nach außen, Abbau nach dem Lauf, Werkzeugaufrufe bleiben im Containerprotokoll sichtbar
Modellzugriff	OpenAI-kompatible Schnittstelle des Gateways KI-Connect-NRW	Einzig verfügbarer Zugang und damit auch die Vorgabe für die Modellauswahl
Agentenschleife	Eigenimplementierung gegen die Chat-Completions-Schnittstelle	Kein Framework, damit Werkzeugauswahl, Schrittgrenze und Protokollierung vollständig kontrollierbar bleiben
Architekturwissen	MADR-basierte ADRs im Repository [24]	Versionierte, textbasierte Ablage gemeinsam mit dem Code, wie es das Prinzip „Docs-as-Code“ vorsieht [5]

TODO: Wirtschaftlichkeit ergänzen. Kosten je Million Token für `gpt-5.4-mini` und `gpt-5.4-nano` (Quelle etwa models.dev) und daraus die ungefähren Kosten eines Reviews je Regel abschätzen. Die Zahl der Modellaufrufe je Regel liegt aus Abschnitt 6.6 vor.

Diese Werkzeugkette konkurriert nicht mit den Werkzeugen aus Abschnitt 3.1. Sie ist deutlich weniger ausgereift und hat weder die Linter-Anbindung von CodeRabbit [10] noch die Filterstufen von BitsAI-CR [50]. Dafür liest sie die Architekturdokumentation des Projekts und schließt damit die Lücke aus Abschnitt 3.2.4. Sie zeigt, dass diese Anbindung machbar ist, und ist kein fertiges Produkt.

7.2. Gesamtkonzept und Ablauf eines Reviews

Der Ablauf verteilt sich auf drei Orte. In GitLab entsteht der Anlass und landet am Ende das Ergebnis. Das Next.js-Backend läuft dauerhaft und nimmt die Meldungen entgegen. Geprüft wird weder dort noch in der CI/CD-Pipeline, sondern in einer Sandbox, die für den Lauf gestartet und danach wieder gelöscht wird. Abbildung 7.1 zeigt den ganzen Weg, den die folgenden Unterabschnitte Schritt für Schritt durchgehen.

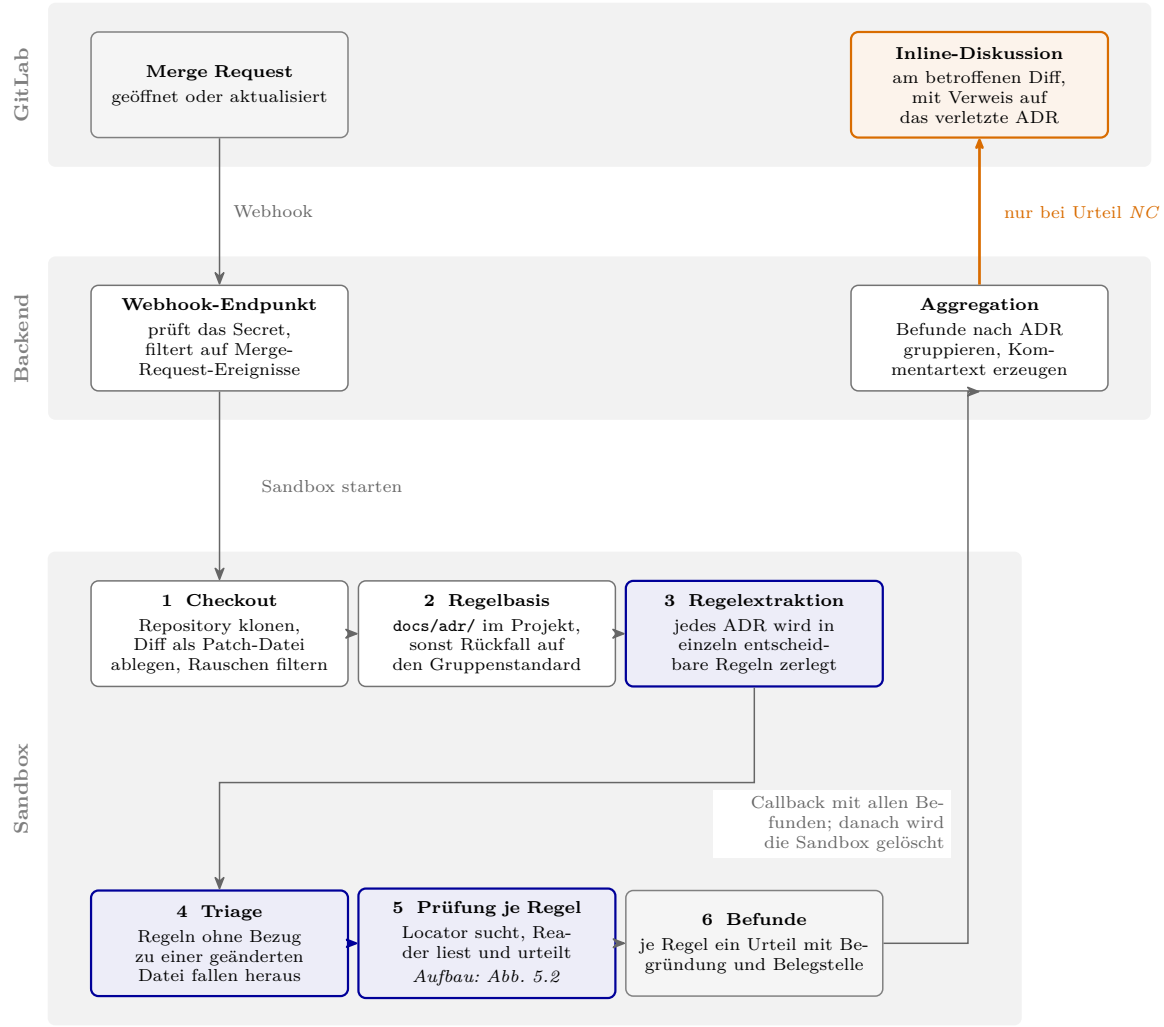


Abbildung 7.1.: Ablauf eines Reviews im Prototyp, von der Merge-Request-Meldung bis zur Inline-Diskussion. Blau hervorgehoben sind die Schritte, in denen ein Sprachmodell arbeitet, orange die Stelle, an der aus einem Urteil ein Kommentar wird. Das Innenleben der Prüfschleife ist nicht wiederholt, es entspricht Abbildung 5.2. Eigene Darstellung.

7.2.1. Auslösung und Ausführungsumgebung

Den Anstoß gibt GitLab über einen Webhook auf Merge-Request-Ereignisse [55]. Das Backend prüft zuerst das mitgesandte Geheimnis, damit niemand von außen einen Review auslösen kann. Danach filtert es auf den Ereignistyp, sodass nur neue oder um Commits ergänzte Merge Requests einen Lauf starten. Der naheliegende Alternativweg wäre ein zusätzlicher Job in der CI/CD-Pipeline gewesen. Ein Prüflauf mit mehreren Modellaufrufen je Regel würde die Pipeline aber für Minuten belegen und hinge an deren projektspezifischer Konfiguration. Der Webhook trennt beides voneinander. Pipeline und Review laufen nebeneinander, und ein hängender Review hält keinen Merge auf (FA-12, NFA-9). Damit die Entwickelnden trotzdem sehen, dass etwas läuft, setzt das Backend einen kurzen Hinweiskommentar an den Merge Request und löscht ihn wieder, sobald das Ergebnis vorliegt.

Das Backend startet dann einen Container für den gesamten Prüflauf. Dieser klont das Repository auf den Stand des Merge Requests und erzeugt den Diff gegen den Zielbranch. Dabei fliegt heraus, was für eine Architekturaussage nichts beiträgt, aber Kontext kostet, also Sperrdateien der Paketverwaltung, Binärformate und generierte Grafiken (FA-13). Die Kapselung erfüllt gleich drei Anforderungen. Der Container hat einen eigenen Dateibaum, den die Werkzeuge des Agenten nicht verlassen können (NFA-6). Er wird je Merge Request neu erzeugt und danach gelöscht, sodass nichts aus einem Review in den nächsten überläuft. Und in ihm werden jeder Werkzeugaufruf und jede Modellantwort protokolliert (NFA-3). Nur so lässt sich ein unplausibler Befund später auf den Aufruf zurückführen, der ihn ausgelöst hat.

Der gefilterte Diff geht nicht als Text in den Modellaufruf, sondern liegt als Datei `mr_diff.patch` im geklonten Repository. Der Grund ist eine Grenze des Betriebssystems, das die Länge von Kommandozeilenargumenten deckelt (`ARG_MAX`, meist rund 130.000 Zeichen). Liest der Agent den Diff mit demselben Werkzeug wie den übrigen Quellbaum, fällt diese Grenze weg und das System kommt auch mit großen Änderungssätzen zurecht (NFA-8). Der Diff ist dann nur eine weitere Datei im Suchraum.

7.2.2. Regelbasis und Zerlegung in Prüfregeln

Als Nächstes klärt die Sandbox, gegen welche Architekturentscheidungen überhaupt geprüft wird. Sie sucht zuerst im geklonten Repository nach ADRs unter `docs/adr/`. Findet sie dort nichts, greift sie auf eine organisationsweite Regelbasis zurück, die als Gruppenstandard im Container mitgeliefert wird (FA-3). Das bildet die Lage in Unternehmen ab, in denen abteilungsweite Basisregeln und eigene Vorgaben einzelner Dienste nebeneinander gelten. Vor allem lässt sich ein neues Repository so ohne jede Vorbereitung prüfen (NFA-9).

Ein ADR ist allerdings kein Prüfkriterium, sondern ein Dokument. Es begründet eine Entscheidung, wägt Alternativen ab und nennt Folgen. Meist stecken darin mehrere Festlegungen, die im Code an verschiedenen Stellen auftauchen. Das ADR in Anhang A.1 verlangt gleich drei Dinge. Externe Aufrufe müssen unterhalb von `services/` liegen, außerhalb davon darf kein HTTP-Client verwendet werden, und jede Service-Funktion muss ein Zeitlimit setzen. Ein einziges Urteil über dieses Dokument wäre wertlos, weil unklar bliebe, welcher der drei Punkte gemeint ist. Vorgeschaltet ist deshalb ein eigener Modellaufruf, der jedes ausgewählte ADR in einzeln entscheidbare Regeln zerlegt (FA-1). Jede Regel ist ein Satz mit genau einer Aussage und führt den Titel ihres ADR mit. Geprüft wird danach nicht das Dokument und nicht der Merge Request, sondern das Paar aus

einer Regel und einem Codestand. Genauso waren auch die Messungen aufgebaut.
Der Schritt löst noch ein zweites Problem. Ein ADR enthält Text, der für Menschen wichtig, für ein prüfendes Modell aber schädlich ist, allen voran die verworfenen Alternativen. Solcher Ballast im Kontext verschlechtert die Schlussfolgerungen von Sprachmodellen messbar [47]. Verworfene Optionen sind dabei besonders heikel, weil sie meist nur indirekt verneint werden und Modelle mit negativen Vorgaben erkennbar Mühe haben [17]. Die Regelextraktion räumt das mit weg, denn zur Regel wird nur, was Teil der getroffenen Entscheidung ist (FA-2). Im Repository bleibt das vollständige MADR für die menschlichen Leser erhalten.
Diese Entscheidung hätte auch anders ausfallen können. Eine frühere Fassung dieser Arbeit wollte das MADR-Format um formale Prüfsektionen erweitern und die Regeln dort direkt hinterlegen. Das hätte die Zerlegung gespart, aber jedes Projekt gezwungen, seine Architekturdokumentation in ein nicht standardisiertes Format zu überführen, bevor der erste Review laufen kann. Damit widerspricht die Variante NFA-9 und entwertet zugleich den Rückfall auf eine organisationsweite Regelbasis (FA-3). Die gewählte Lösung kostet dafür einen zusätzlichen Modellaufruf je Dokument, und die Zerlegung selbst kann fehlerhaft ausfallen.

7.2.3. Vorauswahl der einschlägigen Regeln

Vor der eigentlichen Prüfung steht ein sehr kurzer Modellaufruf. Er sieht nur die Liste der geänderten Dateipfade und die extrahierten Regeln in Kurzform. Daraus entscheidet er, welche Regeln für diesen Änderungssatz überhaupt in Frage kommen (FA-4). Ändert sich etwa nur ein Stylesheet, fallen die Regeln zur Datenbankanbindung sofort heraus. Der Schritt setzt um, was Abschnitt 2.2.8 theoretisch begründet hat. Wandern alle hinterlegten Vorgaben ungefiltert in die Prüfung, verliert das Modell durch die irrelevanten Anteile messbar an Präzision [47]. Die Vorauswahl spart Modellaufrufe und Laufzeit. Wichtiger ist, dass sie Fehlalarme senkt, denn eine Regel, die nie geprüft wird, kann auch nicht falsch als verletzt gemeldet werden.
Der Schritt hat eine klare Schwäche. Er entscheidet allein auf Dateipfaden und damit auf sehr dünner Grundlage. Eine Regel kann verloren gehen, wenn ihre Verletzung in einer unauffällig benannten Datei steckt. Die Alternative wäre, jedes Mal alle Regeln zu prüfen. Das kostet Laufzeit im Verhältnis zur Größe der Regelbasis und wäre bei einem organisationsweiten Regelsatz nicht mehr tragbar. Der Prototyp nimmt deshalb das Risiko der übersehenen Regel in Kauf. Der Fehler geht dabei in Richtung Übersehen und nicht in Richtung Fehlalarm, und das ist für die Akzeptanz die bessere Richtung (NFA-10).

7.2.4. Prüfung der Einzelregel

Jede verbliebene Regel bekommt einen eigenen Prüflauf. So vermischen sich mehrere Regeln nicht im selben Kontext und das Modell verwechselt keine Vorgaben. Innerhalb eines Laufs arbeitet genau das Verfahren, das Kapitel 6 vermessen hat, und der Prototyp übernimmt es unverändert. Der *Locator* hat nur die Suchwerkzeuge `list_files` und `grep`. Er geht damit Schritt für Schritt durch den Quellbaum und meldet am Ende Kandidatenstellen mit Pfad und Zeilenbereich. Der *Reader* hat nur `read_file`, öffnet diese Stellen und fällt das Urteil. Reichen ihm die Belege nicht, startet das Programm eine gezielte Nachsuche über den Locator (Abbildung 5.2).
Der Unterschied zu einem klassischen Linter ist groß. Statt Muster im Text zu suchen,

arbeitet der Agent am tatsächlichen Quellbaum und zieht daraus Schlüsse. So findet er auch Verstöße, die über mehrere Dateien verteilt sind und an keiner einzelnen Stelle vollständig sichtbar werden. Der Änderungssatz ist dabei nur der Ausgangspunkt. Der Agent liest die Patch-Datei, um zu wissen, was sich geändert hat, beurteilt die Regel aber am Zustand des ganzen Repositorys nach der Änderung. Gewählt ist dieser Aufbau wegen der Messung und nicht, weil er naheliegt. Abschnitt 6.2 weist ihm gegenüber einer vorab zusammengestellten Auswahl 19,6 Prozentpunkte Vorsprung nach ($p = 0{,}0023$). Der Grund liegt nicht darin, dass der Agent mehr Material sichtet. Er kann feststellen, wenn etwas *nicht* vorhanden ist (Abschnitt 6.3.1). Ein vorab gefüllter Kontext kann das nicht, weil er immer nur eine Auswahl zeigt.

Suchraum Der Agent sieht mehr als nur Quellcode-Dateien. Diese Festlegung geht auf Anhang A.3.2 zurück. Die Mehrheit der dort als nicht entscheidbar eingestuften Regeln hat Belege im Repository, nur liegen die in der CI-Konfiguration, in Build-Dateien oder in Linter-Regeln. Ein Prüfsystem, das diese Dateien ausblendet, kann einen großen Teil realer Architekturentscheidungen gar nicht beurteilen. Der Prototyp hält sich deshalb nicht an die Whitelist des Messaufbaus, die dort nur als Kontrollvariable diente (Abschnitt 5.1.3). Ausgeschlossen sind allein die ADR-Dateien selbst, damit die geprüfte Regel sich nicht als eigener Beleg wiederfindet (FA-5).

Werkzeugbudget Die Zahl der Werkzeugaufrufe je Regel ist nach oben begrenzt. Erreicht der Agent diese Grenze, gilt das als Zeichen von Unsicherheit und nicht als Anlass, weiterzusuchen (FA-7). Auch das folgt aus einer Messung und nicht nur aus Kostengründen. Nach Abschnitt 6.3.2 brauchten falsche Urteile *mehr* Werkzeugaufrufe als richtige. Ein Agent, der lange sucht, hat die Antwort meist nicht gefunden, sondern sich in ähnlichen Themen verlaufen. Die konkrete Grenze ist ebenfalls gemessen. Die Ablation in Abschnitt 6.6 zeigt eine Sättigungskurve, und der Prototyp übernimmt die Stufe an deren Knick, also acht Suchrunden, fünf Datei-Öffnungen und zwei Nachsuchen. Beide Richtungen sind teuer. Ein zu kleines Budget lässt den Vorteil des Werkzeugzugriffs fast verschwinden. Ein zu großes kostet Laufzeit ohne Gegenwert und verleitet das Modell dazu, schwache Hinweise überzubewerten.

7.2.5. Rückweg aus der Sandbox

Die Prüfläufe laufen nacheinander ab und liefern jeweils eine eigene JSON-Antwort mit Urteil, Begründung und Belegstelle (FA-10). Ein Skript in der Sandbox sammelt die Antworten ein und schickt sie gebündelt per HTTP-Callback an das Backend. Danach wird der Container beendet und gelöscht, und der geklonte Quellcode existiert nirgends mehr. Das Backend sortiert die Befunde nach dem ADR, aus dem ihre Regel stammt, und baut daraus den Kommentartext. Jeder Befund führt seine Herkunft mit, etwa `[Global] 001-datenbank.md`. So ist sofort sichtbar, ob eine Entscheidung des Projekts oder eine Vorgabe des Gruppenstandards betroffen ist. Antworten im falschen Format werden verworfen und als Fehlschlag protokolliert, statt als leeres Urteil ins Ergebnis zu wandern (NFA-2).

7.3. Rückmeldung im Merge Request

Zum Schluss gehen die Befunde dorthin, wo die Entwickelnden ohnehin arbeiten. Über die Discussions-Schnittstelle der GitLab-REST-API setzt das Backend zu jedem Befund eine Inline-Diskussion an die betroffene Codezeile (FA-14). Eine solche Diskussion braucht neben dem Text eine vollständige Positionsangabe aus Datei, Ausgangsstand und Zielstand des Diffs. Lässt sich die Belegstelle keiner geänderten Zeile zuordnen, fällt das System auf einen normalen Kommentar am Merge Request zurück.
Wichtig ist, welche Antworten überhaupt einen Kommentar erzeugen. Der Agent gibt je Regel eines von drei Urteilen ab. Die Regel ist eingehalten, sie ist verletzt, oder sie lässt sich mit den vorhandenen Belegen nicht entscheiden (FA-8). Nur der zweite Fall erzeugt einen Befund am Merge Request. Diese Trennung ist der folgenreichste Einzelbefund der Messungen. Der Judge-Prompt des Referenzverfahrens weist an, bei fehlenden Belegen auf Verstoß zu erkennen. Damit fallen „der Code widerspricht der Regel“ und „es wurde nichts gefunden“ unter dasselbe Label, was allein 46 % des gemessenen Unterschieds zwischen beiden Verfahren erklärt (Abschnitt 6.4). Im Entwicklungsalltag wiegt das schwerer als in einer Messung. Ein Befund, der nur aus einer Lücke in den Belegen entsteht, ist genau der Fehlalarm, an dem statische Analysewerkzeuge ihre Akzeptanz verlieren [20] (NFA-10). Fälle ohne ausreichende Belege werden deshalb protokolliert, aber nicht kommentiert.
Der Nutzen für die Entwickelnden liegt in der Verbindung zwischen Codestelle und Dokumentation. Wer einen Befund nicht versteht, findet die Begründung nicht in einem allgemeinen Modelltext, sondern in dem ADR, aus dem die Regel stammt (NFA-11). Zusätzlich führt jeder Review auf eine eigene Detailseite des Backends. Dort stehen alle geprüften Regeln mit Begründung, Belegstelle und den zugehörigen Werkzeugaufrufen, auch die eingehaltenen und die unentscheidbaren. Der Merge Request bleibt so frei von Rauschen, und nachvollziehbar bleibt der Lauf trotzdem.

7.4. Grenzen der Regelaufbereitung

Die ADRs sind in diesem Aufbau der Wissensspeicher des Projekts. Daraus entstehen drei Probleme, die der Prototyp nicht löst.
Das erste ist der Status eines ADR. Streng genommen dürfte sich der Agent nur auf ADRs mit dem Status *accepted* stützen, denn Entwürfe und abgelöste Entscheidungen führen zu Urteilen auf Basis überholter Vorgaben (Abschnitt 2.4). Der Prototyp wertet das Statusfeld nicht aus und behandelt jedes gefundene Dokument gleich. Der Punkt ist deshalb auch nicht als erfüllt ausgewiesen (Abschnitt 4.1.4).
Das zweite ist die Zuordnung von ADR und Änderungssatz. Sie läuft über das Triage-Routing auf Dateipfaden (Abschnitt 7.2.3) und damit über ein grobes Kriterium. Eine reine Vektorsuche wäre nicht ohne Weiteres besser. Die Ähnlichkeit zwischen Regeltext und Codeausschnitt sagt nur wenig darüber aus, ob die Regel dort überhaupt gilt, und genau diese Schwäche weist Abschnitt 6.2 für die statische Baseline nach.
Das dritte sind widersprüchliche Vorgaben. Allgemeine Clean-Code-Regeln aus dem Trainingswissen des Modells können einem projektspezifischen ADR widersprechen. Der Prompt legt die Rangfolge fest und gibt dem ADR im Zweifel Vorrang. Ob das Modell sich daran hält, ist nicht gemessen worden.

7.5. Demonstration des Prototyps

Die bisherigen Abschnitte beschreiben die Bausteine einzeln. Dieser Abschnitt zeigt sie im Zusammenspiel und weist nach, dass die funktionalen Anforderungen an die Integration erfüllt sind. Genauigkeitswerte stehen hier nicht. Der Prototyp läuft auf einem einzigen Projekt und ohne Vergleichsfall, also lässt sich an ihm nichts über die Güte der Urteile sagen. Das hat der kontrollierte Vergleich bereits geklärt. Hier geht es nur darum, dass sich das gemessene Verfahren in den Entwicklungsalltag einbetten lässt.

7.5.1. Ablauf eines vollständigen Merge-Request-Reviews

TODO: Ablauf eines vollständigen Merge-Request-Reviews anhand eines konkreten Beispiels zeigen. Ausgelöster Webhook, gewählte Regelbasis (Projekt-ADRs oder Gruppenstandard), extrahierte Regeln, vom Triage-Routing verworfene Regeln, Werkzeugaufrufe des Agenten, erzeugte Kommentare am Merge Request. Screenshots der GitLab-Oberfläche und der Detailseite einfügen. Nachweis für FA-3, FA-4, FA-12, FA-13, FA-14 und NFA-9 (Abschnitt 4.1).

7.5.2. Qualität der erzeugten Review-Kommentare

TODO: Qualität der erzeugten Review-Kommentare berichten. Falls keine systematische Erhebung stattgefunden hat, das hier offen ausweisen und den Abschnitt streichen. NFA-10 und NFA-11 sind dann auf „offen“ zu setzen.

8. Diskussion

Kapitel 6 berichtet die Messwerte und beantwortet damit die drei Forschungsfragen. Kapitel 7 baut daraus ein lauffähiges System. Dieses Kapitel ordnet beides ein. Es klärt, was die Zahlen tragen, was aus ihnen für den Prototyp folgt, wo sie an Grenzen stoßen und was sich aus dem Vorgehen für vergleichbare Untersuchungen lernen lässt. Die Werte selbst werden nicht noch einmal vollständig aufgeführt, sondern nur dort genannt, wo ein Argument sie braucht.

8.1. Einordnung der Ergebnisse

8.1.1. Der Referenzwert der Originalarbeit misst nicht die Trefferquote

Die Originalarbeit berichtet 91,15 % Accuracy. Anhang A.2.3 zeigt, wie dieser Wert zustande kommt. Das bewertete Modell erzeugt seine eigene Referenz mit. Drei Prüfmodelle bekommen sein Urteil samt Begründung vorgelegt und liefern bei Zustimmung dieselbe Antwort zurück, und der validierende Mensch wählt am Ende aus genau diesen vier Antworten aus. In 301 von 305 Fällen stimmt das finale Label deshalb mit mindestens einer Modellantwort überein.
Gemessen wird damit, wie gut ein Verfahren eine bestimmte Kalibrierung reproduziert. Das ist nicht dasselbe wie seine Trefferquote, und der Unterschied entscheidet über die Rangfolge. Gegen unabhängige Labels fällt derselbe Judge auf 71,7 %, und der kuratierte Labelsatz schneidet mit 66,7 % sogar schlechter ab als die Modellausgabe, die er korrigieren sollte. Ein Verfahren, dessen Antworten in der Auswahlstufe nie zur Verfügung standen, kann gegen einen solchen Maßstab kaum gewinnen, unabhängig davon, wie gut es urteilt. Für die Bewertung des agentischen Ansatzes folgt daraus zweierlei. Die naheliegende Lesart der Ausgangslage ist nicht haltbar, wonach das statische Verfahren mit 91 % dem eigenen Lauf mit rund der Hälfte davon überlegen wäre. Die Gegenrichtung ist aber ebenso wenig haltbar. Die 91,15 % sind kein Beleg gegen den agentischen Ansatz und wären auch dann keiner, wenn sie sich hätten reproduzieren lassen. Sie sind schlicht keine Aussage über die Urteilsgüte. Der Vergleich musste deshalb auf einer neuen Grundlage geführt werden, und alle Aussagen dieser Arbeit beruhen allein auf dieser.

8.1.2. Ein großer Teil des Vorsprungs war ein Prompt-Effekt

Gegen die unabhängige Referenz liegt der agentische Judge 19,6 Prozentpunkte vor dem statischen. Der Abstand hält der gepaarten Prüfung stand und übersteigt die Wiederholungsstreuung um ein Vielfaches. Ohne das Zweifaktorenfeld in Abschnitt 6.4 wäre daraus ein Architekturvorteil von 19,6 Punkten berichtet worden. Tatsächlich gehen 10,7 Punkte auf eine einzige Anweisung im Judge-Prompt der Originalarbeit zurück. Sie verlangt bei fehlender Evidenz ein *Not-Compliant*-Urteil und wirft damit zwei verschiedene

Sachverhalte zusammen, nämlich „der Code widerspricht der Regel“ und „es wurde nichts gefunden“.
Bemerkenswerter noch ist, dass sich beide Effekte nicht sauber trennen lassen. Der agentische Judge reagiert mit 15,2 Prozentpunkten sogar stärker auf dieselbe Umformulierung als der statische mit 10,7, und der Architekturvorteil hängt seinerseits davon ab, welcher Prompt gesetzt ist (4,4 gegenüber 8,9 Punkte). Eine Aufteilung des Vorsprungs in feste Prozentanteile ist damit gar nicht definierbar. Wer sie dennoch berichtet, verdeckt eine Wechselwirkung von 4,5 Punkten.
Dieser Befund reicht über die Arbeit hinaus. Wer eine neue Architektur gegen eine Referenzimplementierung stellt, vergleicht in aller Regel nicht nur Architekturen, sondern auch Prompts. Ohne eine Variante, die den Prompt-Anteil isoliert, lässt sich ein gemessener Unterschied nicht eindeutig zuordnen. Genau diese Trennung verlangt Forschungsfrage 2, und die Literatur hat sie bisher offengelassen (Abschnitt 3.2.4). Aðalsteinsson et al. berichten einen Vorsprung retrieval-gestützter Reviews, konnten ihn aber nicht von der veränderten Aufgabenformulierung trennen [2].
Die Prompt-Korrektur beseitigt die Schwäche des statischen Verfahrens außerdem nicht, sie kehrt sie um. Der permissive Prompt hebt den *CIA*-Recall von 36,8 auf 94,7 %, stuft dafür aber 17 von 36 konformen Regeln fälschlich als unentscheidbar ein. Beide statischen Varianten sind also jeweils in eine Richtung verzerrt. Der agentische Judge ist als einziges der drei Verfahren im Recall über alle drei Klassen ausgewogen. Der eigentliche Unterschied liegt damit weniger in der Gesamtgenauigkeit als in der Kalibrierung, und die ließ sich beim statischen Judge durch keine der geprüften Prompt-Varianten herstellen.

8.1.3. Der verbleibende Vorteil beruht darauf, Abwesenheit zu erkennen

Die Werkzeugprotokolle widerlegen zuerst die naheliegende Erklärung. Der Agent ist nicht deshalb besser, weil er mehr sucht. Falsche Urteile gingen mit mehr Werkzeugaufrufen einher als richtige (Abschnitt 6.3.2). Ein hoher Suchaufwand ist damit ein Warnsignal und kein Gütezeichen. Steht die Antwort klar im Code, findet der Agent sie schnell.
Die tragfähige Erklärung steht ebenfalls in den Protokollen. Vergibt der Agent *CIA*, laufen im Mittel 4,29 Suchen ins Leere und er öffnet praktisch keine Datei mehr. Bei einem *C*-Urteil laufen nur 1,26 Suchen leer (Abschnitt 6.3.1). Er weicht also nicht aus, weil er wenig gesucht hätte, sondern weil er gesucht und nichts gefunden hat. In der aussagekräftigen Streitrichtung behält er in 6 von 8 Fällen recht (Abschnitt 6.3.3). Die Budget-Ablation stellt denselben Zusammenhang gezielt her, denn mit wachsendem Budget fällt die Zahl der Enthaltungen von 42 auf 11 (Abschnitt 6.6).
Damit ist der Gewinn des Live-Zugriffs benannt. Der Agent kann feststellen, dass etwas nicht da ist. Eine Ähnlichkeitssuche kann das nicht. Sie liefert immer fünf Ausschnitte, ob relevant oder nicht, und der Judge kann nicht unterscheiden, ob der maßgebliche Beleg fehlt oder nur nicht abgerufen wurde. Eine Enthaltung der statischen Pipeline sagt deshalb vor allem etwas über deren Kontextfenster aus, eine Enthaltung des Agenten dagegen über den geprüften Code.
Die Formulierung, der Agent verstehe mehr, decken diese Daten nicht. Belegt ist nur, dass er mehr findet und dass er zuverlässiger bemerkt, wenn es nichts zu finden gibt. Die Budget-Ablation zeigt zugleich die Grenze des Arguments. Bei minimalem Budget schrumpft der Vorsprung auf 2,4 Punkte. Werkzeugzugriff als Prinzip genügt also nicht, er muss groß genug bemessen sein.

8.1.4. Der Evidenzzugang wiegt schwerer als die Modellgröße

Bei der Modellrotation ist nicht überraschend, dass sich der Vorteil mit `gpt-5.4-nano` reproduziert. Überraschend ist, dass er dort größer ausfällt (Abschnitt 6.7). Die Konfusionsmatrizen erklären das. Der statische `nano`-Lauf folgt der problematischen Prompt-Anweisung noch mechanischer als das größere Modell und kollabiert fast vollständig auf eine einzige Klasse. Mit Werkzeugzugang verschwindet dieses Verhalten. Ein schwaches Modell mit gutem Evidenzzugang muss weniger raten, und Raten ist genau das, worin schwache Modelle schlecht sind.
Praktisch folgt daraus eine Reihenfolge, die der verbreiteten Erwartung widerspricht. Der agentische `nano`-Lauf übertrifft den statischen `mini`-Lauf um 12,4 Punkte, obwohl er das schwächere Modell verwendet. Wer die Urteilsgüte eines solchen Systems verbessern will, sollte deshalb zuerst am Evidenzzugang ansetzen und erst danach am Modell. Das trifft sich mit der Lage in vielen Organisationen, in denen die Modellwahl ohnehin durch Verfügbarkeit und Vertraulichkeit vorgegeben ist (Abschnitt 5.2.2). Einzuschränken bleibt, dass beide Modelle vom selben Anbieter und aus derselben Generation stammen. Der Befund zeigt, dass der Effekt nicht an einem einzelnen Modell hängt, und nicht, dass er über Anbieter hinweg gilt.

8.1.5. Ein Teil der gemessenen Grenze liegt am Benchmark

Ein Befund betrifft weder das eine noch das andere Verfahren, sondern die Datenbasis. Von den 30 Fällen, die in der unabhängigen Annotation als unentscheidbar eingestuft sind, haben 18 sehr wohl einen Beleg im Repository. Er liegt nur außerhalb der Datei-Endungs-Whitelist des Original-Indexers, also in CI-Konfiguration, Build-Dateien, Linter-Regeln oder Dokumentation (Anhang A.3.2).
60 % der *CIA*-Fälle sind damit Artefakte des Benchmark-Designs und keine Grenze der Modellfähigkeit. Der absolute Genauigkeitsbereich beider Verfahren ist nach oben gedeckelt durch eine Datenauswahl, die einen erheblichen Teil der Regeln gar nicht prüfbar macht. Inhaltlich ist das kein Zufall. Ein großer Teil realer Architekturentscheidungen betrifft Werkzeugketten, Build- und Auslieferungsprozesse, also genau die Dateien, die kein Programmiersprachen-Filter erfasst. Für ein Prüfsystem im Einsatz folgt daraus unmittelbar, dass der Suchraum nicht auf Quellcode beschränkt werden darf. Wie viel des *CIA*-Anteils bei vollem Repository-Zugriff verschwindet, ist offen und im Ausblick beschrieben (Kapitel 9).

8.2. Folgen für den Prototyp

Der Prototyp aus Kapitel 7 ist der Ort, an dem diese Befunde wirksam werden. Tabelle 7.1 führt die Zuordnung im Einzelnen auf. Drei Punkte verdienen hier eine Einordnung.
Der erste betrifft die Fehlalarme. Die Trennung von „widerspricht der Regel“ und „nichts gefunden“ ist im Messaufbau eine Frage der Kalibrierung. Im Entwicklungsalltag entscheidet sie über die Akzeptanz, denn ein Befund aus einer Evidenzlücke ist genau der Fehlalarm, an dem statische Analysewerkzeuge scheitern [20]. Der Prototyp meldet deshalb nur Verstöße und protokolliert Enthaltungen still (Abschnitt 7.3). Dieselbe Prompt-Anweisung, die im Messaufbau 10,7 Prozentpunkte erzeugt hat, hätte im Betrieb vor allem Rauschen erzeugt. Wie weit das reicht, zeigt der agentische Lauf mit restriktivem

Prompt: Dort fällt die *NC*-Precision auf 6,7 %, das heißt vierzehn von fünfzehn gemeldeten Verstößen wären Fehlalarme gewesen.
Der zweite betrifft den Preis. Eine Regel kostet im Standardbudget 74 Sekunden und rund das 51-Fache an Tokens gegenüber der statischen Baseline. Für einen Review am Merge Request ist das vertretbar, weil er ereignisgesteuert und außerhalb der Pipeline läuft. Für eine Prüfung bei jedem Commit wäre es das nicht. Die Vorauswahl der Regeln (Abschnitt 7.2.3) ist deshalb keine Feinheit, sondern die Bedingung dafür, dass das Verfahren mit einer wachsenden Regelbasis überhaupt umgehen kann.
Der dritte betrifft die verbliebene Schwäche. Die Precision der Klasse *NC* liegt auch beim agentischen Judge bei 37,5 %, weniger als die Hälfte der gemeldeten Verstöße ist also einer. Der Wert beruht auf nur fünf wahren *NC*-Fällen und ist statistisch nicht belastbar (Abschnitt 8.3). Er zeigt aber in dieselbe Richtung wie die empirischen Arbeiten zur Akzeptanz maschineller Reviews, nach denen Entwickelnde vor allem an der Fehlalarmquote scheitern [7]. Für den produktiven Einsatz ist das die Größe, an der zuerst gearbeitet werden müsste.

8.2.1. Erweiterbarkeit

Regelaufbereitung, Evidenzbeschaffung, Urteilsbildung und Ausgabe sind im Prototyp getrennt gehalten (NFA-12). Das hat zwei praktische Folgen. Zum einen lässt sich der Prüfkern gegen ein anderes Modell tauschen, ohne die Anbindung an GitLab anzufassen, und die Modellrotation in Abschnitt 6.7 hat davon bereits Gebrauch gemacht. Zum anderen ist die Regelbasis nicht an ADRs gebunden. Alles, was sich in einzeln entscheidbare, imperative Sätze zerlegen lässt, kann an dieselbe Stelle treten, etwa Sicherheitsvorgaben oder Kodierrichtlinien. Geprüft ist das nicht. Die Zerlegung und die Vorauswahl arbeiten aber bereits auf reinem Text und setzen keine ADR-spezifische Struktur voraus.

8.3. Einschränkungen

Neun Punkte begrenzen die Reichweite der berichteten Zahlen. Sie stehen hier und nicht im Ergebniskapitel, weil sie nicht die Messung selbst betreffen, sondern deren Auslegung.

- **Die Referenz beruht auf einer einzigen Annotation.** Sie wurde vom Verfasser erstellt und nicht von einer zweiten Person gegengeprüft. Ein Maß für die Übereinstimmung zweier Annotierender liegt deshalb nicht vor (Abschnitt 5.3.2). Vor allem die 3 Fälle niedriger und die 14 Fälle mittlerer Konfidenz sollten von einer zweiten Person geprüft werden, bevor die Ergebnisse als gesichert gelten. Dass der Befund nicht an diesen Fällen hängt, zeigt Anhang A.3.5. Ein Ersatz für eine zweite Annotation ist das aber nicht.

- **Die Studie ist für mittlere Effekte unterdimensioniert.** Mit $n = 60$ und daraus resultierenden sieben bis fünfundzwanzig diskordanten Fällen werden nur die großen Unterschiede signifikant, nämlich der Hauptvergleich E1 ($p = 0{,}0023$), der Prompt-Effekt beim agentischen Judge ($p = 0{,}0075$) und die Modellstufen. Differenzen in der Größenordnung von fünf bis neun Prozentpunkten lassen sich dagegen nicht absichern; das betrifft die Rollentrennung in E3 ($p = 0{,}1250$) und den bei angeglichener Kontextmenge verbleibenden Abstand in E6 ($p = 0{,}1185$). Beide Befunde werden deshalb deskriptiv und über die Werkzeugprotokolle geführt, nicht

über die Genauigkeit. Nicht jede gemessene Differenz dieser Arbeit ist ein abgesicherter Befund, und das ist eine Eigenschaft des Aufbaus und keine Auffälligkeit einzelner Läufe.

- **Prompt- und Architektureffekt sind nicht additiv trennbar.** Das Zweifaktorenfeld in Abschnitt 6.4 weist eine Wechselwirkung von 4,5 Prozentpunkten aus. Angaben wie „x % des Vorsprungs gehen auf den Prompt zurück“ sind deshalb nicht definierbar, und die Arbeit verzichtet bewusst darauf. Wie groß der Architekturvorteil ausfällt, lässt sich nur mit Angabe der Prompt-Bedingung sagen.

- **Die Klasse *NC* ist unterbesetzt.** Im Zufallsteil entfallen nur fünf Fälle auf sie. Alle klassenspezifischen Werte für *NC* sind deshalb statistisch nicht belastbar und werden nur der Vollständigkeit halber berichtet. Das betrifft auch die unter FF1 genannte *NC*-Precision.

- **Richtung 1 der Streitfallanalyse beruht auf 7 Fällen** und erlaubt keine Aussage. Nur Richtung 2 mit 8 Fällen und einem Verhältnis von 6 zu 2 wird interpretiert, und auch dort ist die Basis schmal.

- **Die vollen 81 Fälle sind gegen den Original-Judge verzerrt**, da sie alle 27 überstimmten Fälle enthalten. Sämtliche berichteten Werte beziehen sich deshalb ausschließlich auf den Zufallsteil. Als Härtetest in die ungünstige Richtung sind sie in Anhang A.3.5 dennoch mitgeführt.

- **Die Aufschlüsselung der *CIA*-Ursachen ist von Hand erstellt.** Die Aufteilung in Tabelle A.2 beruht auf einer manuellen Durchsicht der 30 Begründungen und lässt sich, anders als alle übrigen Tabellen des Ergebniskapitels, nicht aus dem Replication-Package nachrechnen.

- **Nur drei Modelle desselben Anbieters geprüft.** Der Befund reproduziert sich auf allen drei Stufen (Abschnitt 6.7), diese stammen jedoch sämtlich vom selben Anbieter. Ob der Effekt auch bei Modellen anderer Anbieter auftritt, bleibt offen; die Gateway-Restriktionen (Abschnitt 5.2.2) ließen keine breitere Rotation zu. Für `gpt-5.6-luna` liegt zudem nur ein Lauf je Bedingung vor, weshalb dessen Abstand nicht der Größe nach mit den beiden anderen Stufen verglichen wird. Bei `gpt-5.4-nano` widersprechen sich Mehrheits- und Vorzeichentest ($p = 0{,}1078$ gegenüber $p = 0{,}0051$), was auf besonders instabile Einzelurteile hindeutet.

- **Die Budget-Ablation variiert nur eine Dimension gemeinsam.** E4 verändert Suchrunden, Datei-Öffnungen und Nachsuchen stets im Verbund (Tabelle 6.20). Welche der drei Grenzen den gemessenen Effekt trägt, lässt sich daraus nicht ableiten.

Zwei Punkte, die frühere Auswertungsstände belasteten, zählen dagegen nicht mehr dazu. Die Streuung durch nichtdeterministische Modellausgaben ist mit dem Kontrolllauf quantifiziert (Abschnitt 6.2.4). Und die Vergleichsläufe wurden eigens auf die annotierten Fälle beschränkt durchgeführt, statt nachträglich aus größeren Läufen gefiltert zu werden (Abschnitt 5.4).
Für den Prototyp kommt eine eigene Einschränkung hinzu. Er ist auf einem einzelnen Projekt und ohne Vergleichsfall gelaufen, und eine Erhebung zur Qualität seiner Kommentare hat nicht stattgefunden. Die Aussagen über die Urteilsgüte stammen deshalb durchgehend aus dem kontrollierten Vergleich und nicht aus dem Betrieb.

8.4. Methodische Lehren für vergleichende Evaluationen

Fünf Beobachtungen aus dem Verlauf der Untersuchung sind über diese Arbeit hinaus übertragbar.

Eine berichtete Accuracy sagt nichts, solange die Herkunft der Labels ungeklärt ist. Der auffälligste Einzelbefund dieser Arbeit ist kein Messwert, sondern die Rekonstruktion einer Datenkette. Weder die Publikation noch die Dateibenennung ließen erkennen, dass die Referenz aus den bewerteten Modellausgaben abgeleitet ist. Sichtbar wurde das erst beim Nachvollziehen der Skripte. Wo eine Referenz mit Unterstützung von Sprachmodellen entstanden ist, gehört die Frage nach oben, ob die geprüften Antworten zur Auswahl standen.

Ohne Wiederholungsläufe ist kein Unterschied interpretierbar. Die Streuungsmessung ergibt für dieselbe Konfiguration 1,7 bis 1,9 Prozentpunkte Standardabweichung und Fall für Fall nur 83 bis 86 % identische Urteile. Ein großer Teil der im Verlauf dieser Arbeit als Fortschritt oder Rückschritt gedeuteten Versionsunterschiede liegt unterhalb dieser Schwelle. Der Punkt wurde erst nachträglich erkannt, nachdem eine frühere Auswertung zwei Läufe verglichen hatte, die sich in mehr als einer Variable unterschieden.

Kleine Stichproben kehren Bewertungen um. Mehrfach änderte sich die Beurteilung einer Architekturänderung, sobald die Stichprobe größer wurde, in einem Fall um mehr als zehn Prozentpunkte. Hätte die Untersuchung an einem Zwischenstand mit $n = 20$ geendet, wäre eine sachlich falsche Entscheidung als Erfolg berichtet worden.

Undokumentierte Konfigurationsdetails verschieben das Ergebnis erheblich. Die im Original verwendete Datei-Endungs-Whitelist verschob die gemessene Genauigkeit der statischen Pipeline je nach Modell um neun bis 18 Prozentpunkte (Abschnitt 5.1.3). Dass sie Markdown ausschließt und ADR-Dateien deshalb nie Teil des Retrieval-Index waren, war der Publikation nicht zu entnehmen und musste aus den Skripten rekonstruiert werden. Reproduktionsstudien sind gegenüber solchen Details empfindlich, gerade weil sie in Veröffentlichungen selten stehen.

Auch die Wahl des Nenners verändert die berichteten Zahlen. Für dieselben Rohdaten ergeben sich unterschiedliche Werte, je nachdem, wie nicht auswertbare Fälle behandelt werden. Sie können aus Zähler und Nenner ausgeschlossen werden, im Nenner verbleiben und damit als Fehlurteil zählen, oder nur dann als Fehlurteil zählen, wenn die Zeile überhaupt in der Ergebnisdatei steht. Bei Modellen mit vielen Ausfällen wird der Unterschied erheblich. Da die Konvention in Publikationen selten ausgewiesen wird, ist sie eine leicht übersehene Quelle scheinbarer Abweichungen zwischen Reproduktion und Original. Sie wird deshalb in Abschnitt 5.5 explizit festgelegt.

TODO: die besucht am in dem literaturverzeichnis auf start der BA setzen
TODO: bei oliver kopp in literaturverzeichnis fehlt quelle

9. Fazit und Ausblick

9.1. Fazit

Diese Arbeit ist der Frage nachgegangen, ob ein Sprachmodell Architekturverstöße besser erkennt, wenn es seine Belege selbst im Repository sucht, statt sie vorab zusammengestellt zu bekommen. Der erste Befund betrifft allerdings nicht die Verfahren, sondern den Maßstab. Der in der Literatur berichtete Referenzwert von 91,15 % misst nicht die Trefferquote, sondern wie gut ein Verfahren eine Kalibrierung reproduziert, an deren Zustandekommen es selbst beteiligt war. Der Vergleich musste deshalb auf einer eigens erhobenen, unabhängigen Referenz geführt werden.
Gegen diese Referenz urteilt der agentische Judge deutlich besser als der statische, nämlich mit 75,6 gegenüber 55,9 % im Mittel dreier Läufe. Der Abstand von 19,6 Prozentpunkten lässt sich vom Zufall trennen ($p = 0{,}0023$), übersteigt die Streuung von Wiederholungsläufen um ein Vielfaches und tritt auf allen drei geprüften Modellen auf. Er ist damit belastbar, aber er ist nicht der Architektur allein zuzurechnen. Zwei weitere Bedingungen tragen ihn mit. Die erste ist die Urteilsanweisung: Sie bringt dem statischen Judge 10,7 Prozentpunkte und dem agentischen sogar 15,2, und beide Effekte greifen so ineinander, dass sich der Vorsprung gar nicht in feste Anteile zerlegen lässt (Abschnitt 6.4). Die zweite ist die schiere Menge an Evidenz: Gibt man der Baseline so viel Kontext, wie der Agent selbst zusammenträgt, schließen sich 10,8 der 19,6 Punkte allein dadurch (Abschnitt 6.8). Was danach bleibt, sind 8,9 Punkte, die bei dieser Fallzahl nicht mehr statistisch abzusichern sind.
Der verbleibende Anteil lässt sich benennen, wenn auch nicht mehr statistisch sichern. Er beruht nicht darauf, dass der Agent mehr Material sichtet, denn falsche Urteile gingen mit mehr Werkzeugaufrufen einher als richtige. Er beruht darauf, dass der Agent feststellen kann, wenn ein Beleg nicht existiert. Eine Ähnlichkeitssuche liefert immer eine Auswahl und kann zwischen einer Lücke im Code und einer Lücke im Abruf nicht unterscheiden. Belegt ist damit nicht, dass der Agent mehr versteht, sondern dass er mehr findet und fehlende Belege als solche erkennt.
Das eigentliche Ergebnis dieser Arbeit ist deshalb ein anderes, als die Ausgangsthese erwarten ließ. Der Werkzeugzugang ist notwendig, aber für sich genommen wirkungslos. Nimmt man dem Agenten die Urteilsanweisung, fällt er trotz vollem Repository-Zugriff fast auf Baseline-Niveau. Nimmt man ihm die Trennung von Locator und Reader, öffnet er in der Mehrzahl der Fälle keine einzige Datei. Nimmt man ihm das Suchbudget, verschwindet der Vorsprung ganz. Der Gewinn entsteht erst aus dem Zusammenspiel dieser drei Bedingungen. Das ist eine schwächere Aussage als „agentisch ist besser“, aber eine, die sich anwenden lässt, weil sie sagt, worauf beim Bauen zu achten ist.
Der Prototyp aus Kapitel 7 überführt diese Befunde in ein lauffähiges System. Es setzt am Merge Request an, zerlegt die ADRs des Projekts in einzeln prüfbare Regeln und schreibt jeden Befund als Kommentar an die betroffene Codezeile zurück. Seine wesentlichen Festlegungen sind aus den Messungen abgeleitet und nicht gesetzt, allen voran die strikte Trennung zwischen einem Verstoß und einer Evidenzlücke sowie das Werkzeugbudget am

gemessenen Sättigungspunkt. Nachgewiesen ist damit die Machbarkeit dieser Anbindung. Eine Aussage über die Nützlichkeit der erzeugten Kommentare im Arbeitsalltag ist das nicht, denn eine Erhebung dazu hat nicht stattgefunden.
Die Reichweite der Ergebnisse ist begrenzt. Sie beruhen auf 60 Fällen und auf einer Referenz, die von einer einzigen Person erstellt wurde. Die Klasse *Not Compliant* ist mit fünf Fällen zu schwach besetzt, um klassenspezifische Aussagen zu tragen, und gerade dort liegt mit einer Precision von 37,5 % die praktisch wichtigste Schwäche des Verfahrens. Innerhalb dieser Grenzen tragen zwei Aussagen. Der Zugang zur Evidenz wiegt schwerer als die Größe des Modells, denn der agentische Lauf mit dem schwächeren Modell übertrifft den statischen Lauf mit dem stärkeren um 12,4 Punkte. Und wer zwei Architekturen vergleicht, vergleicht ohne eine Kontrollvariante immer auch zwei Prompts.

9.2. Ausblick

Die Arbeit lässt an vier Stellen offene Enden. Der erste Block betrifft den Maßstab, an dem gemessen wurde. Der zweite betrifft Messungen, die vorbereitet, aber nicht abgeschlossen sind. Der dritte betrifft das Prüfverfahren selbst. Der vierte betrifft den Prototyp, der bisher nur demonstriert und nicht evaluiert ist.

9.2.1. Den Maßstab absichern

Zweitannotation der Referenz. Der dringlichste offene Punkt betrifft nicht die Verfahren, sondern den Maßstab. Die unabhängige Referenz beruht auf einer einzigen Annotation und ist damit kein Goldstandard (Abschnitt 5.3.2). Jede Zeile trägt eine Belegangabe mit Datei und Zeilennummer, deshalb ist eine Gegenprüfung durch eine zweite Person mit überschaubarem Aufwand möglich. Sie wäre zuerst auf die 3 Fälle niedriger und die 14 Fälle mittlerer Konfidenz zu richten. Erst danach lassen sich die berichteten Werte als gesichert behandeln.

Modellrotation über eine Anbieterfamilie hinaus. Die Rotation über `gpt-5.4-nano` zeigt, dass der Befund nicht an einem einzelnen Modell hängt und dass das schwächere Modell sogar stärker profitiert (Abschnitt 6.7). Beide Modelle stammen aber vom selben Anbieter und aus derselben Generation, denn die Gateway-Restriktionen ließen keine breitere Auswahl zu (Abschnitt 5.2.2). Offen ist vor allem die Gegenrichtung. Marco-o1 war nicht verfügbar (Anhang A.2.1), also ist ungeprüft, ob der Vorsprung auch bei einem starken, dediziert auf Reasoning trainierten Modell bestehen bleibt. Er könnte dort schrumpfen, weil ein solches Modell die Schwächen der Retrieval-Stufe ausgleicht. Der beobachtete Trend spricht dafür, denn je schwächer das Modell, desto größer war der Nutzen des Evidenzzugangs.

9.2.2. Die vorbereiteten Messungen abschließen

Erweiterung des Suchraums über Quellcode hinaus. 60 % der als unentscheidbar annotierten Fälle haben einen Beleg im Repository, der lediglich außerhalb der Datei-Endungs-Whitelist liegt (Anhang A.3.2). Der Prototyp hat diese Beschränkung bereits aufgehoben (Abschnitt 7.2.4), gemessen ist ihre Wirkung aber nicht. Die dafür nötige Scope-Ablation ist implementiert. Es fehlt eine zweite Annotation der 18 betroffenen Fälle unter erweitertem Suchraum. Von allen offenen Messungen ist das die mit dem

größten zu erwartenden Effekt. **TODO: Ergebnis der Scope-Ablation eintragen. Wie viel des *CIA*-Anteils verschwindet, wenn der Agent auf das vollständige Repository statt auf die Endungs-Whitelist zugreifen darf. Der Code ist fertig, es fehlt eine zweite Ground-Truth-Annotation für die 18 Fälle aus Tabelle A.2, deren Evidenz außerhalb der Whitelist liegt.**

Auflösung des Werkzeugbudgets in seine Einzelgrenzen. Die Budget-Ablation variiert Suchrunden, Datei-Öffnungen und Nachsuchen stets gemeinsam (Abschnitt 6.6). Sie weist damit einen klaren Sättigungspunkt nach, lässt aber offen, welche der drei Grenzen ihn verursacht. Eine Ablation, die jede Grenze einzeln variiert, würde eine deutlich genauere Budgetempfehlung erlauben. Sie ließe sich mit dem vorhandenen Aufbau ohne Codeänderung durchführen, da alle drei Grenzen bereits über getrennte Umgebungsvariablen gesteuert werden.

Wiederholungsläufe für die Prompt-Zerlegung. Die Aufteilung des Vorsprungs in einen Prompt- und einen Evidenzanteil beruht auf einem einzigen Lauf mit korrigiertem Prompt (Abschnitt 6.4). Innerhalb der gemessenen Streuung statischer Läufe verschiebt sich der Prompt-Anteil zwischen rund 40 und 52 %. Die Kernaussage hält über diese ganze Spanne, die genaue Aufteilung wird aber erst mit drei Wiederholungen belastbar. Der Aufwand dafür ist gering, weil die Konfiguration bereits vorliegt.

9.2.3. Das Prüfverfahren verbessern

Gezielte Arbeit an der Klasse *Not Compliant*. Diese Klasse bleibt die schwächste der drei. Der agentische Judge erreicht dort 60,0 % Recall bei einer Precision von nur 37,5 % (Tabelle 6.15). Da das Auffinden von Verstößen der eigentliche Zweck des Verfahrens ist, wäre eine eigene Iterationsreihe zu dieser Fehlerklasse lohnend, etwa durch eine Gegenprüfung, ob der gefundene Code die Regel tatsächlich umsetzt oder sie nur thematisch berührt. Belastbar bewerten lässt sich eine solche Änderung allerdings erst auf einer Stichprobe mit deutlich mehr als den hier vorliegenden fünf *NC*-Fällen.

Abbruchkriterium statt festem Werkzeugbudget. Falsche Urteile gingen mit mehr Werkzeugnutzung einher als richtige (Abschnitt 6.3.2), und jenseits des Sättigungspunkts beginnt der Agent, schwache Hinweise überzubewerten (Abschnitt 6.6). Der Suchaufwand taugt damit nicht nur als Deckel, sondern als Signal. Ein Fall, in dem der Agent ungewöhnlich lange sucht, ist ein Kandidat für eine Enthaltung oder für die Vorlage an einen Menschen und nicht für mehr Budget. Ein solches adaptives Kriterium wäre dem festen Budget vermutlich überlegen, weil es den Aufwand dort investiert, wo er etwas bringt, und ihn dort abbricht, wo er in die Irre führt.

9.2.4. Den Prototyp evaluieren und vervollständigen

Messung am Änderungssatz statt am Gesamtbestand. Beide Evaluationen prüfen ganze Repositories gegen einen festen Regelsatz. Der Prototyp stellt die Aufgabe dagegen inkrementell, denn er beurteilt einen Änderungssatz vor dem Hintergrund des übrigen Repositorys (Abschnitt 7.2.4). Ob sich die gemessenen Genauigkeiten auf diesen deutlich engeren Fall übertragen, ist offen und wäre gesondert zu messen.

Erhebung zur Qualität der Review-Kommentare. Kapitel 7 weist nach, dass sich das Verfahren in den Merge-Request-Workflow einbetten lässt. Über die Nützlichkeit der erzeugten Kommentare sagt das nichts, und NFA-10 sowie NFA-11 bleiben damit unbelegt. Eine solche Erhebung müsste über mehrere Projekte und über einen längeren Zeitraum laufen, weil die Fehlalarmlast erst im Arbeitsalltag sichtbar wird [20].

Auswertung des ADR-Status und bessere Zuordnung. Der Prototyp behandelt jedes gefundene ADR gleich und wertet das Statusfeld nicht aus (Abschnitt 7.4). Entwürfe und abgelöste Entscheidungen können deshalb zu Urteilen auf Basis überholter Vorgaben führen. Die Auswertung des Feldes selbst ist wenig Aufwand. Schwerer wiegt die Zuordnung von ADR und Änderungssatz, die derzeit allein auf Dateipfaden beruht (Abschnitt 7.2.3). Wie gut diese Zuordnung trifft, ließe sich mit dem vorhandenen Aufbau messen, indem die Vorauswahl abgeschaltet und gegen den vollen Regelsatz geprüft wird.

10. Literaturverzeichnis

[1] *About GitHub Copilot Code Review*. GitHub Docs. URL: https://docs.github.com/en/copilot/concepts/agents/code-review (besucht am 28. 07. 2026).

[2] Fannar Steinn Aðalsteinsson u. a. *Rethinking Code Review Workflows with LLM Assistance: An Empirical Study*. 22. Mai 2025. DOI: 10.48550/arXiv.2505.16339. arXiv: 2505.16339 [cs.SE]. URL: http://arxiv.org/abs/2505.16339 (besucht am 28. 07. 2026). Vorveröffentlichung.

[3] Bardha Ahmeti u. a. „Architecture Decision Records in Practice: An Action Research Study“. In: *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, September 3–6, 2024, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 1. Sep. 2024, S. 333–349. ISBN: 978-3-031-70796-4. DOI: 10.1007/978-3-031-70797-1_22. URL: https://doi.org/10.1007/978-3-031-70797-1_22 (besucht am 07. 07. 2026).

[4] Alberto Bacchelli und Christian Bird. „Expectations, Outcomes, and Challenges of Modern Code Review“. In: *2013 35th International Conference on Software Engineering (ICSE)*. 2013 35th International Conference on Software Engineering (ICSE). Mai 2013, S. 712–721. DOI: 10.1109/ICSE.2013.6606617. URL: https://ieeexplore.ieee.org/document/6606617 (besucht am 28. 07. 2026).

[5] Héctor Cadavid, Vasilios Andrikopoulos und Paris Avgeriou. *Documentation-as-Code for Interface Control Document Management in Systems of Systems: A Technical Action Research Study*. 23. Juni 2022. DOI: 10.48550/arXiv.2206.11668. arXiv: 2206.11668 [cs.SE]. URL: http://arxiv.org/abs/2206.11668 (besucht am 07. 07. 2026). Vorveröffentlichung.

[6] Davide Chicco und Giuseppe Jurman. „The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation“. In: *BMC Genomics* 21.1 (2. Jan. 2020), S. 6. ISSN: 1471-2164. DOI: 10.1186/s12864-019-6413-7. URL: https://doi.org/10.1186/s12864-019-6413-7 (besucht am 30. 07. 2026).

[7] Kowshik Chowdhury u. a. *From Industry Claims to Empirical Reality: An Empirical Study of Code Review Agents in Pull Requests*. 3. Apr. 2026. DOI: 10.48550/arXiv.2604.03196. arXiv: 2604.03196 [cs.SE]. URL: http://arxiv.org/abs/2604.03196 (besucht am 06. 07. 2026). Vorveröffentlichung.

[8] *CI/CD Pipelines*. GitLab Docs. URL: https://docs.gitlab.com/ci/pipelines/ (besucht am 12. 06. 2026).

[9] *Code Review Flow*. GitLab Docs. URL: https://docs.gitlab.com/user/duo_agent_platform/flows/foundational_flows/code_review/ (besucht am 28. 07. 2026).

[10] *CodeRabbit Documentation - AI Code Reviews on Pull Requests, IDE, and CLI*. CodeRabbit. URL: https://docs.coderabbit.ai/guides/code-review-overview (besucht am 28. 07. 2026).

[11] Jacob Cohen. „A Coefficient of Agreement for Nominal Scales“. In: *Educational and Psychological Measurement* 20.1 (1. Apr. 1960), S. 37–46. ISSN: 0013-1644. DOI: 10.1177/001316446002000104. URL: https://doi.org/10.1177/001316446002000104 (besucht am 28. 07. 2026).

[12] Justin Dalrymple. *GitBeaker*. 28. Juli 2026. URL: https://github.com/jdalrymple/gitbeaker (besucht am 28. 07. 2026).

[13] M. E. Fagan. „Design and Code Inspections to Reduce Errors in Program Development“. In: *IBM Systems Journal* 15.3 (1. Sep. 1976), S. 182–211. ISSN: 0018-8670. DOI: 10.1147/sj.153.0182. URL: https://doi.org/10.1147/sj.153.0182 (besucht am 28. 07. 2026).

[14] *GitLab Duo Code Review (Non-Agentic)*. GitLab Docs. URL: https://docs.gitlab.com/user/gitlab_duo/code_review/ (besucht am 28. 07. 2026).

[15] *GraphQL API*. GitLab Docs. URL: https://docs.gitlab.com/api/graphql/ (besucht am 12. 06. 2026).

[16] Xinyi Hou u. a. *Large Language Models for Software Engineering: A Systematic Literature Review*. 10. Apr. 2024. DOI: 10.48550/arXiv.2308.10620. arXiv: 2308.10620 [cs.SE]. URL: http://arxiv.org/abs/2308.10620 (besucht am 28. 07. 2026). Vorveröffentlichung.

[17] Joel Jang, Seonghyeon Ye und Minjoon Seo. *Can Large Language Models Truly Understand Prompts? A Case Study with Negated Prompts*. 26. Sep. 2022. DOI: 10.48550/arXiv.2209.12711. arXiv: 2209.12711 [cs.CL]. URL: http://arxiv.org/abs/2209.12711 (besucht am 08. 07. 2026). Vorveröffentlichung.

[18] A. Jansen und J. Bosch. „Software Architecture as a Set of Architectural Design Decisions“. In: *5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05)*. 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05). Nov. 2005, S. 109–120. DOI: 10.1109/WICSA.2005.61. URL: https://ieeexplore.ieee.org/document/1620096 (besucht am 28. 07. 2026).

[19] Ziwei Ji u. a. „Survey of Hallucination in Natural Language Generation“. In: *ACM Computing Surveys* 55.12 (3. März 2023), 248:1–248:38. ISSN: 0360-0300. DOI: 10.1145/3571730. URL: https://dl.acm.org/doi/10.1145/3571730 (besucht am 30. 07. 2026).

[20] Brittany Johnson u. a. „Why Don't Software Developers Use Static Analysis Tools to Find Bugs?“ In: *Proceedings of the 2013 International Conference on Software Engineering*. ICSE '13. San Francisco, CA, USA: IEEE Press, 18. Mai 2013, S. 672–681. ISBN: 978-1-4673-3076-3. URL: https://dl.acm.org/doi/10.5555/2486788.2486877 (besucht am 28. 07. 2026).

[21] Jeff Johnson, Matthijs Douze und Herve Jegou. „Billion-Scale Similarity Search with GPUs“. In: *IEEE Transactions on Big Data* 7.3 (1. Juli 2021), S. 535–547. ISSN: 2332-7790, 2372-2096. DOI: 10.1109/TBDATA.2019.2921572. URL: https://ieeexplore.ieee.org/document/8733051/ (besucht am 28. 07. 2026).

[22] Taufiqul Islam Khan u. a. *A Survey of Code Review Benchmarks and Evaluation Practices in Pre-LLM and LLM Era*. 13. Feb. 2026. DOI: 10.48550/arXiv.2602.13377. arXiv: 2602.13377 [cs.SE]. URL: http://arxiv.org/abs/2602.13377 (besucht am 06. 07. 2026). Vorveröffentlichung.

[23] Jens Knodel und Daniel Popescu. „A Comparison of Static Architecture Compliance Checking Approaches“. In: *2007 Working IEEE/IFIP Conference on Software Architecture (WICSA'07)*. 2007 Working IEEE/IFIP Conference on Software Architecture (WICSA'07). Jan. 2007, S. 12–12. DOI: 10.1109/WICSA.2007.1. URL: https://ieeexplore.ieee.org/document/4077029 (besucht am 28. 07. 2026).

[24] Oliver Kopp, Anita Armbruster und Olaf Zimmermann. „Markdown Architectural Decision Records: Format and Tool Support“. In: *Proceedings of the 10th Central European Workshop on Services and Their Composition (ZEUS 2018)*. Bd. 2072. CEUR Workshop Proceedings. CEUR-WS.org, 3. Apr. 2018, S. 55–62. URL: https://ceur-ws.org/Vol-2072/paper8.pdf.

[25] Momcilo Krunic. „Documentation as Code in Automotive System/Software Engineering“. In: *Elektronika ir Elektrotechnika* 29 (31. Aug. 2023), S. 61–75. DOI: 10.5755/j02.eie.33843.

[26] J. Richard Landis und Gary G. Koch. „The Measurement of Observer Agreement for Categorical Data“. In: *Biometrics* 33.1 (1977), S. 159–174. ISSN: 0006-341X. DOI: 10.2307/2529310. JSTOR: 2529310. URL: https://www.jstor.org/stable/2529310 (besucht am 28. 07. 2026).

[27] Patrick Lewis u. a. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 12. Apr. 2021. DOI: 10.48550/arXiv.2005.11401. arXiv: 2005.11401 [cs.CL]. URL: http://arxiv.org/abs/2005.11401 (besucht am 30. 07. 2026). Vorveröffentlichung.

[28] Hao Li, Haoxiang Zhang und Ahmed E. Hassan. *AIDev: Studying AI Coding Agents on GitHub*. 9. Feb. 2026. DOI: 10.1145/3793302.3797249. arXiv: 2602.09185 [cs.SE]. URL: http://arxiv.org/abs/2602.09185 (besucht am 28. 07. 2026). Vorveröffentlichung.

[29] Ruiyin Li u. a. *Towards Automated Identification of Violation Symptoms of Architecture Erosion*. 4. Juli 2026. DOI: 10.48550/arXiv.2306.08616. arXiv: 2306.08616 [cs.SE]. URL: http://arxiv.org/abs/2306.08616 (besucht am 28. 07. 2026). Vorveröffentlichung.

[30] Zhiyu Li u. a. „Automating Code Review Activities by Large-Scale Pre-Training“. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 9. Nov. 2022, S. 1035–1047. ISBN: 978-1-4503-9413-0. DOI: 10.1145/3540250.3549081. URL: https://dl.acm.org/doi/10.1145/3540250.3549081 (besucht am 28. 07. 2026).

[31] Junwei Liu u. a. „Large Language Model-Based Agents for Software Engineering: A Survey“. In: *ACM Transactions on Software Engineering and Methodology* (5. März 2026). ISSN: 1049-331X. DOI: 10.1145/3796507. URL: https://dl.acm.org/doi/10.1145/3796507 (besucht am 28. 07. 2026).

[32] Nelson F. Liu u. a. „Lost in the Middle: How Language Models Use Long Contexts“. In: *Transactions of the Association for Computational Linguistics* 12 (23. Feb. 2024), S. 157–173. ISSN: 2307-387X. DOI: 10.1162/tacl_a_00638. URL: https://doi.org/10.1162/tacl_a_00638 (besucht am 28. 07. 2026).

[33] B. W. Matthews. „Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme“. In: *Biochimica et Biophysica Acta (BBA) - Protein Structure* 405.2 (20. Okt. 1975), S. 442–451. ISSN: 0005-2795. DOI: 10.1016/0005-2795(75)90109-9. URL: https://www.sciencedirect.com/science/article/pii/0005279575901099 (besucht am 28. 07. 2026).

[34] Shane McIntosh u. a. „The Impact of Code Review Coverage and Code Review Participation on Software Quality: A Case Study of the Qt, VTK, and ITK Projects“. In: *Proceedings of the 11th Working Conference on Mining Software Repositories*. MSR 2014. New York, NY, USA: Association for Computing Machinery, 31. Mai 2014, S. 192–201. ISBN: 978-1-4503-2863-0. DOI: 10.1145/2597073.2597076. URL: https://dl.acm.org/doi/10.1145/2597073.2597076 (besucht am 30. 07. 2026).

[35] Quinn McNemar. „Note on the Sampling Error of the Difference between Correlated Proportions or Percentages“. In: *Psychometrika* 12.2 (Juni 1947), S. 153–157. ISSN: 1860-0980. DOI: 10.1007/BF02295996. URL: https://doi.org/10.1007/BF02295996 (besucht am 03. 08. 2026).

[36] *Merge Requests*. GitLab Docs. URL: https://docs.gitlab.com/user/project/merge_requests/ (besucht am 12. 06. 2026).

[37] Michael Nygard. *Documenting Architecture Decisions*. Cognitect.com. 15. Nov. 2011. URL: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions (besucht am 28. 07. 2026).

[38] Stack Overflow. *2025 Stack Overflow Developer Survey: AI*. URL: https://survey.stackoverflow.co/2025/ai (besucht am 28. 07. 2026).

[39] Leonardo Passos u. a. „Static Architecture-Conformance Checking: An Illustrative Overview“. In: *IEEE Software* 27.5 (1. Sep. 2010), S. 82–89. ISSN: 0740-7459. DOI: 10.1109/MS.2009.117. URL: https://doi.org/10.1109/MS.2009.117 (besucht am 28. 07. 2026).

[40] Dewayne E. Perry und Alexander L. Wolf. „Foundations for the Study of Software Architecture“. In: *ACM SIGSOFT Software Engineering Notes* 17.4 (1. Okt. 1992), S. 40–52. ISSN: 0163-5948. DOI: 10.1145/141874.141884. URL: https://dl.acm.org/doi/10.1145/141874.141884 (besucht am 30. 07. 2026).

[41] *PR-Agent*. The PR Agent, 28. Juli 2026. URL: https://github.com/The-PR-Agent/pr-agent (besucht am 28. 07. 2026).

[42] Nils Reimers und Iryna Gurevych. „Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks“. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. EMNLP-IJCNLP 2019. Hrsg. von Kentaro Inui u. a. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, S. 3982–3992. DOI: 10.18653/v1/D19-1410. URL: https://aclanthology.org/D19-1410/ (besucht am 28. 07. 2026).

[43] Peter C. Rigby und Christian Bird. „Convergent Contemporary Software Peer Review Practices“. In: *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. ESEC/FSE 2013. New York, NY, USA: Association for Computing Machinery, 18. Aug. 2013, S. 202–212. ISBN: 978-1-4503-2237-9. DOI: 10.1145/2491411.2491444. URL: https://dl.acm.org/doi/10.1145/2491411.2491444 (besucht am 28. 07. 2026).

[44] Caitlin Sadowski u. a. „Modern Code Review: A Case Study at Google“. In: *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*. ICSE-SEIP '18. New York, NY, USA: Association for Computing Machinery, 27. Mai 2018, S. 181–190. ISBN: 978-1-4503-5659-6. DOI: 10.1145/3183519.3183525. URL: https://dl.acm.org/doi/10.1145/3183519.3183525 (besucht am 28. 07. 2026).

[45] Timo Schick u. a. *Toolformer: Language Models Can Teach Themselves to Use Tools*. 9. Feb. 2023. DOI: 10.48550/arXiv.2302.04761. arXiv: 2302.04761 [cs.CL]. URL: http://arxiv.org/abs/2302.04761 (besucht am 30. 07. 2026). Vorveröffentlichung.

[46] Mrinank Sharma u. a. *Towards Understanding Sycophancy in Language Models*. 10. Mai 2025. DOI: 10.48550/arXiv.2310.13548. arXiv: 2310.13548 [cs.CL]. URL: http://arxiv.org/abs/2310.13548 (besucht am 30. 07. 2026). Vorveröffentlichung.

[47] Freda Shi u. a. *Large Language Models Can Be Easily Distracted by Irrelevant Context*. 6. Juni 2023. DOI: 10.48550/arXiv.2302.00093. arXiv: 2302.00093 [cs.CL]. URL: http://arxiv.org/abs/2302.00093 (besucht am 07. 07. 2026). Vorveröffentlichung.

[48] Charlie Snell u. a. *Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters*. 6. Aug. 2024. DOI: 10.48550/arXiv.2408.03314. arXiv: 2408.03314 [cs.LG]. URL: http://arxiv.org/abs/2408.03314 (besucht am 30. 07. 2026). Vorveröffentlichung.

[49] Ruoyu Su u. a. *Evaluating Large Language Models for Detecting Architectural Decision Violations*. 7. Feb. 2026. DOI: 10.48550/arXiv.2602.07609. arXiv: 2602.07609 [cs.SE]. URL: http://arxiv.org/abs/2602.07609 (besucht am 29. 06. 2026). Vorveröffentlichung.

[50] Tao Sun u. a. *BitsAI-CR: Automated Code Review via LLM in Practice*. 25. Jan. 2025. DOI: 10.48550/arXiv.2501.15134. arXiv: 2501.15134 [cs.SE]. URL: http://arxiv.org/abs/2501.15134 (besucht am 28. 07. 2026). Vorveröffentlichung.

[51] Xunzhu Tang u. a. *CodeAgent: Autonomous Communicative Agents for Code Review*. 24. Sep. 2024. DOI: 10.48550/arXiv.2402.02172. arXiv: 2402.02172 [cs.SE]. URL: http://arxiv.org/abs/2402.02172 (besucht am 28. 07. 2026). Vorveröffentlichung.

[52] Kla Tantithamthavorn u. a. „RovoDev Code Reviewer: A Large-Scale Online Evaluation of LLM-based Code Review Automation at Atlassian“. In: *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. 12. Apr. 2026, S. 34–45. DOI: 10.1145/3786583.3786851. arXiv: 2601.01129 [cs.SE]. URL: http://arxiv.org/abs/2601.01129 (besucht am 28. 07. 2026).

[53] Theo Theunissen, Stijn Hoppenbrouwers und Sietse Overbeek. „Approaches for Documentation in Continuous Software Development“. In: *Complex Systems Informatics and Modeling Quarterly* 32 (28. Okt. 2022), S. 1–27. ISSN: 2255-9922. DOI: 10.7250/csimq.2022-32.01. URL: https://csimq-journals.rtu.lv/csimq/article/view/csimq.2022-32.01 (besucht am 07. 07. 2026).

[54] Ashish Vaswani u. a. *Attention Is All You Need*. 2. Aug. 2023. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs.CL]. URL: http://arxiv.org/abs/1706.03762 (besucht am 30. 07. 2026). Vorveröffentlichung.

[55] *Webhooks*. GitLab Docs. URL: https://docs.gitlab.com/user/project/integrations/webhooks/ (besucht am 12. 06. 2026).

[56] Jason Wei u. a. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 10. Jan. 2023. DOI: 10.48550/arXiv.2201.11903. arXiv: 2201.11903 [cs.CL]. URL: http://arxiv.org/abs/2201.11903 (besucht am 30. 07. 2026). Vorveröffentlichung.

[57] *What Is a DevOps Platform?* GitLab. URL: https://about.gitlab.com/topics/devops-platform/ (besucht am 12. 06. 2026).

[58] *What Is GitLab Flow?* GitLab. URL: https://about.gitlab.com/topics/version-control/what-is-gitlab-flow/ (besucht am 12. 06. 2026).

[59] Shunyu Yao u. a. *ReAct: Synergizing Reasoning and Acting in Language Models*. 10. März 2023. DOI: 10.48550/arXiv.2210.03629. arXiv: 2210.03629 [cs.CL]. URL: http://arxiv.org/abs/2210.03629 (besucht am 28. 07. 2026). Vorveröffentlichung.

[60] Yuntong Zhang u. a. *Code Review Agent Benchmark*. 7. Apr. 2026. DOI: 10.48550/arXiv.2603.23448. arXiv: 2603.23448 [cs.SE]. URL: http://arxiv.org/abs/2603.23448 (besucht am 28. 07. 2026). Vorveröffentlichung.

[61] Yu Zhao u. a. *Marco-O1: Towards Open Reasoning Models for Open-Ended Solutions*. 25. Nov. 2024. DOI: 10.48550/arXiv.2411.14405. arXiv: 2411.14405 [cs.CL]. URL: http://arxiv.org/abs/2411.14405 (besucht am 28. 07. 2026). Vorveröffentlichung.

[62] Lianmin Zheng u. a. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. 24. Dez. 2023. DOI: 10.48550/arXiv.2306.05685. arXiv: 2306.05685 [cs.CL]. URL: http://arxiv.org/abs/2306.05685 (besucht am 30. 07. 2026). Vorveröffentlichung.

[63] Suzhen Zhong u. a. *Human-AI Synergy in Agentic Code Review*. 16. März 2026. DOI: 10.48550/arXiv.2603.15911. arXiv: 2603.15911 [cs.SE]. URL: http://arxiv.org/abs/2603.15911 (besucht am 06. 07. 2026). Vorveröffentlichung.

A. Anhang

TODO: jaccard matrix zu den ähnlichkeiten der modelle TODO: validator ergebnisse

A.1. Beispiel eines Architecture Decision Record

Das folgende ADR im MADR-Format veranschaulicht den in Abschnitt 2.4 beschriebenen Aufbau vollständig. Es dient ausschließlich der Illustration und stammt nicht aus dem in Kapitel 6 ausgewerteten Datensatz. Die Entscheidung ist bewusst so formuliert, dass sie sich am Quellcode überprüfen lässt: Sie benennt ein konkretes Verzeichnis, eine verbotene Aufrufform und eine erwartete Rückgabestruktur. Im Repository liegt die Datei unter `docs/adr/0005-externe-api-aufrufe.md`.

```
# ADR-0005: Externe API-Aufrufe nur über die Service-Schicht

## Status

accepted (2026-02-11) -- ersetzt ADR-0002 (superseded)

## Kontext

Die Anwendung bindet mehrere fremde HTTP-Schnittstellen ein
(Zahlungsdienst, Geokodierung, Benachrichtigungsversand).
Bisher rufen sowohl UI-Komponenten als auch Hintergrundjobs diese
Dienste unmittelbar auf. Zeitlimits, Wiederholungsversuche und die
Fehlerbehandlung sind dadurch an jeder Aufrufstelle unterschiedlich
umgesetzt. Ein Anbieterwechsel würde Änderungen an vielen über das
Projekt verteilten Stellen erfordern, und in Tests lässt sich der
Zugriff nach außen nur schwer ersetzen.

## Entscheidung

Wir werden alle Aufrufe externer Dienste ausschließlich in
Modulen unterhalb von ‘services/‘ kapseln.

- UI-Komponenten und Hintergrundjobs greifen nur über die dort
  exportierten Funktionen auf externe Dienste zu.
- Ein direkter Aufruf von ‘fetch‘ oder eines HTTP-Clients außerhalb
  von ‘services/‘ ist nicht zulässig.
- Jede Service-Funktion setzt ein Zeitlimit und gibt Fehler als
  typisiertes Ergebnisobjekt zurück, nicht als geworfene Ausnahme.

## Konsequenzen

+ Zeitlimits und Fehlerbehandlung sind an einer Stelle definiert.
+ Ein Anbieterwechsel betrifft nur die Service-Schicht.
+ Externe Aufrufe sind in Tests zentral ersetzbar.
- Für jeden neuen Endpunkt entsteht eine zusätzliche Indirektion.
```

```
- Bestehende direkte Aufrufe müssen migriert werden (siehe #412).
```

Listing A.1: Beispielhaftes ADR im MADR-Format.

A.2. Details der Reproduktion der Originalpipeline

Die folgenden Punkte betreffen die Nachbildung des Originalverfahrens. Sie sind für den Vergleich der beiden Verfahren nicht maßgeblich, weil beide auf demselben Codestand und demselben Suchraum arbeiten, dokumentieren aber, worauf die Baseline beruht.

A.2.1. Das Original-Judge-Modell

Aus der Dokumentation des Original-Repositories geht hervor, dass der Original-Judge mit **Marco-o1** betrieben wurde [61]. Dabei handelt es sich um ein dediziertes Reasoning-Modell mit rund sieben Milliarden Parametern auf Basis von Qwen2, das gezielt auf Chain-of-Thought-Reasoning trainiert (siehe Abschnitt 2.2) und im Original lokal über vLLM auf einem A100-GPU-Cluster gehostet wurde. Der Dateiname `judge_marco.csv` verweist unmittelbar auf dieses Modell.
Über den in dieser Arbeit genutzten LLM-Gateway ist dieses Modell nicht verfügbar und konnte für keinen der Vergleiche reproduziert werden. Sein rohes Urteil (*clf1*) liegt jedoch als Datei vor und geht deshalb als eigener Referenzpunkt in den Vergleich ein (Abschnitt 6.2).
Abzugrenzen sind hiervon die drei *Validator*-Modelle des Originals (Mistral Nemo, Llama-3.1-8B-Instruct und Qwen3-14B-Base). Sie prüfen in einem nachgelagerten Schritt lediglich die Plausibilität des Judge-Urteils und sind nicht mit dem Judge-Modell zu verwechseln. Welche Rolle sie bei der Entstehung von y_{true} spielen, ist Gegenstand von Anhang A.2.3.

A.2.2. Der berichtete Baseline-Wert ist verifizierter Originaloutput

Die Original-Pipeline erreicht auf den 305 Fällen eine Genauigkeit von **91,15 %** (278/305) gegenüber y_{true}. Diese Zahl ist weder eine Nachrechnung noch eine Schätzung: Im unberührten Original-Repository liegt mit `judge_marco.csv` der rohe Output des tatsächlichen historischen Judge-Laufs vor (1471 Zeilen, 109 Repositories). Ein stichprobenartiger Abgleich dieser Datei mit der Urteilsspalte des 305er-Datensatzes ergab eine Übereinstimmung von 100 % (40 von 40 geprüften Fällen). Der Wert ist damit ein belastbarer Fixpunkt und kein Cache-Artefakt. Was er tatsächlich misst, klärt der folgende Abschnitt.

A.2.3. Warum y_{true} von den bewerteten Modellen abhängt

Die Originalarbeit berichtet 91,15 % Accuracy, die hier nachgebildete Baseline erreicht gegen denselben Labelsatz mit `gpt-5.4-mini` dagegen nur rund 45 %, in zwei Läufen über dieselben 305 Fälle 44,2 und 46,1 %. Dass die Nachbildung fehlerhaft sein könnte, schließt Abschnitt **??** aus. Die Lücke hat eine andere Ursache, und sie ist zugleich der Grund, warum y_{true} in dieser Arbeit nicht als Maßstab dient.
Rekonstruiert man die Datenkette aus den Skripten und Zwischendateien des Replication-Packages, ergeben sich vier aufeinander aufbauende Stufen.

1. **Marco-o1 urteilt** über 1471 Regeln aus 109 Repositories. Das Ergebnis ist die Spalte *clf1*.

2. **Drei weitere Modelle prüfen** dieses Urteil (Mistral, Qwen, Llama). Ihr Prompt enthält Marco-o1s Urteil *und dessen Begründung* zusammen mit der Frage, ob sie zustimmen. Stimmen sie zu, ist ihre Antwort per Konstruktion identisch mit der von Marco-o1, was sich in 100 % der Ja-Fälle verifizieren lässt. Über die 305 hier verwendeten Fälle trägt die Spalte *Verify_ Judgement* 258 Zustimmungen für Mistral, 289 für Qwen und 259 für Llama, also zwischen 85 und 95 %. Ergebnis sind die Spalten *clf2* bis *clf4*.

3. **Ein Mensch validiert 316 Regeln.** Er sieht alle vier Modellantworten und trägt in der Spalte *Human Validate* ein, *welchen Modellen* er zustimmt. Er urteilt also nicht frei, sondern wählt aus vier vorgelegten Antworten aus.

4. y_{true} **entsteht als die Antwort der gewählten Modelle** und wird in 305 der 316 Fälle unverändert übernommen.

Was diese Konstruktion anrichtet, lässt sich beziffern (Tabelle A.1).

Tabelle A.1.: Kennzahlen zur Abhängigkeit von y_{true} von den bewerteten Modellausgaben.

Beobachtung	Wert
Fälle, in denen y_{true} mit mindestens einer der vier Modellantworten übereinstimmt	301 von 305
Fälle mit einem Label, das kein Modell geliefert hat	4
Accuracy eines einfachen Mehrheitsentscheids von *clf1*–*clf4* gegen y_{true}	91,8 %
Spannweite der vier Einzelklassifikatoren gegen y_{true}	82,95–91,15 %
Fälle, die der Mensch mit „Remove from consideration“ oder „None“ markierte und die dennoch im Datensatz verblieben (pauschal als *CIA* gelabelt)	8

Der Mehrheitsentscheid der vier Klassifikatoren erreicht 91,8 % und liegt damit praktisch auf den berichteten 91,15 %. Dass zudem alle vier Klassifikatoren eng zwischen 82,95 und 91,15 % beieinanderliegen, wäre für vier voneinander unabhängige Verfahren unplausibel hoch.
Die 91,15 % sind also eine Bestätigungsquote und keine Accuracy. Gemessen wird, wie oft ein Modell die Kalibrierung von Marco-o1 reproduziert, und nicht, wie oft es richtig liegt. Jedes Verfahren, dessen Antworten in Stufe 3 nie zur Auswahl standen, wird von diesem Maßstab strukturell benachteiligt. Für den Vergleich zweier Judges taugt er deshalb nicht. Werte, die dennoch gegen ihn gemessen wurden, gehen in keine Aussage dieser Arbeit ein. Zwei Beobachtungen aus dem Hauptvergleich (Abschnitt 6.2) bestätigen das. Gegen die unabhängige Referenz erreicht Marco-o1, dessen Ausgaben die Grundlage der berichteten 91,15 % bilden, nur noch 71,7 %. Und y_{true} selbst kommt auf 66,7 % und liegt damit fünf Prozentpunkte unter der Modellantwort, die es korrigieren sollte. Erklären lässt sich das über Stufe 3 der Datenkette, denn der Mensch übernahm überwiegend eine der vier vorgelegten Antworten, statt frei zu urteilen, und traf diese Auswahl auf derselben schmalen Evidenzbasis von fünf Codeausschnitten. Eine Auswahl aus vorgelegten Antworten kann aber nicht besser sein als die beste vorgelegte Antwort, wohl aber schlechter.

A.3. Ergänzende Auswertungen zum Hauptvergleich

Dieser Anhang enthält Auswertungen rund um den Hauptvergleich, die Kapitel 6 nur zusammenfasst oder gar nicht führt, weil sie zu keinem der fünf Experimente gehören.

A.3.1. Was die unabhängige Referenz enthält

Wie die Referenz erhoben wurde, beschreibt Abschnitt 5.3. Über alle 81 annotierten Fälle verteilen sich die Label auf 44 *C*, 7 *NC* und 30 *CIA*, bei einer Konfidenzeinschätzung von 64 hoch, 14 mittel und 3 niedrig. Für den maßgeblichen Zufallsteil ($n = 60$) ergeben sich 36 *C*, 5 *NC* und 19 *CIA* bei 45 hoch, 12 mittel und 3 niedrig eingeschätzten Fällen.
Stellt man dem die Verteilung von y_{true} über alle 305 Fälle gegenüber (58,7 % *C*, 21,3 % *NC*, 20,0 % *CIA*), zeigt sich eine deutliche Verschiebung. Die unabhängige Annotation vergibt mit 37 % fast doppelt so oft *CIA*. Das ist der auffälligste Unterschied zwischen beiden Referenzen und verlangt eine Erklärung.

A.3.2. Zusammensetzung der *CIA*-Klasse

Diese Erklärung liefert eine Auswertung der 30 Begründungen, mit denen *CIA* vergeben wurde (Tabelle A.2).

Tabelle A.2.: Ursachen der *CIA*-Vergabe in der unabhängigen Referenz.

Ursache	Anzahl
Evidenz existiert im Repository, liegt aber außerhalb der Whitelist (CI-YAML, `CMakeLists.txt`, `pom.xml`, `.eslintrc`, Markdown)	**18**
Grundsätzlich keine Code-Frage (Kosten, Prozess, Absichtserklärung, Git-Operationen)	10
Nicht eindeutig zuordenbar	2
Summe	30

60 % der *CIA*-Fälle gehen damit auf das Benchmark-Design zurück und nicht auf die Grenzen der Modellfähigkeit. Viele ADR-Regeln betreffen CI-Pipelines, Build-Konfiguration oder Dokumentation, also genau jene Dateitypen, die der Original-Indexer wegen seiner Endungs-Whitelist nie erfasst hat (Abschnitt 5.1.3). Die Aufgabe ist in diesen Fällen auf der gegebenen Evidenzbasis nicht lösbar, gleichgültig wie gut ein Modell sucht oder liest. Der Original-Prompt verlangt trotzdem ein *C*- oder *NC*-Urteil.
Die Werkzeugprotokolle belegen das unabhängig von der Annotation. Der agentische Judge versucht im Laufe der Vergleichsläufe zehnmal, eine Datei zu öffnen, die außerhalb des zugelassenen Suchraums liegt, und wird jedes Mal abgewiesen. Beim `nano`-Modell sind es sechs solcher Versuche. Er erkennt also, wo der Beleg läge, und kommt nicht heran. Was hier gemessen wird, ist an diesen Stellen die Grenze des Suchraums und nicht die Grenze des Verfahrens.

A.3.3. Suchproduktivität der agentischen Läufe

Tabelle A.3.: Suchproduktivität und abgewiesene Zugriffe aus den Event-Logs beider agentischer Läufe.

Kennzahl	`gpt-5.4-mini`	`gpt-5.4-nano`
`grep`-Aufrufe ohne Treffer	190 von 567 (33,5 %)	263 von 659 (39,9 %)
Trefferzeilen je Fall (Median)	153,7 (119)	94,4 (61)
Getroffene Dateien je Fall	48,9	35,9
Ergebnisgröße je `grep`	8920 Zeichen	6424 Zeichen
`read_file` abgewiesen, außerhalb des Suchraums	10	6
`read_file` abgewiesen, Datei nicht gefunden	5	6
Erzwungene Endurteile bei erschöpftem Budget	72 (0,89 je Fall)	–

A.3.4. Aufwand je geprüfter Regel

Abschnitt 2.6 verlangt, den Aufwand mitzuerheben, den ein Urteil verursacht.

Tabelle A.4.: Aufwand je geprüfter Regel, gemittelt über die 81 annotierten Fälle.

Lauf	Tokens gesamt	davon Reasoning	LLM-Aufrufe	Laufzeit
Statisch, `mini`	2456	459	1,00	4,0 s
Agentisch, `mini`, min-Budget	25 885	3973	5,37	26,8 s
Agentisch, `mini`, Standard	**126 375**	6891	**12,86**	**73,2 s**
Agentisch, `mini`, max-Budget	150 032	7757	13,57	165,6 s
Statisch, `nano`	2205	178	1,00	4,4 s
Agentisch, `nano`	127 576	6089	15,15	100,3 s

Vier Sekunden gegenüber gut einer Minute je Regel entscheidet darüber, ob ein Prüflauf noch in eine Pipeline passt (NFA-7). Auffällig ist dabei, dass der Sprung vom min- zum Standardbudget die Tokenmenge verfünffacht, während die Zahl der Modellaufrufe sich nur gut verdoppelt. Die Kosten wachsen also nicht mit der Zahl der Schritte, sondern mit der Menge des gelesenen Materials, das in jedem weiteren Aufruf erneut im Kontext steht.

A.3.5. Robustheit gegen die Wahl der Auswertungsbasis

Der gepaarte Vergleich wurde auf drei Basen gerechnet, nämlich auf dem Zufallsteil, auf allen 81 Fällen und auf den 45 mit hoher Konfidenz annotierten. Der Vorsprung hält auf allen dreien, mit 20,4, 18,7 und 20,5 Prozentpunkten, und er bleibt überall signifikant (Tabelle A.6). Die vollen 81 Fälle sind dabei der Härtetest in die ungünstige Richtung, weil sie alle 27 vom Original überstimmten Fälle enthalten (Abschnitt 5.3.1).
Jedes Label der Referenz trägt eine Konfidenzeinschätzung (Abschnitt 5.3.2). Beschränkt man die Auswertung auf die 45 Fälle hoher Konfidenz, ändert sich am Bild nichts (Tabelle A.5).

Tabelle A.5.: Genauigkeit über alle Fälle des Zufallsteils gegenüber der Teilmenge, deren Label mit hoher Konfidenz vergeben wurde.

Verfahren	alle 60 Fälle	nur hohe Konfidenz ($n = 45$)
Statisch, Original-Prompt	55,9 %	56,8 %
Statisch, permissiv	65,0 %	77,8 %
Agentisch, `mini`	**76,7 %**	**77,8 %**
Statisch, `nano`	42,4 %	43,2 %
Agentisch, `nano`	**68,3 %**	**66,7 %**

Kein Verfahren verschiebt sich um mehr als 1,6 Punkte, mit einer Ausnahme. Der permissive statische Prompt gewinnt 12,8 Punkte und zieht mit dem agentischen Judge gleich. Das passt zu seinem Profil, denn er vergibt *CIA* großzügig und profitiert davon, wenn die schwierigen Fälle wegfallen.

Tabelle A.6.: Gepaarter Vergleich beider Verfahren über die drei möglichen Auswertungsbasen, jeweils Lauf 1. Der maßgebliche Test über alle drei Läufe steht in Tabelle 6.5.

Basis	n	Statisch	Agentisch	nur stat.	nur agent.	McNemar
Zufallsteil	59	55,9 %	**76,3 %**	2	14	$p = 0{,}0042$
alle 81 Fälle	80	53,8 %	**72,5 %**	8	23	$p = 0{,}0107$
hohe Konfidenz	44	56,8 %	**77,3 %**	1	10	$p = 0{,}0117$

A.3.6. Konfusionsmatrizen der weiteren Modellstufen

Die folgenden Matrizen ergänzen die Modellrotation aus Abschnitt 6.7. Alle Werte stammen aus Lauf 1 der jeweiligen Bedingung. Die beiden ersten zeigen, woran das schwächere Modell in der statischen Bedingung scheitert.

Tabelle A.7.: Statisch, `gpt-5.4-nano` (Accuracy 42,4 %, $n = 59$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	19	8	8	54,3 %
NC	1	3	1	60,0 %
CIA	1	**15**	3	**15,8 %**
Precision	90,5 %	**11,5 %**	25,0 %	

Tabelle A.8.: Agentisch, `gpt-5.4-nano` (Accuracy 68,3 %, $n = 60$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	28	5	3	77,8 %
NC	2	3	0	60,0 %
CIA	2	7	10	**52,6 %**
Precision	87,5 %	20,0 %	**76,9 %**	

Von 19 tatsächlich unentscheidbaren Fällen labelt der statische `nano`-Lauf ganze 15 als *NC*. Der *CIA*-Recall bricht dadurch auf 15,8 % ein und die *NC*-Precision auf 11,5 %. Mit

Werkzeugzugang verschwindet dieses Verhalten wieder, denn der *CIA*-Recall steigt auf 52,6 % und die *CIA*-Precision von 25,0 auf 76,9 %.
Auch das Suchmuster aus Abschnitt 6.3.1 reproduziert sich (Tabelle A.9). Bei *CIA*-Urteilen laufen 4,08 greps leer gegenüber 1,69 bei *C*, und es werden weniger Dateien geöffnet, nämlich 1,85 gegenüber 3,44.

Tabelle A.9.: Mittlere Werkzeugnutzung des agentischen Judge mit `gpt-5.4-nano`, gruppiert nach gewähltem Label.

Gewählt	n	`list_files`	`grep`	davon leer	`read_file`	Tools ges.
C	32	0,88	6,69	**1,69**	**3,44**	11,00
NC	15	1,53	11,13	5,93	2,07	14,73
CIA	13	1,08	8,31	**4,08**	1,85	11,23

Dritte Modellstufe `gpt-5.6-luna`**.** Für dieses Modell liegt nur ein Lauf je Bedingung vor. Die beiden Matrizen zeigen dasselbe Muster wie bei `nano`, nur noch ausgeprägter: Der statische Lauf schiebt 14 der 19 unentscheidbaren Fälle nach *NC* und erreicht dort eine Precision von 14,3 %; mit Werkzeugzugang steigt die *CIA*-Precision auf 100 %.

Tabelle A.10.: Statisch, `gpt-5.6-luna` (Accuracy 48,3 %, $n = 60$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	23	10	3	63,9 %
NC	1	4	0	80,0 %
CIA	3	**14**	2	**10,5 %**
Precision	85,2 %	**14,3 %**	40,0 %	

Tabelle A.11.: Agentisch, `gpt-5.6-luna` (Accuracy 75,0 %, $n = 60$).

wahr \ vorhergesagt	*C*	*NC*	*CIA*	Recall
C	31	5	0	86,1 %
NC	2	3	0	60,0 %
CIA	5	3	11	57,9 %
Precision	81,6 %	27,3 %	**100,0 %**	

Auch die Werkzeugnutzung passt ins Bild. Der agentische luna-Lauf sammelt mit 223 Trefferzeilen je Fall mehr Evidenz ein als `gpt-5.4-mini` mit 157, bei 6,92 `grep`-Aufrufen und 2,57 geöffneten Dateien. Dass dasselbe Modell in der statischen Bedingung das schwächste der drei ist, obwohl es das größte Kontextfenster besitzt, ist der deutlichste Beleg dafür, dass die Fenstergröße den Evidenzzugang nicht ersetzt.
In den Streitfällen der Richtung 2 trennt die Evidenzlage beim `nano`-Lauf klarer als bei `gpt-5.4-mini` (Abschnitt 6.3.3). Von zehn Fällen beurteilt der Agent acht richtig. Wo er recht hat, laufen 6,00 greps leer bei 0,50 geöffneten Dateien, wo er unrecht hat, nur 1,00 leerer grep bei 5,00 geöffneten Dateien. Er liegt also genau dann richtig, wenn er gesucht und nichts gefunden hat.

A.3.7. Abstand zwischen makro-gemitteltem und gewichtetem F_1

Abschnitt 2.6 führt diesen Abstand als eigene Diagnosegröße ein. Ist er groß, wird die Gesamtleistung überwiegend von der häufigsten Klasse getragen.

Tabelle A.12.: Abstand zwischen makro-gemitteltem und gewichtetem F_1-Wert. Ein kleiner Abstand bedeutet, dass die Leistung nicht überwiegend von der häufigsten Klasse getragen wird.

Verfahren	F_1 (Makro)	F_1 (gewichtet)	Abstand
Statisch, Original-Prompt	0,493	0,596	+0,103
Statisch, permissiv	0,627	0,651	**+0,024**
Agentisch, `mini`	0,676	0,771	+0,096
Statisch, `nano`	0,355	0,481	+0,126
Agentisch, `nano`	0,583	0,717	+0,134

Den kleinsten Wert erreicht der permissive statische Prompt mit 0,024, und zwar obwohl er in der Gesamtgenauigkeit elf Punkte hinter dem agentischen Judge liegt. Der agentische Judge liegt mit 0,096 im Mittelfeld und beim schwächeren Modell mit 0,134 am oberen Ende. Ausgewogen ist er also im *Recall* über die Klassen, nicht in der Kombination aus Recall und Precision. Wer eine über die Klassen gleichmäßige Leistung sucht und dafür Genauigkeit einzutauschen bereit ist, fährt mit dem permissiven Prompt besser.